

The Blueprint (Full Book 5)

August 30, 2026

THE BLUEPRINT

Book 5: मोठं Software कसं बांधतात

(System design + elite शिस्त. Devanagari मराठी + English tech शब्द. तीन भाग एका खंडात: सोयीनं छापण्यासाठी.)

अनुक्रम

PART 1: प्रवास: एक User ते कोटी

- 1.1 एक Server, एक स्वप्न
- 1.2 पहिली भिंत: Database चा जन्म ...
- 1.3 Index: गोदामाची वही
- 1.4 Cache: हातासरशी उत्तर, आणि तू...
- 1.5 Replication: प्रती आणि त्यांचं...
- 1.6 Sharding: वहीचे तुकडे आणि Hot...
- 1.7 Queue: रांगा, आणि "पुन्हा-पुनः"
- 1.8 Load Balancer: वाहतूक-पोलिस
- 1.9 CDN: अंतर कापणारं जाळं
- 1.10 CAP: तीनातले दोनच मिळतात
- 1.11 अपयश हेच सामान्य: Timeouts, ...
- 1.12 दिसणं: Observability आणि p99...
- 1.13 शव-विच्छेदन: ज्या दिवशी जग ब...
- 1.14 Cloud-Bill चं शरीर आणि Egres...
- 1.15 आरतीचं पूर्ण चित्र

PART 2: रचनेचे निर्णय

- 2.1 Data कशी रचायची: Data Modeling
- 2.2 SQL vs NoSQL: गाजलेला वाद [दो...]
- 2.3 Database चं हृदय: B-tree आणि ...
- 2.4 Transactions आणि Race Condi...
- 2.5 API: सेवांमधला करार
- 2.6 Monolith vs Microservices [दो...]
- 2.7 Search आतून: Inverted Index
- 2.8 Real-Time: WhatsApp चं तत्त्व
- 2.9 चालत्या गाडीचं चाक: Migrations
- 2.10 Architecture Diagram वाचण्या...

PART 3: Elite शिस्त + खऱ्या इमारती

- 3.1 Git: इतिहास-यंत्र आणि Review-...
- 3.2 Testing चे थर
- 3.3 CI/CD आणि सुरक्षित Deploy
- 3.4 Incident-संस्कृती: दोष नको, ध...
- 4.1 इमारत: WhatsApp: 50 माणसं, 90...
- 4.2 इमारत: UPI: करार जो देश जोडतो
- 4.3 इमारत: Zerodha: कंटाळवाणं Tec...
- 4.4 इमारत: Netflix: मुद्दाम अपयश ...
- 4.5 AI-युगाची रचना: नजर हीच lever...
- 4.6 अंतिम परीक्षा
- 4.7 पाच हात-नकाशे (टेबलावर चिकटवा)

वाचनाची टीप: SPINE = मुख्य साखळी (क्रमाने वाचा); DEPTH = खोलात (घाई असेल तर नंतर).
आरतीचं किराणा-app पूर्ण पुस्तकभर वाढत जातं; प्रत्येक chapter शेवटी "विचार करा" चं उत्तर पुढच्या chapter चं बीज.

THE BLUEPRINT

Book 5, Part 1: प्रवास: एक User ते कोटी

हे पुस्तक कुणासाठी: Book 1 (The Machine) ने “internet चालतं कसं” सांगितलं. हे पुस्तक पुढचं सांगतं: टिकणारं, कोटी-लोकांचं **software** बांधतात कसं, आणि जगातल्या उत्तम **teams** काम कशा करतात. का महत्वाचं? कारण तुमचं code आता agents लिहितील (Book 4 ने दाखवलं); पण कोणतं बांधायचं, कुठली रचना निवडायची: हा निर्णय अजून तुमचा. आणि तोच खरा फरक: top talent ची ओळख code टाईप करण्याच्या वेगात नाही, **रचनेच्या नजरेत** आहे.

या पुस्तकाचं एकमेव सूत्र

मोठेपणा फुकट नसतो.
प्रत्येक रचना-निर्णय म्हणजे काहीतरी विकून काहीतरी विकत घेणं.
Trade-off न दिसणं हीच खरी चूक.

उलगडून: software मध्ये “सगळ्यात चांगली” रचना नसते; फक्त “या परिस्थितीला योग्य” रचना असते. वेग हवा तर जागा (किंवा पैसा) विका; खात्री हवी तर वेग विका. जो engineer प्रत्येक निर्णयामागचा **सौदा** बघतो तो top; जो “हे भारी tech आहे म्हणून वापरू” म्हणतो तो नवखा. प्रत्येक chapter शेवटी हे सूत्र परत भेटेल.

आरतीचं किराणा-app: या पुस्तकाचं चालतं उदाहरण

आरती. वय 26. पुण्याजवळच्या तालुक्यातली. तिने एक साधं app बनवलं: गावातल्या किराणा-दुकानांना घरपोच ऑर्डर. सुरुवात: एक दुकान, एक गाव, दिवसाला 10 ऑर्डर, एका जुन्या laptop वर चालणारं. Part 1 मध्ये आपण तिच्यासोबत प्रवास करू: 10 ऑर्डर ते **कोटी** ऑर्डर. आणि हा प्रवास **भिंतींचा** आहे: प्रत्येक टप्प्यावर एक नवी भिंत आदळते (app हळू झालं, database बुडाली, दिवाळीला crash झालं), आणि प्रत्येक भिंत एक नवी संकल्पना शिकवते. आरतीच्या जागी स्वतःला ठेवा: तिचा प्रत्येक पेच तुमचा आहे.

कपाटावर एक नजर

Book 1	The Machine	tech चा पाया
Book 2	The Thinking Machine	AI यंत्र
Book 3	The Land Game	जमीन-धंदा
Book 4	The Harness	AI वर धंदा
Book 5	The Blueprint	<- हे: मोठं software कसं बांधतात

तीन भागांचा नकाशा: तुम्ही इथे आहात

PART 1	प्रवास: एक user ते कोटी	<- तुम्ही इथे
	server, database, cache, replication, sharding, queue, load balancer, CDN:	भिंती एकेक
PART 2	रचनेचे निर्णय	
	SQL/NoSQL, monolith/microservices, API, search	
PART 3	Elite शिस्त + खऱ्या इमारती	
	git, testing, incident-संस्कृती, WhatsApp/UPI/ Zerodha/Netflix	उलगडून

Book 1 ची उजळणी: 4 प्रश्न (बोलून; उत्तरं खाली)

1. Server म्हणजे काय? (Book 1)

दुसऱ्यांच्या विनंती ऐकून उत्तर देणारा, सतत चालू असलेला computer.

2. Request-response एका ओळीत?

तुमचा computer विनंती पाठवतो, server उत्तर परत पाठवतो; internet म्हणजे या देवाणघेवाणीचं जाळं.

3. Database का लागते?

माहिती **टिकवायला** आणि पुन्हा शोधायला: वीज गेली तरी राहणारी, रचलेली साठवण.

4. Latency म्हणजे?

विनंती आणि उत्तर यांच्यातला वेळ: अंतर जितकं जास्त, latency तितकी जास्त.

चौघे पक्के? मग आरतीच्या पहिल्या server पासून सुरू करू.

विभाग 1: भिंती एकेक

हा विभाग एक प्रवास आहे, chapters नव्हे. आरतीचं app वाढत जाईल, आणि प्रत्येक टप्प्यावर एक भिंत आदळेल: पहिली विनंती, पहिली database, पहिला हळूपणा, पहिला crash. प्रत्येक भिंत एक संकल्पना आणि एक **सौदा** शिकवेल. **हे तुमच्या धंद्यात कुठे येईल:** तुमचं कुठलंही app/सेवा वाढली की याच भिंती याच क्रमाने येतात: WhatsApp लाही आल्या, UPI लाही. आधी दिसल्या की तयारीने सामोरे जाता; न दिसल्या की crash होऊन शिकता (महाग रस्ता).

Chapter 1.1 [SPINE]: एक Server, एक स्वप्न

आरतीचं app पहिल्या दिवशी एका जुन्या laptop वर चालत होतं. एक दुकान, दहा गिन्हाईकं. आणि तरी त्या छोट्या यंत्रात पुढच्या कोटी-ऑर्डरचं संपूर्ण बीज आहे: कारण एक विनंती जे प्रवास करते, तेच कोटी विनंत्याही करतात: फक्त गर्दी वेगळी. म्हणून आधी **एका विनंतीचं पूर्ण शरीर** बघू.

एका ऑर्डरचा प्रवास. गावातला रमेश app उघडतो, “2 किलो साखर” वर बोट ठेवतो. त्या एका स्पर्शापासून काय-काय घडतं:

1. रमेशचा फोन एक विनंती (request) बनवतो: "साखर, 2 किलो, रमेश, हा पत्ता"
2. ती विनंती internet वरून आरतीच्या server कडे जाते (Book 1: request-response)
3. Server ती वाचतो, विचार करतो (code चालवतो): दुकानात साखर आहे का? भाव किती? रमेशचा पत्ता खरा का?
4. Server database ला विचारतो (माहिती कुठे टिकलीये)
5. उत्तर तयार करतो: "ऑर्डर घेतली, 40 रुपये, अर्ध्या तासात"
6. ते उत्तर परत रमेशच्या फोनवर

सहा पायऱ्या. आणि इथे तीन वेगळी कामं एकाच laptop मध्ये एकत्र नांदत आहेत, हे ओळखा: **ऐकणारा-बोलणारा** (web server: विनंत्या घेतो-उत्तरं देतो), **विचार करणारा** (application: तुमचं code, धंद्याचे नियम), आणि **लक्षात ठेवणारा** (database: माहिती टिकवतो). लहान दुकानाची उपमा: एकच माणूस गल्ल्यावर बसतो, गिन्हाईकाशी बोलतो, हिशेब करतो, **आणि** वही सांभाळतो. चालतं: कारण गिन्हाईकं दहा.

पहिला सौदा (सूत्र सुरू होतं): सोपेपणा वि. वाढ. सगळं एका यंत्रात असणं म्हणजे **सोपं**: एकच जागा, एकच gadget, समजायला सोपं, बांधायला जलद. आरतीने अगदी बरोबर केलं: दहा ऑर्डरसाठी तीन वेगळे computers घेणं म्हणजे मुंगीला हत्तीचा साज. **छोट्या सुरुवातीला “एकत्र सगळं” हीच शहाणी रचना आहे.** पण या सोपेपणाची एक किंमत लपलेली आहे, आणि ती पुढच्या भिंतीवर दिसेल: तो एक laptop म्हणजे **एकच अपयश-बिंदू**: तो बंद पडला की पूर्ण धंदा बंद.

आणि एक महत्त्वाची जोड (Book 1 शी): “server” म्हणजे खास जादूई यंत्र नाही: तो तुमच्यासारखाच computer आहे, फक्त तो **सतत चालू** असतो आणि **दुसऱ्यांच्या विनंत्या ऐकतो**. आरतीचा जुना laptop server आहे कारण त्याने ते **काम** केलं: यंत्रामुळे नाही, भूमिकेमुळे. उद्या तो cloud मधल्या भाड्याच्या यंत्रावर जाईल (Book 1 चं cloud), पण भूमिका तीच राहील.

महाग चूक

”सुरुवातीलाच ‘मोठ्या company सारखं’ बांधू: microservices, अनेक servers, सगळा जामानिमा.” नवशिक्यांची सगळ्यात महाग चूक, आणि तिला नाव आहे: **premature scaling** (वेळेआधी मोठेपणा). दहा ऑर्डरसाठी शंभर-ऑर्डरची गुंतागुंत बांधणारा सहा महिने रचनेतच अडकतो आणि app कधी निघतच नाही. सूत्र उलटं लावा: **आजच्या गर्दीला जेवढं लागतं तेवढंच; भिंत आली की मग पुढंच**. आरती दहा ऑर्डरला एक laptop वापरते: हा भोळेपणा नाही, शहाणपण आहे.

नकाशावर

प्रवासाचा नकाशा (वाढत जाईल):

- 1.1 एक यंत्र, तीन कामं (एक-विचार-लक्षात ठेव); सोपं पण एकच अपयश-बिंदू

सूत्रावर: एका laptop ची रचना सोपेपणा विकत घेते, वाढ विकून. दहा ऑर्डरला हा सौदा फायद्याचा; उद्या तोट्याचा होईल: आणि तेव्हा रचना बदलेल. **कुठलीही रचना कायमची बरोबर नसते; ती त्या-त्या गर्दीला बरोबर असते.**

स्वतः बघा (5 मिनिटं)

तुमच्या रोज वापरातल्या एका app चा विचार करा (WhatsApp, कुठलंही). एक कृती निवडा (message पाठवणं) आणि वरच्या सहा पायऱ्या त्याला लावून बघा: विनंती काय, server काय विचार करत असेल, काय लक्षात ठेवत असेल, उत्तर काय. दिसणारं app साधं; मागचं शरीर हेच.

विचार करा

आरतीला एका रात्री धक्का बसला: laptop चा data चुकून पुसला गेला, आणि आदल्या दिवसाच्या सगळ्या ऑर्डर, दुकानांचे भाव: सगळं गेलं. “माहिती code मध्येच का ठेवत नाही? वेगळी database कशाला?”

कारण code आणि माहिती यांचं आयुष्य वेगळं असतं: code तुम्ही लिहिता-बदलता, माहिती **रोज वाढते आणि टिकावी लागते** (वीज गेली, यंत्र बदललं तरी). दोन्ही एकत्र ठेवणं म्हणजे आरतीची रात्र: पुन्हा-पुन्हा. म्हणून माहितीला स्वतःचं, टिकाऊ, रचलेलं घर लागतं: पहिली खरी भिंत, पुढचा chapter: database चा जन्म.

Chapter 1.2 [SPINE]: पहिली भिंत: Database चा जन्म आणि ACID

आरतीच्या पुसलेल्या-data च्या रात्रीतून पहिली खरी संकल्पना जन्मते. माहितीला वेगळं, टिकाऊ घर हवं: **database**. पण database म्हणजे नुसती “मोठी वही” नाही; ती एका करारावर उभी असते: आणि तो करार समजला की पुढचं अर्ध पुस्तक सोपं होतं.

Database म्हणजे नक्की काय जास्त देते? किराणा-दुकानाची वही आठवा: नुसती यादी. आता बँकेची वही आठवा: तिथे नियम कडक आहेत: प्रत्येक नोंद पूर्ण किंवा मुळीच नाही; एकाच वेळी दोन कारकून एकाच खात्याला हात लावले तर गोंधळ होऊ नये; आणि एकदा नोंदलं की वीज गेली तरी टिकलं पाहिजे. **database ही अशी शिस्तीची वही आहे.** त्या शिस्तीला चार अक्षरांचं नाव: **ACID**. हे चार शब्द पाठ करा; हजारो engineers चं ते समान पाया-ज्ञान आहे:

- A Atomicity (अखंडता): व्यवहार पूर्ण किंवा मुळीच नाही. रमेशचे पैसे कापले पण ऑर्डर नोंदली नाही: असं अर्धवट कधीच नाही. दोन्ही, किंवा काहीच नाही.
- C Consistency (सुसंगती): नियम कधीच मोडत नाहीत. साखर -5 किलो शिल्लक असं अशक्य; नियम database राखते.
- I Isolation (एकांत): दोन ऑर्डर एकाच वेळी आल्या तरी एकमेकांत मिसळत नाहीत; प्रत्येकीला वाटतं ती एकटीच.
- D Durability (टिकाऊपणा): एकदा “झालं” म्हटलं की वीज गेली, यंत्र बंद पडलं तरी नोंद राहते.

आरतीला हे चारही हवे आहेत. साखरेची शिल्लक, रमेशचे पैसे, दुकानाचा हिशेब: इथे “अर्धवट” किंवा “मिसळलेलं” म्हणजे धंदा बुडाला. म्हणून ती एक खरी database वापरते (PostgreSQL: जगात 2026 त developers मध्ये सर्वात लोकप्रिय, Stack Overflow च्या पाहणीत सलग पहिली): नुसती file नाही.

पण इथे सूत्र लपलंय: ACID ची किंमत. ही शिस्त फुकट नाही: प्रत्येक “झालं” म्हणण्याआधी database ला खात्री करावी लागते (खरंच लिहिलं? टिकलं?), एकांत राखायला कुलपं लावावी लागतात. म्हणजे ACID = **खात्री विकत, वेग विकून**. बँकेची वही किराणा-वहीपेक्षा **हळू** असते: कारण ती जास्त **खात्रीशीर** असते. हा सौदा आत्ता (पैसे, शिल्लक) फायद्याचा; पण पुढे आरतीला काही माहिती अशी भेटेल जिथे कडक खात्रीपेक्षा वेग महत्त्वाचा (कोण-किती-वेळा-app-उघडलं सारखी): तिथे हाच सौदा उलटवावा लागेल (Part 2 चा SQL-vs-NoSQL वाद याच फाट्यावर उभा).

खरी जगातली भीषण आठवण (postmortem-चव): database किती नाजूक, हे 2017 त **GitLab** ला महाग कळलं. एका engineer ने रात्री, replication-चा गोंधळ सोडवताना, **चुकीच्या** server वरची database-directory पुसली: ~300 GB. आणि मग कळलं की त्यांच्या **पाच** backup-पद्धतींपैकी एकही

धड चालत नव्हती (backup-files काही bytes च्या, दुसरी रिकामी). शेवटी ~6 तास जुन्या एका snapshot वरून वाचवलं; ~6 तासांचा data (हजारो projects, नोंदी) कायमचा गेला: आणि हे सगळं त्यांनी **थेट YouTube वर live** दुरुस्त केलं. धडा दुहेरी: database हे धंद्याचं हृदय, **आणि** “backup आहे” म्हणणं वेगळं, “backup तपासलेला आहे” म्हणणं वेगळं (Book 4, 4.5 ची तीच शिकवण, इथे रक्ताने). न तपासलेला backup म्हणजे नसलेला backup.

महाग चूक

“माहिती Excel/file मध्येच ठेवू, database कशाला?” छोट्या, न-वाढणाऱ्या, एकट्याच्या यादीला file ठीक. पण जिथे **पैसे, शिल्लक, किंवा एकाच वेळी अनेक लोक** माहिती बदलतात: तिथे file म्हणजे आरतीची पुसली-गेली रात्र आणि त्याहून वाईट: दोन लोकांनी एकाच वेळी लिहिलं तर **गुपचूप** चुकीचा आकडा (isolation नाही). ACID ची गरज “मोठेपणा” वर नाही, **माहितीच्या स्वभावावर** ठरते: पैसा-शिल्लक-नोंद = ACID हवीच.

नकाशावर

- 1.1 एक यंत्र, तीन कामं; एकच अपयश-बिंदू
- 1.2 database = शिस्तीची वही (ACID: अखंड-सुसंगत-एकांत-टिकाऊ); खात्री विकत, वेग विकून

सूत्रावर: ACID हा पहिला मोठा जाणीवपूर्वक सौदा: वेग देऊन खात्री घेतली. Top engineer ला विचारा “ACID हवी का?” तो “कशासाठी?” विचारेला: पैशाला हो, like-count ला कदाचित नाही. सौदा दिसणं = नजर.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनमधल्या दोन गोष्टींचा विचार करा: bank-app मधली शिल्लक, आणि कुठल्याही app मधली “views/likes” ची संख्या. कुठली **अचूक** असलीच पाहिजे (एक रुपयाही चुकून चालणार नाही), आणि कुठली “साधारण बरोबर” चालते? ही रेषा म्हणजे ACID कुठे हवी याची रेषा: तुम्ही ती रोज नकळत अनुभवता.

विचार करा

आरतीचं app वाढलं: 10 वरून 500 ऑर्डर रोज. आणि अचानक app **हळू** झालं: दुकानाची यादी उघडायला 8-10 सेकंद. Database तीच, नियम तेच, मग हळू का? आरती गोंधळली: “माहिती तर आहेच आत, मग शोधायला एवढा वेळ का?”

कारण database माहिती **कशी शोधते** यावर वेग ठरतो, आणि डिफॉल्ट पद्धत म्हणजे “पूर्ण वही पहिल्या पानापासून चाळणं”: 500 नोंदींना ठीक, 5 लाखांना संकट. वही भराभर उघडायला एक जुनी कारकुनी युक्ती लागते: अनुक्रमणिका. पुढची भिंत: index.

Chapter 1.3 [SPINE]: Index: गोदामाची वही

आरतीचं app हळू का झालं? उत्तर एका जुन्या कारकुनी युक्तीत आहे, आणि ती युक्ती समजली की database चा अर्धा वेग-गूढ संपतो.

पूर्ण वही चाळणं वि. अनुक्रमणिका. समजा एका जाड पुस्तकात “साखर” शब्द कुठे आहे शोधायचंय. दोन मार्ग: (1) पहिल्या पानापासून प्रत्येक ओळ वाचत जाणं: 500 पानी पुस्तकाला वेळ लागेल, पण चालेल; 5 लाख पानी पुस्तकाला **आयुष्य** लागेल. (2) मागच्या **अनुक्रमणिकेत** “साखर” बघणं: “पान 213”: थेट तिथे. Database ची डिफॉल्ट पद्धत पहिली आहे (full scan: पूर्ण table चाळणं). आरतीच्या 500 ऑर्डरला ते चालत होतं; 5 लाखांना कोसळलं. **Index म्हणजे database ची अनुक्रमणिका.**

Index कसं काम करतं. आरती database ला सांगते: “रमेशच्या फोन-नंबरवर मी वारंवार शोधते: त्याची अनुक्रमणिका बनव.” Database एक वेगळी, **क्रमाने लावलेली** यादी ठेवते: फोन-नंबर -> “ती नोंद इथे आहे.” आता शोध पूर्ण-चाळणीचा नसतो; अनुक्रमणिकेतला उडी-मारत शोध (5 लाखांत ~19 उड्यांत सापडतं: half-half कापत: Book 6 चा binary search इथे कामाला येईल). 8 सेकंदांचं काम काही milliseconds वर. आरतीचं app पुन्हा भुरकन.

पण सूत्र इथेही हजर: index ची तिहेरी किंमत. फुकट वेग नाहीच: (1) **जागा:** अनुक्रमणिका ही वेगळी यादी: database मोठी होते (पुस्तकाच्या मागची 50 पानी index पुस्तक जाड करते). (2) **लिहिण्याचा वेग:** आता प्रत्येक नव्या ऑर्डरला **दोन** जागी लिहावं लागतं: मूळ नोंद आणि अनुक्रमणिका (नवा शब्द आला की index पुस्तकातही भर घालावी लागते). म्हणजे index वाचन जलद करते पण लेखन हळू. (3) **सांभाळ:** प्रत्येक index राखावा लागतो. निष्कर्ष: ****index = वाचन-वेग विकत, लेखन-वेग**

- जागा विकून.**

आणि इथे नवख्याची खरी परीक्षा: “मग सगळ्या स्तंभांवर index टाकू, सगळं जलद!” चूक. वाचनाला जलद, पण **प्रत्येक** लेखन तेवढ्याच index मध्ये भर घालत बसतं: 15 index असलेली table मध्ये एक ऑर्डर लिहायला 15 अनुक्रमणिका update: लेखन कासवगती. Top engineer index टाकतो **जिथे वारंवार शोध होतो तिथेच:** रमेशचा नंबर, ऑर्डरची तारीख: हो; कचित शोधले जाणारे स्तंभ: नको. **कुठे index, कुठे नाही:** हा नजरेचा निर्णय आहे, आणि तो सौदा बघण्यातून येतो.

महाग चूक

”App हळू आहे = मोठा server घेऊ (जास्त पैसे).” आरतीने जवळपास हीच चूक केली: हळूपणा बघून महागडा server भाड्याने घ्यायला निघाली. पण खरी अडचण server ची ताकद नव्हती, **गहाळ index** होती: एका ओळीच्या index ने 10x महागड्या server पेक्षा जास्त वेग दिला, फुकट. **हळूपणाचं पहिलं निदान “मोठा server” नसून “कुठली शोध-रचना” हे असतं.** पैसे ओतण्याआधी नजर टाका: software मधल्या 90% हळूपणाचं मूळ रचनेत असतं, ताकदीत नाही.

नकाशावर

- 1.1 एक यंत्र, तीन कामं; एकच अपयश-बिंदू
- 1.2 database = शिस्तीची वही (ACID); खात्री विकत, वेग विकून
- 1.3 index = अनुक्रमणिका; वाचन जलद, लेखन+जागा विकून;
जिथे शोध तिथेच

सूत्रावर: index हा वाचन-वि.-लेखन चा शुद्ध सौदा: तुमचं app जास्त काय करतं (वाचतं की लिहितं) यावर निर्णय. सौदा न बघता “सगळीकडे index” म्हणणारा नवखा; “इथे वाचन जास्त म्हणून index” म्हणणारा जाणकार.

स्वतः बघा (5 मिनिटं)

कुठलंही पाठ्यपुस्तक उघडा आणि मागची अनुक्रमणिका/index बघा. एक शब्द निवडा, index वापरून शोधा (2 सेकंद), मग तोच शब्द पहिल्या पानापासून चाळत शोधायचा कल्पना करा (मिनिटं). हाच फरक database मध्ये milliseconds वि. सेकंद. आणि लक्षात घ्या: त्या index पानांनी पुस्तक जाड केलंय: तीच जागेची किंमत.

विचार करा

आरतीचं app आणखी वाढलं: दिवाळी आली, एका दिवसात 50,000 ऑर्डर. Index असूनही database धापा टाकू लागली: कारण हजारो लोक **एकाच वेळी** तीच लोकप्रिय माहिती (आजचे भाव, सणाच्या ऑफर) पुन्हा-पुन्हा विचारत होते. Database प्रत्येक वेळी तीच गोष्ट नव्याने शोधून देत होती. आरती विचारते: “एकच उत्तर हजार वेळा तेच असेल, तर दर वेळी नव्याने का शोधायचं?”

बरोबर पकडलंस. तेच-तेच उत्तर हातासरशी ठेवायची युक्ती म्हणजे cache: आणि ती software मधली सगळ्यात शक्तिशाली, आणि सगळ्यात **धोकादायक** युक्ती आहे. पुढची भिंत.

Chapter 1.4 [SPINE]: Cache: हातासरशी उत्तर, आणि त्याचं दुखणं

Cache ही software मधली सगळ्यात मोठी वेग-देणारी युक्ती आहे, आणि तिच्याबद्दल एक जगप्रसिद्ध विनोद आहे: ”**Computer science मध्ये फक्त दोनच अवघड गोष्टी आहेत: cache मधून गोष्ट कधी काढायची, आणि गोष्टींना नावं देणं.**” का अवघड, ते आरतीच्या दिवाळीत दिसेल.

Cache म्हणजे काय. दुकानदाराची उपमा: रोज विचारला जाणारा भाव (साखर, तेल) दुकानदार गोदामात जाऊन बघत नाही; तो गल्ल्याजवळ एका कागदावर लिहून ठेवतो: **हातासरशी**. Cache तेच: वारंवार लागणारं उत्तर database मध्ये न जाता एका **जलद, जवळच्या** जागी (memory त) ठेवायचं. आरती आजचे भाव, लोकप्रिय दुकानांच्या याद्या cache मध्ये ठेवते: हजारो लोक विचारले तरी database ला त्रास नाही, उत्तर गल्ल्याच्या कागदावरून. दिवाळीचा भार पेलला. Cache मुळे वाचनं 10-100 पट जलद-स्वस्त होऊ शकतात: म्हणून प्रत्येक मोठं app cache वापरतं.

पण इथे सूत्राचं सगळ्यात कपटी रूप: cache = वेग विकत, ताजेपणा विकून. गल्ल्यावरचा कागद जुना झाला तर? साखरेचा भाव गोदामात बदलला पण कागदावर जुनाच राहिला: दुकानदार चुकीचा भाव सांगतो. Cache मधली माहिती आणि खरी (database मधली) माहिती **वेगळी पडणं**: याला **stale cache** (शिळं cache) म्हणतात, आणि हेच ते “कधी काढायची” चं दुखणं. आरतीच्या app मध्ये हे घडलं: एका दुकानाने साखर संपली म्हणून काढून टाकली, पण cache मध्ये “उपलब्ध” राहिली: रमेशने ऑर्डर केली, दुकानात गेल्यावर “नाही” म्हणाले. एक शिळ्या-cache ची चूक = एक नाराज गिन्हाईक.

म्हणून cache चा खरा प्रश्न: जुनं कधी फेकायचं (invalidation). तीन ढोबळ रीती (प्रत्येकीला सौदा): (1) **वेळ ठरवा (TTL):** “हा भाव 5 मिनिटं ताजा माना, मग नव्याने आणा”: सोपं, पण 5 मिनिटं जुनी माहिती चालेल तरच. (2) **बदललं की फेका:** भाव database मध्ये बदलला की cache मधून लगेच काढा: ताजं, पण गुंतागुंतीचं (प्रत्येक बदलाला cache-ला कळवावं लागतं). (3) **जागा संपली की जुनं-न-वापरलेलं फेका:** cache भरली की सगळ्यात कमी वापरलेलं बाहेर. निवड कशावर? ”**किती जुनी माहिती चालेल**” यावर: भावाला काही मिनिटं चालतं; शिल्लक/पैसे यांना **शून्य** शिळेपणा (तिथे cache टाळाच, किंवा अति-कडक).

नवख्याचा धडा (आरतीने घेतला): cache हे “app हळू झालं की टाकायचं मलम” नाही. **आधी विचारा: ही माहिती किती वेळ जुनी चालेल?** “अजिबात नाही” (पैसे) तर cache नको; “काही मिनिटं चालेल” (भाव, याद्या) तर cache सोनं. हा एक प्रश्न न विचारता cache टाकणारे engineers शिळ्या-माहितीचे bugs वर्षानुवर्ष शोधत बसतात: कारण bug **कधीतरीच** दिसतो (cache ताजं असताना नाही, शिळं असताना), म्हणून पकडणं कठीण.

महाग चूक

"Cache टाकलं, app जलद झालं, काम फत्ते." Cache टाकणं सोपं; **कधी काढायचं** ठरवणं अवघड: आणि तेच न ठरवता cache टाकणारे सगळ्यात कपटी bug विकत घेतात: गिन्हाईकाला चुकीचा भाव/शिल्लक, जो तुमच्या testing मध्ये कधीच दिसत नाही (कारण तुमचं cache ताजं असतं). Stale cache चे bugs हे software मधले सर्वात जास्त वेळ लपून राहणारे bugs आहेत. Cache टाकताना पहिला प्रश्न "जलद होईल का" नाही, **"जुनं कधी, कसं फेकायचं"** हा आहे.

नकाशावर

- 1.1 एक यंत्र, तीन कामं 1.2 ACID: खात्री विकत
- 1.3 index: वाचन जलद, लेखन विकून
- 1.4 cache: वेग विकत, ताजेपणा विकून; खरा प्रश्न "जुनं कधी फेकायचं"; "किती जुनं चालेल?" आधी विचारा

सूत्रावर: cache हा सगळ्यात मोहक सौदा (प्रचंड वेग) आणि सगळ्यात लपलेली किंमत (शिळेपणा). जो "किती जुनं चालेल" आधी विचारतो तो cache चा मालक; जो नुसतं वेगासाठी टाकतो तो cache चा गुलाम.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनवर कुठलंही app उघडा, मग "refresh"/खाली-ओढून- ताजं-करा. जे नवं दिसतं ते cache मध्ये जुनं होतं: तुम्ही ते आत्ता फेकलंत (manual invalidation!). कुठली माहिती refresh शिवाय जुनी राहते (cache), कुठली नेहमी ताजी येते (no cache): याचा अंदाज घ्या. App-designers नी कुठे "किती जुनं चालेल" ठरवलंय ते तुम्हाला दिसेल.

विचार करा

आरतीचं app आता चांगलं चाललंय. पण एका दुपारी तिचा एकुलता एक server (cloud मधला) बंद पडला: hardware बिघाड. **पूर्ण app बंद**, दोन तास. गिन्हाईकं, दुकानं, सगळे अडकले. आरती हादरली: "एकाच यंत्रावर पूर्ण धंदा: हे धोक्याचं आहे. पण एकच database आहे: तिच्या दोन प्रती कशा ठेवणार? दोन्ही एकसारख्या कशा राहणार?"

आरती आत्ता 1.1 चा “एकच अपयश-बिंदू” प्रत्यक्ष भोगतेय. उत्तर: database ची प्रत ठेवायची: याला replication म्हणतात. पण “दोन्ही एकसारख्या कशा राहणार” हा तिचा प्रश्न या पुस्तकातला सगळ्यात खोल प्रश्नांपैकी एक आहे: पुढची भिंत.

Chapter 1.5 [SPINE]: Replication: प्रती आणि त्यांचा उशीर

आरतीच्या दोन-तास-बंद च्या धक्क्यातून पुढची भिंत: एकच यंत्र म्हणजे एकच अपयश-बिंदू (1.1 चा इशारा खरा ठरला). उपाय सोपा वाटतो: **प्रत ठेवा**. पण त्या “सोप्या” उपायात software मधलं एक खोल सत्य दडलंय.

Replication म्हणजे काय. दुकानाच्या वहीची उपमा: महत्त्वाची वही एकच असेल आणि जळली तर सर्वनाश. म्हणून शहाणा दुकानदार रोज एक **कार्बन-प्रत** दुसऱ्या दुकानात ठेवतो. Database replication तेच: मुख्य database (primary) च्या नोंदी लगोलग एक-दोन प्रतींमध्ये (replicas) उतरतात. फायदे दोन: (1) मुख्य बंद पडली तर प्रत ताबा घेते (आरतीची दोन-तास-बंद टळते); (2) **वाचन वाटता येतात**: हजारो “भाव बघा” च्या विनंत्या प्रतींकडे वळवा, मुख्य फक्त लेखनासाठी: भार हलका.

पण इथे तो खोल प्रश्न: प्रत किती मागे असते? कार्बन-प्रत दुसऱ्या दुकानात पोहोचायला वेळ लागतो: तेवढा वेळ ती प्रत **जुनी** असते. Database मध्ये हा उशीर milliseconds चा असतो, पण **शून्य नसतो**: याला **replication lag** म्हणतात. आणि इथून एक विचित्र, खरा प्रसंग जन्मतो: रमेशने ऑर्डर दिली (मुख्य database वर लिहिली गेली); लगेच “माझ्या ऑर्डर” बघितलं (ती विनंती एका **प्रती**कडे गेली, जी अजून 200 milliseconds मागे होती): आणि रमेशला **स्वतःचीच ऑर्डर दिसली नाही!** “आत्ताच तर केली, गेली कुठे?” त्याने घाबरून पुन्हा ऑर्डर केली: **दुहेरी ऑर्डर**. एका छोट्या उशिराने खरा गोंधळ.

सूत्र इथे नव्या रूपात: प्रती = टिकाव + वाचन-वेग विकत, “ताजेपणाची खात्री” विकून. आणि यातून एक निवड जन्मतो जी पुढच्या दोन chapters ना (CAP, consistency) हात घालते: **रमेशला त्याची स्वतःची ताजी ऑर्डर दाखवणं महत्त्वाचं** (मग त्याची वाचनं मुख्य database कडेच पाठवा, प्रतीकडे नको: “स्वतःचं लिखाण स्वतःला ताजं दिसो” हा नियम); पण **”शेजारच्या गावातल्या दुकानाची यादी 200ms जुनी दिसली तर चालेल** (ती प्रतीकडून घ्या, स्वस्त). एकाच app मध्ये दोन वेगळे निर्णय, माहितीच्या स्वभावानुसार. हेच रचनेचं सौंदर्य: **एकच नियम सगळीकडे लावणं म्हणजे नवखेपणा.**

खरं जग: वाचन-भार वाटणं इतकं मोठं का. बहुतेक app मध्ये वाचनं लेखनांपेक्षा **कितीतरी पट** जास्त असतात (एक ऑर्डर, पण ती-आधी 50 वेळा भाव-याद्या बघितल्या). म्हणून replicas कडे वाचनं वळवणं ही सगळ्यात स्वस्त, सगळ्यात परिणामकारक पहिली scaling-चाल आहे: मुख्य database चा भार नाटकीय घटतो. WhatsApp, बँका, प्रत्येक मोठं app हे करतं. पण lag चं भान ठेवून: **कुठलं वाचन ताजं हवं, कुठलं जुनं चालेल.**

महाग चूक

”प्रती ठेवल्या, आता सगळी वाचनं प्रतींकडे, मुख्य फक्त लेखनाला: जास्तीत जास्त भार-वाटप!” उत्साहात हे केलं की रमेशची-दुहेरी- ऑर्डर सारखे bugs पेरले जातात: वापरकर्त्याला **स्वतःचं ताजं लिखाण दिसत नाही** (read-after-write चा भंग), आणि तो गोंधळतो, पुन्हा करतो, तक्रार करतो. नियम: **वाचन प्रतीकडे वळवताना “हे वाचन किती जुनं चालेल” विचारा:** वापरकर्त्याचं स्वतःचं ताजं काम मुख्याकडून; बाकीचं जग प्रतीकडून. सरसकट भार-वाटप म्हणजे सरसकट गोंधळ.

नकाशावर

- | | |
|---|--------------------------|
| 1.1 एक यंत्र, एकच अपयश-बिंदू | 1.2 ACID |
| 1.3 index: वाचन जलद | 1.4 cache: ताजेपणा विकून |
| 1.5 replication: प्रती = टिकाव + वाचन-वेग विकत, ताजेपणाची खात्री विकून (lag); ताजं-हवं मुख्याकडे, जुनं-चालेल प्रतीकडे | |

सूत्रावर: replication ने आरतीने टिकाव विकत घेतला, पण “क्षणाची अचूकता” विकली. आणि तिला कळलं की माहितीच्या स्वभावानुसार वेगवेगळे सौदे करावे लागतात: हीच प्रौढ नजर.

स्वतः बघा (5 मिनिटं)

WhatsApp वर एखाद्या group मध्ये message पाठवा आणि लक्ष द्या: तुम्हाला तो **लगेच** दिसतो (तुमचं ताजं लिखाण), पण समोरच्याला पोहोचायला (आणि दोन निव्व्या खुणा यायला) क्षणभर जास्त. ते अंतर म्हणजे तुमच्या-फोनवरचं-ताजेपण आणि जगात- पसरण्याचा-उशीर यातलं अंतर: replication lag चा रोजचा अनुभव.

विचार करा

आरतीचं app आता तीन शहरांत, 5 लाख ऑर्डर रोज. प्रतींनी वाचन-भार पेलला, पण आता **लेखनचं** इतकी झाली की एकच मुख्य database धापा टाकतेय: सगळ्या ऑर्डर शेवटी तिच्याकडेच लिहाव्या लागतात (प्रती फक्त वाचनाला). आरती विचारते: “वाचनं वाटता आली, पण लेखनं एकाच जागी: ती कशी वाटणार? एकाच वहीत सगळं जग लिहितंय.”

हाच scaling चा सगळ्यात अवघड टप्पा: लेखन वाटणं. उत्तर: एकच प्रचंड वही न ठेवता तिचे **तुकडे** करायचे: शहरागणिक, पत्त्यागणिक. याला sharding म्हणतात, आणि त्यात एक कपटी सापळाही आहे. पुढची भिंत.

Chapter 1.6 [SPINE]: Sharding: वहीचे तुकडे आणि Hotspot चा सापळा

आरतीच्या “एकाच वहीत सगळं जग लिहितंय” या पेचावर उत्तर: वहीचे तुकडे करा. सोपं वाटतं; पण इथे software मधला एक सुंदर आणि कपटी धडा आहे.

Sharding म्हणजे काय. मोठ्या शहराच्या एकाच पोस्ट-ऑफिसची उपमा: सगळी पत्रं एकाच ठिकाणी आली तर रांग अनंत. उपाय: **विभागवार पोस्ट-ऑफिस**: कोथरूडची पत्रं कोथरूड-ऑफिसला, कॅप ची कॅप-ला. प्रत्येक ऑफिस फक्त स्वतःचा भाग सांभाळतं; भार वाटला. Database sharding तेच: एक प्रचंड table अनेक यंत्रांवर **तुकड्यांत** वाटायची. आरती शहरागणिक तुकडे करते: पुण्याच्या ऑर्डर एका यंत्रावर, मुंबईच्या दुसऱ्यावर, नाशिकच्या तिसऱ्यावर. आता लेखनही वाटली गेली: प्रत्येक यंत्र फक्त स्वतःच्या शहराचं ओझं उचलतं. Scaling ची सर्वात मोठी भिंत ओलांडली.

तुकडा कशाने पाडायचा: हा निर्णय सगळं ठरवतो (shard key). आरतीने “शहर” निवडलं. पण समजा तिने “ऑर्डरची तारीख” निवडली असती: मग **आजच्या** सगळ्या ऑर्डर एकाच यंत्रावर आदळल्या असत्या (सगळे आजच लिहितात!), बाकी यंत्रं रिकामी. एका यंत्रावर गर्दी, बाकी बसून: याला **hotspot** (तापलेला तुकडा) म्हणतात, आणि हा sharding चा सगळ्यात सामान्य सापळा आहे. **चांगला shard key तो जो भार सगळ्या तुकड्यांत सारखा वाटतो.** शहर बरं (सगळी शहरं एकसारखी वापरली तर); पण जर 80% धंदा पुण्यातच असेल, तर “शहर” सुद्धा पुण्याचा तुकडा तापवेल. **Shard key ची निवड = भार कसा पसरेल याचं भाकीत:** चुकला की एकच यंत्र जळतं आणि “sharding केलं तरी हळू” हे कोडं पडतं.

आणि sharding ची जबर किंमत (सूत्र सर्वात महाग इथे): तुकडे केले की एक गोष्ट **फार** अवघड होते: **दोन तुकड्यांना मिळून प्रश्न विचारणं.** “सगळ्या शहरांतली आजची एकूण विक्री किती?": आता हे एका वहीत नाही; प्रत्येक यंत्राला विचारून बेरीज करावी लागते: हळू, गुंतागुंतीचं. आणि “एका shard वरून दुसऱ्याकडे पैसे हलवा” (ACID व्यवहार दोन यंत्रांत!): हे तर भयंकर अवघड. म्हणून धडा: **sharding = लेखन-वाढ विकत, “सोपे एकत्रित प्रश्न” विकून.** म्हणूनच हा शेवटचा उपाय आहे, पहिला नाही: आधी index, cache, replicas (स्वस्त, सोपे); sharding तेव्हाच जेव्हा लेखनच यंत्राच्या ताकदीबाहेर जातात.

खरं जग: का हा शेवटचा उपाय. आरतीने योग्य क्रम पाळला: 5 लाख ऑर्डरपर्यंत index+cache+replicas पुरले; sharding तिने **50 लाख** ऑर्डरला, नाईलाजाने केलं: कारण एकदा sharded केलं की app चं code कायमचं गुंतागुंतीचं होतं (प्रत्येक प्रश्नाला “कुठला तुकडा” विचार). जगातल्या अनुभवी teams चा नियम: **”जोवर टाळता येईल तोवर sharding टाळा.”** नवखे उत्साहात आधीच shard करतात आणि विनाकारण गुंतागुंत ओढवून घेतात.

महाग चूक

”मोठं app म्हणजे sharding, आधीपासूनच करू.” Premature sharding (वेळेआधी तुकडे): 1.1 च्या premature scaling चा जुळा भाऊ. दहा-लाख ऑर्डर नसताना sharding म्हणजे: सोपे प्रश्न कायमचे अवघड, कोड कायमचं गुंतागुंतीचं, आणि तरी फायदा शून्य (कारण एक यंत्र पुरतंच होतं). Sharding चा खरा संकेत एकच: **लेखनंच** एका (शक्तिशाली) यंत्राच्या आवाक्याबाहेर. तोवर index-cache-replicas: स्वस्त, सोपे, पुरेसे.

नकाशावर

- 1.1-1.4 यंत्र | ACID | index | cache
- 1.5 replication: वाचन वाटलं
- 1.6 sharding: लेखन वाटलं (shard key = भार-वाटपाचं भाकीत; hotspot चा सापळा); लेखन-वाढ विकत, एकत्रित प्रश्न विकून; शेवटचा उपाय

सूत्रावर: sharding हा सर्वात मोठा सौदा: प्रचंड लेखन-क्षमता मिळवायला “साधेपणा” कायमचा विकावा लागतो. म्हणून तो जपून, उशिरा, नाईलाजाने: सौदा किती मोठा हे बघून निर्णय.

स्वतः बघा (5 मिनिटं)

तुमच्या शहरातल्या मोठ्या सेवेचा विचार करा (गॅस-कनेक्शन, वीज- बिल): त्यांचे विभागवार office असतात ना? तुमचं काम तुमच्याच विभागात होतं (जलद), पण “पूर्ण शहराची आकडेवारी” काढायला सगळ्या office ना विचारावं लागतं (हळू). हेच sharding: विभागणीने रोजचं काम जलद, पण एकत्रित चित्र अवघड. तुम्ही हे रोज अनुभवता.

विचार करा

आरतीचं app आता प्रचंड आहे. पण एका संध्याकाळी दिवाळीला लाखो ऑर्डर एकदम आल्या आणि app कोसळलं: server, database सगळं तयार होतं, पण ऑर्डर इतक्या **वेगात** आल्या की process करायला वेळच मिळेना: एकावर एक रचल्या गेल्या, गोंधळ. आरती विचारते: “गिन्हाईकाची ऑर्डर लगेच ‘घेतली’ म्हणायला हवी, पण ती पूर्ण करायला (दुकानाला कळवणं, delivery ठरवणं) वेळ लागतो. हे दोन्ही एकाच वेळी, एकाच वेगात कसं जमणार?”

जमणार नाही, आणि जमवायचंही नाही: ती दोन कामं **वेगळी** करायची. “ऑर्डर घेतली” लगेच सांगा; “ऑर्डर पूर्ण करा” एका रांगेत टाका, आरामात उरका. या रांगेचं नाव queue, आणि ती मोठ्या systems ची कणा आहे. पुढची भिंत.

Chapter 1.7 [SPINE]: Queue: रांगा, आणि “पुन्हा-पुन्हा चालणारं” काम

आरतीच्या दिवाळी-कोसळण्यातून एक संकल्पना जन्मते जी मोठ्या systems चा कणा आहे: **queue** (रांगा). आणि तिच्यासोबत एक जुळी संकल्पना, जिच्याशिवाय queue थोकादायक: **idempotency**.

दोन कामं वेगळी करा. आरतीचा पेच: “ऑर्डर घेतली” हे लगेच सांगायचंय (गिन्हाईक वाट बघतंय), पण “ऑर्डर पूर्ण करणं” (दुकानाला sms, delivery-boy ठरवणं, नकाशा काढणं) याला वेळ लागतो. हॉटेलची उपमा: वेटर तुमची ऑर्डर घेतो आणि **लगेच** “आलं साहेब” म्हणतो (तुम्ही निश्चित); मग ती चिठ्ठी स्वयंपाकघराच्या **खुंटीवर** लावतो; आचारी सवडीने, क्रमाने बनवतो. वेटर प्रत्येक ऑर्डर बनेपर्यंत उभा राहिला तर हॉटेल ठप्प. ती **खुंटी** म्हणजे queue: कामं तिथे रांगेत बसतात, आणि मागचे workers ती एकेक उचलून करतात, स्वतःच्या वेगाने.

Queue काय विकत घेते. (1) **धक्का शोषून घेणं:** दिवाळीला लाखो ऑर्डर एकदम आल्या तरी त्या रांगेत शांत बसतात; workers जमेल तसं उरकतात: app कोसळत नाही, फक्त थोडा उशीर. पुराचं पाणी धरणात साठवण्यासारखं. (2) **कामं तोडणं:** ऑर्डर-घेणं आणि ऑर्डर-पूर्ण-करणं वेगळे झाले; एक हळू झालं तरी दुसरं थांबत नाही. (3) **अपयश सोसणं:** delivery-यंत्रणा 2 मिनिटं बंद पडली तरी ऑर्डर रांगेत सुरक्षित; परत आल्यावर उचलल्या जातात: काहीच हरवत नाही.

पण इथे कपटी किंमत: काम दोनदा होऊ शकतं. Queue मध्ये एक विचित्र गोष्ट घडते: worker ने ऑर्डर उचलली, काम केलं, पण “झालं” कळवण्याआधीच तो कोसळला: queue ला वाटतं काम झालंच नाही, ती **तीच ऑर्डर दुसऱ्या worker ला** देते. आता रमेशला **दोन** पोती साखर, दोनदा पैसे. Queue ची हमी बहुधा “किमान एकदा” (at-least-once) असते: म्हणजे “एकदा तरी होईल, कदाचित दोनदाही.” भयंकर वाटतं, पण उपाय सुंदर आहे:

Idempotency: दोनदा केलं तरी परिणाम एकदाच. लिफ्टचं बटण पाच वेळा दाबा; लिफ्ट एकदाच येते: बटण **idempotent** आहे. आरती प्रत्येक ऑर्डरला एक **एकमेव क्रमांक** देते; worker काम करण्याआधी बघतो “हा क्रमांक आधी केलाय का?": केला असेल तर सोडून देतो. आता तीच ऑर्डर दोनदा आली तरी साखर एकदाच, पैसे एकदाच. **मोठ्या systems मध्ये “काम दोनदा येऊ शकतं” हे गृहीत धरून प्रत्येक कृती idempotent बांधणं:** ही top engineer ची सवय आहे; नवखा “दोनदा येणारच नाही” असं गृहीत धरतो आणि दुहेरी-payment चे bugs पेरतो.

महाग चूक

”Queue टाकली म्हणजे विश्वासार्ह झालं, निश्चित.” Queue विश्वासार्हता देते पण **”काम एकदाच होईल” ची हमी देत नाही**: उलट “दोनदा होऊ शकतं” हे तिचं डिफॉल्ट वर्तन आहे. हे न समजता queue वापरणारे सर्वात वाईट bugs पेरतात: दुहेरी पैसे, दुहेरी sms, दुहेरी ऑर्डर: आणि हे bugs फक्त गर्दीच्या वेळी (worker कोसळण्याच्या क्षणी) दिसतात, म्हणून testing मध्ये कधीच सापडत नाहीत. Queue वापरायची तर idempotency **सोबतच**: एक नाण्याच्या दोन बाजू.

नकाशावर

- 1.1-1.4 यंत्र | ACID | index | cache
- 1.5-1.6 replication (वाचन) | sharding (लेखन)
- 1.7 queue: धक्का शोषतो, कामं तोडतो; पण “किमान एकदा” =
दोनदा शक्य -> idempotency (दोनदा केलं तरी परिणाम एकदाच)

सूत्रावर: queue ने आरतीने लवचिकता-टिकाव विकत घेतले, पण “लगेच झालं” ची खात्री विकली (आता “होईल, थोड्या वेळात”). आणि “किमान एकदा” ही किंमत idempotency ने भरून काढली: प्रत्येक सोयीमागे सौदा, प्रत्येक सौद्यावर उपाय.

स्वतः बघा (5 मिनिटं)

तुम्ही ऑनलाइन payment करताना “back” दाबलंत किंवा page refresh केलंत तर “दोनदा पैसे कापले जातील का” ही भीती वाटते ना? चांगल्या app मध्ये कापले जात नाहीत: कारण ते idempotent आहेत (तोच व्यवहार-क्रमांक ओळखून दुसऱ्यांदा टाळतात). पुढच्या वेळी एखादं app “तुमची विनंती processing आहे, थांबा” म्हणतं तेव्हा ओळखा: मागे queue चालू आहे.

विचार करा

आरतीचं app आता प्रचंड. पण एका गोष्टीचा ताण वाढला: लाखो गिन्हाईकांच्या विनंत्या **कुठल्या** server कडे पाठवायच्या? आता एक server नाही, वीस आहेत (भार पेलायला). पण गिन्हाईकांच्या फोनला कसं कळणार कुठला server मोकळा आहे? आणि एखादा server बंद पडला तर विनंत्या तिकडे जाऊच नयेत, हे कोण बघणार?

एक “वाहतूक-पोलिस” लागतो: जो येणाऱ्या विनंत्या मोकळ्या servers कडे वळवतो, बंद पडलेले टाळतो. त्याचं नाव load balancer, आणि तो प्रत्येक मोठ्या app च्या दारात उभा असतो. पुढची भिंत.

Chapter 1.8 [SPINE]: Load Balancer: वाहतूक-पोलिस

आरतीच्या “कुठल्या server कडे पाठवायचं” या पेचावर उत्तर एका साध्या पण अपरिहार्य यंत्रात आहे: **load balancer**. आणि त्याच्यासोबत एक कल्पना जी मोठ्या systems चा पाया आहे: **horizontal scaling**.

दोन मार्गांनी मोठं होणं. आरतीचा एक server कमी पडतोय. दोन उपाय: (1) **तोच server अधिक शक्तिशाली करा** (जास्त RAM, वेगवान CPU): याला vertical scaling म्हणतात: एक बैल अधिक ताकदीचा करणं. सोपं, पण मर्यादा आहे (जगातला सर्वात शक्तिशाली एकच बैल किती ओढेल?) आणि महाग, आणि तो एकच बैल पडला तर पुन्हा सर्वनाश. (2) **अनेक साधे servers लावा** आणि भार वाटा: horizontal scaling: एका महाबलाऐवजी वीस सामान्य बैल. हेच मोठ्या systems चं रहस्य: **अनेक स्वस्त यंत्रं > एक महागडं यंत्र**: कारण अमर्याद वाढता येतं (अजून बैल जुंपा) आणि एक पडला तरी बाकी ओढतात.

पण अनेक बैल जुंपले की एक अडचण: येणारी विनंती **कुठल्या** बैलाकडे? इथे load balancer येतो: app च्या दारात उभा वाहतूक-पोलिस. प्रत्येक विनंती तो बघतो आणि **मोकळ्या** server कडे वळवतो: कधी आळीपाळीने (एकेकाला एक), कधी “सर्वात कमी भार असलेल्याकडे.” आणि सर्वात महत्वाचं: तो सतत **तब्येत तपासतो** (health check): एखादा server बंद पडला की त्याला विनंत्या पाठवणं **थांबवतो**: गिन्हाईकाला कळतही नाही की एक server मेला. आरतीची 1.4 मधली दोन-तास-बंद आता अशक्य: एक server पडला तरी वाहतूक-पोलिस बाकी एकोणीसकडे वळवतो.

यासाठी एक अट: servers “विसराळू” हवेत (stateless). वाहतूक-पोलिस कुठल्याही server कडे विनंती पाठवू शकतो, तेव्हाच जेव्हा **सगळे servers सारखे** असतात: कुणाकडेही खास माहिती साठलेली नसते. जर रमेशची “cart” (निवडलेल्या वस्तू) एकाच server च्या memory त साठली, तर पुढची विनंती दुसऱ्या server कडे गेली की cart गायब! म्हणून नियम: **servers ने काहीही स्वतःकडे साठवायचं नाही; सगळी माहिती database/cache मध्ये (सामायिक जागी).** मग प्रत्येक server “विसराळू पण सारखा”: कुठलाही कुठलीही विनंती हाताळू शकतो. हे stateless तत्त्व horizontal scaling चा पाया आहे: याशिवाय अनेक-servers चं गणित कोसळतं.

सूत्र इथे: horizontal scaling = अमर्याद वाढ + टिकाव विकत, पण “साधेपणा” विकून (आता अनेक यंत्रं, load balancer, stateless- शिस्त सांभाळा). लहान app ला एक server सोपा; मोठ्याला अनेक-पण-गुंतागुंतीचे. पुन्हा तोच सूत्र: सोपेपणा आणि वाढ यांचा सौदा, आता उलट दिशेने.

महाग चूक

”Server ची memory आहेच, त्यात गिन्हाईकाची माहिती (login, cart) ठेवू: जलद मिळेल.” हे stateful servers: horizontal scaling चा सगळ्यात सामान्य घात. एका server वर माहिती साठवली की load balancer ला त्याच गिन्हाईकाला **त्याच** server कडे पाठवावं लागतं (sticky): मग तो server पडला की त्या सगळ्या गिन्हाईकांची माहिती गेली, आणि भार-वाटपही बिघडलं. नियम: **server विसराळू ठेवा; माहिती सामायिक जागी**. ही एक शिस्त पाळली की तुमचं app वीस काय, दोनशे serversपर्यंत सहज वाढतं.

नकाशावर

1.1-1.4 यंत्र | ACID | index | cache
 1.5-1.7 replication | sharding | queue
 1.8 load balancer = वाहतूक-पोलिस (मोकळ्याकडे वळवतो, मेलेले टाळतो); horizontal scaling (अनेक स्वस्त > एक महाग);
 अट: servers विसराळू (stateless)

सूत्रावर: load balancer + stateless servers नी आरतीने “एकच अपयश-बिंदू” कायमचा संपवला (1.1 चं वर्तुळ पूर्ण): पण साधेपणा विकून. आता तिचं app अनेक यंत्रांचं जिवंत जाळं आहे.

स्वतः बघा (5 मिनिटं)

मोठ्या दुकानात/बँकेत गेलात तर “पुढील ग्राहक: counter 3” अशी व्यवस्था बघितली असेल: तो load balancer आहे! एकच रांग, अनेक counters, आणि व्यवस्थापक तुम्हाला मोकळ्या counter कडे पाठवतो; एखादा counter बंद झाला तर तिथे कुणाला पाठवत नाही. Software मध्ये अगदी हेच, फक्त milliseconds मध्ये आणि आपोआप.

विचार करा

आरतीचं app आता भारतभर. पण दिल्लीच्या गिन्हाईकाला app उघडायला मुंबईच्या पेक्षा जास्त वेळ लागतोय: कारण आरतीचे सगळे servers मुंबईत, आणि दिल्लीहून विनंती मुंबईला जाऊन-येऊन अंतर कापते (Book 1: latency = अंतर). आरती विचारते: “माहिती तीच आहे, मग दिल्लीवाल्याला हळू का? आणि दुकानाचे फोटो, app ची images: ती तर प्रत्येक वेळी मुंबईहून का पाठवायची?”

कारण अंतर हेच latency, आणि प्रकाशाच्या वेगालाही मर्यादा आहे. उपाय: गिन्हाईकाच्या **जवळ** माहितीची प्रत ठेवायची: विशेषतः न-बदलणारी (फोटो, images). याचं नाव CDN, आणि ते internet ला वेगवान ठेवणारं अदृश्य जाळं आहे. विभागाची शेवटची भिंत.

Chapter 1.9 [SPINE]: CDN: अंतर कापणारं जाळं

आरतीच्या “दिल्लीवाल्याला हळू का” या पेचावर उत्तर एका साध्या सत्यात आहे: **अंतर हेच वेळ**. आणि ते कापणारं जाळं म्हणजे CDN.

अंतराचं गणित (Book 1 खोल). प्रकाश जगातला सर्वात वेगवान, पण त्यालाही मर्यादा: मुंबई-दिल्ली सिग्नल जाऊन-यायला काही milliseconds; मुंबई-अमेरिका ~200-250 milliseconds (तपासलेला आकडा). एका विनंतीत हे ठीक; पण एक page उघडायला अनेक विनंत्या (images, फोटो, याद्या): प्रत्येकीला अंतराचा कर: दिल्लीवाल्याचं app रेंगाळतं. आणि हा उशीर server च्या ताकदीचा नाही, **भूगोलाचा** आहे: जगातला वेगवान server सुद्धा दूर असेल तर हळूच.

CDN म्हणजे काय. Content Delivery Network: गिन्हाईकाच्या **जवळ** माहितीच्या प्रती ठेवणारं, जगभर पसरलेलं यंत्र-जाळं. पाणपोईची उपमा: प्रत्येक तहानलेल्याला विहिरीपर्यंत (मुंबई-server) पाठवण्याऐवजी, गावोगाव पाणपोई ठेवायची: तहान जवळच भागते. आरती दुकानांचे फोटो, app च्या images, न-बदलणारं सगळं CDN वर टाकते: दिल्लीवाल्याला ते **दिल्लीच्या** पाणपोईतून मिळतं, मुंबईपर्यंत जावं लागत नाही. app भुरकन उघडतं.

महत्त्वाचा भेद: काय CDN वर जातं, काय नाही. CDN चांगलं काम करतं **न-बदलणाऱ्या** (static) गोष्टींसाठी: फोटो, video, app चं design, images: हे सगळ्यांना सारखं आणि क्वचित बदलतं, म्हणून जवळ प्रत ठेवणं सुरक्षित. पण **बदलणारी, वैयक्तिक** माहिती (रमेशची आजची ऑर्डर, त्याची शिल्लक) CDN वर ठेवता येत नाही: ती प्रत्येकाची वेगळी आणि सतत बदलणारी (1.4 चं cache-invalidation चं दुखणं परत!). म्हणून नियम: **static जवळ (CDN), dynamic मुळाशी (server)**. आरतीचं app अर्थ जवळून (design, फोटो: जलद), अर्थ मुळापासून (तिची ऑर्डर: ताजी) येतं: दोन्हीचं उत्तम.

सूत्र इथे: CDN = वेग (जवळीक) विकत, ताजेपणा + जागा विकून. हे cache चं भौगोलिक रूप आहे (1.4): जगभर प्रती, म्हणून जगभर तोच “जुनं कधी फेकायचं” चा प्रश्न: फोटो बदलला तर सगळ्या पाणपोईना कळवायला वेळ लागतो. पण static गोष्टी क्वचित बदलतात, म्हणून हा सौदा बहुधा सहज फायद्याचा.

विभाग-अखेरचं मोठं चित्र: आरती इथवर पोहोचली: एक laptop पासून जगभर पसरलेल्या जाळ्यापर्यंत. आणि प्रत्येक भिंतीने एक सौदा शिकवला. पण एक कडू सत्य उरलंय, जे पुढच्या दोन chapters चा विषय आहे: **हे सगळं जाळं गुंतागुंतीचं झालंय, आणि गुंतागुंतीच्या जाळ्यात काहीतरी बिघडणारच.** मोठं system कधीच “पूर्ण चालू” नसतं; त्याचे तुकडे सतत पडत-उठत असतात. ते गृहीत धरून बांधणं: हीच पुढची, आणि सर्वात प्रौढ, नजर.

महाग चूक

”CDN म्हणजे फक्त मोठ्या video-company साठी, आमच्या साध्या app ला कशाला?” आजच्या जगात कुठलंही app जगभर वापरलं जाऊ शकतं, आणि images/design हे प्रत्येक app मध्ये असतंच: CDN शिवाय दूरचा गिन्हाईक प्रत्येक फोटोला अंतराचा कर भरतो, app रेंगाळतं, तो सोडून जातो. उलट चूकही आहे: **बदलणारी वैयक्तिक माहिती CDN वर टाकणं** (रमेशला सुनीलची जुनी शिल्लक दिसणं): CDN फक्त static साठी. “काय जवळ, काय मुळाशी” ही रेषा न ओळखणं म्हणजे एकतर हळू app किंवा चुकीची माहिती.

नकाशावर

1.1-1.4 यंत्र | ACID | index | cache
 1.5-1.8 replication | sharding | queue | load balancer
 1.9 CDN = गिन्हाईकाजवळ प्रती (cache चं भौगोलिक रूप);
 static जवळ, dynamic मुळाशी; वेग विकत, ताजेपणा विकून

सूत्रावर: CDN ने आरतीने जगभर वेग विकत घेतला, ताजेपणाची थोडी किंमत देऊन: पण फक्त static साठी. विभागभर एकच धडा गिरवला: **प्रत्येक सोय एक सौदा; सौदा दिसणं हीच अभियांत्रिकीची नजर.**

स्वतः बघा (5 मिनिटं)

कुठलंही मोठं app/website उघडा: design, images भरभर येतात (CDN, जवळून), पण तुमची वैयक्तिक माहिती (तुमच्या ऑर्डर, संदेश) क्षणभर उशिरा (मुळापासून). हा सूक्ष्म फरक लक्षात घ्या: “सगळ्यांचं सारखं” जवळून, “फक्त तुमचं” दुरून. आता तुम्हाला मागचं जाळं दिसतंय.

विचार करा

आरतीचं app जगभर, वेगवान, टिकाऊ. पण एका सकाळी अचानक सगळंच ठप्प: server चालू, database चालू, पण एका छोट्या आतल्या सेवेने (payment-तपासणी) उत्तर द्यायचं थांबवलं, आणि तिची वाट बघत **बाकी सगळं** अडकून बसलं: एका दुव्याने पूर्ण साखळी थांबली. आरती हादरली: “प्रत्येक भाग तयार असूनही एक छोटा भाग अडला की सगळं का थांबतं? आणि मोठ्या company मध्येही असं होतं का?”

होतं, आणि जगातल्या सर्वात मोठ्या company नाही होतं: Facebook, Amazon, अख्खं internet काही तासांसाठी बंद पडलंय. मोठं system कधीच “पूर्ण चालू” नसतं: अपयश हेच त्याचं सामान्य रूप आहे. ते गृहीत धरून बांधण्याची कला: पुढची भिंत, आणि ती सर्वात महत्त्वाची.

Chapter 1.10 [SPINE]: CAP: तीनातले दोनच मिळतात

Replication पासून (1.5) एक प्रश्न घोंगावतोय: प्रती एकसारख्या कशा राहणार? आता त्याचं मूळ उत्तर: आणि ते एका कठोर, सुंदर नियमात आहे: **CAP theorem**. हा नियम एकदा समजला की distributed systems मधले अर्थ वाद संपतात.

तीन इच्छा. अनेक यंत्रांच्या (distributed) system कडून आपल्याला तीन गोष्टी हव्या असतात:

- C Consistency (सुसंगती): कुठल्याही यंत्राला विचारा, तेच, ताजं उत्तर. रमेशची ऑर्डर सगळ्या प्रतींवर लगेच सारखी.
- A Availability (उपलब्धता): विचारलं की उत्तर मिळतंच, कधीच "थांब, माहीत नाही" नाही.
- P Partition tolerance (फूट-सहनशीलता): दोन यंत्रांमधला संपर्क तुटला (network फुटली) तरी system चालूच राहते.

कठोर सत्य: तिन्ही एकदम मिळत नाहीत. आणि गंमत अशी की अनेक-यंत्रांच्या जगात **P (फूट) टाळताच येत नाही**: तारा तुटतात, network मध्ये अडथळे येतात: कधीतरी दोन यंत्रांचा संपर्क तुटणारच. म्हणून खरा निर्णय दोनातलाच उरतो: **फूट पडली त्या क्षणी, C निवडायचं की A?**

दोन कारकुनांची उपमा (हीच पूर्ण CAP आहे): दोन गावांतल्या दोन कारकुनांकडे एकाच खात्याची वही आहे, आणि त्यांच्यातला फोन बंद पडलाय (partition). आता गिन्हाईक एका कारकुनाकडे येऊन विचारतो "माझी शिल्लक?" कारकुनापुढे दोनच पर्याय:

- **A निवड (उपलब्धता):** "ही घ्या शिल्लक" (स्वतःच्या वहीतली): उत्तर मिळालं, पण दुसऱ्या गावात कदाचित बदल झाला असेल, म्हणजे हे उत्तर **कदाचित जुनं** (C सोडलं).
- **C निवड (सुसंगती):** "फोन येईपर्यंत थांबा, खात्री करून सांगतो": उत्तर **अचूकच** असेल, पण तोवर गिन्हाईक **ताटकळला** (A सोडलं).

एकच "बरोबर" उत्तर नाही: **माहितीच्या स्वभावावर** ठरतं. आणि इथे या पुस्तकाचं सूत्र शिखरावर: CAP म्हणजे "काय विकून काय घ्यायचं" चा सर्वात शुद्ध नियम.

कुठे काय: आरतीचे निर्णय. (1) **पैसे/शिल्लक: C निवडा.** "थोडं थांबा" चालेल, पण चुकीची शिल्लक नको (दुहेरी खर्च, कायदेशीर घोळ). बँका इथे नेहमी C: फूट पडली तर व्यवहार थांबवतात, चुकीचा आकडा दाखवत नाहीत. (2) **like-count / "किती जणांनी बघितलं" / दुकानाची जुनी यादी: A**

निवडा. उत्तर आलंच पाहिजे (गिन्हाईक थांबणार नाही); 5 सेकंद जुना like-count चालतो. Social media इथे नेहमी A. **एकाच app मध्ये, माहितीगणिक वेगळी निवड:** हीच प्रौढ रचना (replication chapter चा “ताज-हवं की जुन-चालेल” तोच धागा, आता नियमात बांधलेला).

महाग चूक

”आम्हाला तिन्ही हवेत: नेहमी ताजं, नेहमी उपलब्ध, आणि फूट सहन करणारं.” हे मागणं म्हणजे भौतिकशास्त्राशी भांडण: तिन्ही एकदम अशक्य आहेत (सिद्ध झालेलं). जो engineer “तिन्ही देतो” असं म्हणतो तो एकतर खोटं बोलतोय किंवा त्याला अजून फूट भेटलेली नाही (आणि ती भेटणारच). नवखा तिन्हीची अपेक्षा करतो; जाणकार **आधीच ठरवतो:** “या माहितीला फूट पडली तर मी C निवडेन की A”: आणि तो निर्णय माहितीच्या स्वभावावर, कोड लिहिण्याआधी.

नकाशावर

- 1.1-1.9 यंत्र...CDN (नऊ भिंती, नऊ सौदे)
 1.10 CAP: फूट अटळ; मग C की A? तीनातले दोनच; पैसे->C
 (थांबा पण अचूक), like->A (जुनं पण उत्तर मिळो)

सूत्रावर: CAP हा सूत्राचा गाभा-नियम: distributed जगात “सगळं एकदम” असं काहीच नसतं; प्रत्येक क्षणी एक विकून एक घ्यावंच लागतं. ही निवड टाळता येत नाही; फक्त **जाणून** किंवा **नकळत** करता येते. जाणकार जाणून करतो.

स्वतः बघा (5 मिनिटं)

दोन अनुभव आठवा: (1) बँकेत/UPI त कधी “system busy, थोड्या वेळाने प्रयत्न करा” आलंय? ती बँकेची **C निवड** होती: अचूकतेसाठी उपलब्धता सोडली. (2) Social media वर like-count कधी मागे-पुढे झालेला दिसला? ती **A निवड**: उत्तर देत राहायला अचूकता सोडली. एकच तत्त्व, दोन विरुद्ध निर्णय: माहितीच्या स्वभावाने.

विचार करा

आरतीला आता एक विचित्र गोष्ट जाणवली: तिच्या मोठ्या system मध्ये छोटे-छोटे बिघाड **सतत** होत असतात: एखादा server क्षणभर अडतो, एखादी सेवा उशिरा उत्तर देते: पण गिन्हाईकाला कळतही नाही. पण कधीकधी एक छोटा बिघाड **पसरतो** आणि सगळं कोसळतं (1.9 चा payment-अडथळा). आरती

विचारते: “छोटे बिघाड तर होणारच. मग ते पसरून सर्वनाश होऊ नये, एका जागी थांबावेत: याची रचना कशी करायची?”

हाच distributed systems मधला सर्वात प्रौढ प्रश्न: **अपयश गृहीत धरून, ते मर्यादित ठेवणारी रचना.** Timeouts, retries, आणि “circuit breaker”: एका बिघाडाला साखळी होऊ न देणारी यंत्रणा. पुढची भिंत.

Chapter 1.11 [SPINE]: अपयश हेच सामान्य: Timeouts, Retries, Circuit Breakers

आरतीचा प्रश्न distributed systems च्या हृदयाला हात घालतो. आणि उत्तराआधी एक मनोवृत्ती-बदल: मोठं system कधीच “पूर्ण चालू” नसतं. त्याचे शेकडो तुकडे सतत पडत-उठत असतात. नवखा “सगळं चालू आहे” गृहीत धरून बांधतो; जाणकार “काहीतरी बिघडलेलंच असणार” गृहीत धरून बांधतो. हा फरक सगळं ठरवतो.

समस्या: एक अडला की साखळी अडते. आरतीच्या 1.9 मधला प्रसंग: payment-सेवा अडली, आणि तिची वाट बघत ऑर्डर-सेवा अडली, आणि तिची वाट बघत app अडलं. एका दुकानासमोर रांग लागली म्हणून अख्खा बाजार ठप्प. याला **cascading failure** (साखळी-कोसळ) म्हणतात, आणि तीन साध्या युक्त्या ती थांबवतात:

युक्ती 1: Timeout (वाट किती बघायची ठरवा). ऑर्डर-सेवा payment-सेवेला विचारते आणि अनंत काळ वाट बघते: हीच मूळ चूक. उपाय: “2 सेकंदांत उत्तर नाही आलं तर वाट सोडून द्या.” हॉटेलात वेटरला “5 मिनिटांत नाही आलं तर सांगा, दुसरं मागवतो” म्हणण्यासारखं. Timeout शिवाय एक हळू सेवा पूर्ण system ला गोठवते. नियम: **प्रत्येक बाहेरच्या विनंतीला timeout: वाट अनंत कधीच नाही.**

युक्ती 2: Retry (पुन्हा प्रयत्न: पण जपून). क्षणिक बिघाड (network चा हिचकी) असेल तर पुन्हा विचारणं शहाणपण. पण इथे सापळा: सेवा आधीच भाराने कोसळतेय आणि सगळे एकदम पुन्हा-पुन्हा विचारू लागले तर ती अजून बुडते (आगीत तेल). म्हणून: retry थांबत-थांबत (आधी 1 सेकंद थांब, मग 2, मग 4: “backoff”), आणि मर्यादित (3 वेळा, मग सोडून द्या). आणि retry सोबत idempotency हवीच (1.7: पुन्हा विनंती = दुहेरी काम नको). नियम: **retry करा, पण थांबत आणि मोजून.**

युक्ती 3: Circuit Breaker (वीज-fuse चं तत्त्व). घरातला fuse आठवा: short-circuit झाला की fuse उडतो आणि पूर्ण घर वाचतो: एका बिघाडाला आग होऊ देत नाही. Circuit breaker तेच: payment-सेवा वारंवार फसतेय हे दिसलं की, ऑर्डर-सेवा तिला विचारणं थांबवते काही काळ (“circuit उघडला”): “ती सेवा आजारी आहे, तिला त्रास देऊन दोघेही बुडायला नको.” तेवढ्या वेळात ती सेवा सावरते; मग हळूच एक-दोन विनंत्या सोडून “बरी झाली का” तपासते. एका आजारी दुव्याला वेगळं काढून बाकी system वाचवणं: हीच circuit breaker ची कला.

तिन्हींचं एक तत्त्व: एका भागाचं अपयश दुसऱ्यात पसरू देऊ नका. प्रत्येक दुवा गृहीत धरतो की समोरचा कधीही फसू शकतो, आणि त्यासाठी आधीच तयार असतो: timeout ने वाट मर्यादित, retry ने क्षणिक बिघाड सावरतो, circuit breaker ने आजारी दुवा वेगळा करतो. हे “अपयश गृहीत धरून बांधणं”

(design for failure) म्हणजे मोठ्या systems ची सर्वात प्रौढ सवय: Book 4 च्या brakes चा (4.5) हा distributed भाऊ.

महाग चूक

”सगळं नीट चालेल असं गृहीत धरून बांधू; बिघडलं तर बघू.” हीच नवख्याची मूळ चूक: “happy path” (सगळं-सुरळीत रस्ता) बांधणं आणि अपयशाची तयारी न करणं. मग खरा भार आला की एक छोटा हिचकी साखळी होऊन सर्वनाश (1.9 चा प्रसंग). Timeout नाही म्हणून गोठतं, retry बेभान म्हणून आग भडकते, circuit breaker नाही म्हणून आजारी दुवा सगळ्यांना बुडवतो. जाणकार उलट बांधतो: **आधी विचारतो “हे फसलं तर?”, मग बांधतो.** “फसलं तर” चा विचार न करणारं प्रत्येक मोठं system शेवटी कोसळतंच: फक्त वेळ ठरलेली नसते.

नकाशावर

1.1-1.10 यंत्र...CAP

1.11 अपयश सामान्य: timeout (वाट मर्यादित) + retry (थांबत, मोजून) + circuit breaker (आजारी दुवा वेगळा); एका भागाचं अपयश पसरू देऊ नका

सूत्रावर: इथे सूत्र मनोवृत्तीत उतरतं: टिकाव विकत घ्यायचा तर “सगळं चालेल” चा भाबडेपणा विकावा लागतो. जो अपयश गृहीत धरतो तोच टिकणारं बांधतो.

स्वतः बघा (5 मिनिटं)

घरातला fuse/MCB बघा: तो का असतो? एका बिघाडाला (short- circuit) पूर्ण घर जळू न देण्यासाठी: तो मुद्दाम “तुटतो” म्हणून घर वाचतं. Software चा circuit breaker अगदी हेच करतो. आणि app वापरताना “काहीतरी चुकलं, पुन्हा प्रयत्न करा” दिसलं तर ओळखा: मागे timeout/retry ची यंत्रणा तुम्हाला अनंत-गोठण्यापासून वाचवतेय.

विचार करा

आरतीच्या app मध्ये हे सगळे बचाव आहेत. पण एका रात्री गिन्हाईकांच्या तक्रारी येऊ लागल्या “app हळू आहे”: आणि आरतीला **कळेचना कुठे** हळू आहे. वीस servers, अनेक सेवा, database, cache, queue: बिघाड नक्की कुठे? ती अंधारात चाचपडत होती. “एवढं मोठं जाळं आहे; आत काय चाललंय हे मला **दिसलंच** नाही तर दुरुस्त कसं करणार?”

हाच मोठ्या systems चा अदृश्य पाया: **दिसणं**. आत काय चाललंय हे मोजता-बघता आलं नाही तर सगळे बचाव आंधळे. प्रत्येक विनंतीचा माग, प्रत्येक सेवेची तब्येत, आणि “टोक” मोजणं: याचं नाव observability. पुढची भिंत (Book 4, 5.4 ची इथे भेट).

Chapter 1.12 [SPINE]: दिसणं: Observability आणि p99 ची गोष्ट

आरतीचा “आत काय चाललंय दिसलंच नाही” हा पेच मोठ्या systems चा अदृश्य पाया आहे. Book 4 त हेच AI-agent साठी भेटलं (5.4); इथे ते पूर्ण system साठी, आणि एका फसव्या आकड्याची गोष्ट घेऊन.

तीन डोळे (Book 4, 5.4 चे तेच, आता system-भर). (1) **Logs** (नोंदी): प्रत्येक घटनेची ओळ: “2:03 ला रमेशची ऑर्डर आली, payment अडलं.” (2) **Metrics** (आकडे): “आता किती विनंत्या/सेकंद, किती फसल्या, किती वेळ लागतोय.” (3) **Traces** (माग): एका विनंतीचा पूर्ण प्रवास: load balancer -> ऑर्डर-सेवा -> database -> payment: कुठे किती वेळ. आरतीच्या “कुठे हळू” चं उत्तर trace मध्ये: तिला दिसलं की payment-सेवा प्रत्येक विनंतीला 3 सेकंद खातेय. अंधारात चाचपडणं संपलं: **दिसलं, म्हणून दुरुस्त झालं.**

आणि आता तो फसवा आकडा: सरासरी. आरतीचं dashboard म्हणत होतं: “सरासरी उत्तर-वेळ: 0.4 सेकंद”: उत्तम! मग तक्रारी का? कारण **सरासरी खोटं बोलते.** कल्पना करा: 100 गिन्हाईकांपैकी 95 जणांना 0.1 सेकंदात उत्तर, पण 5 जणांना 8 सेकंद. सरासरी? ~0.5 सेकंद: गोड दिसतं. पण **तो प्रत्येक विसावा गिन्हाईक 8 सेकंद ताटकळतोय** आणि रागाने app सोडतोय. सरासरी त्या 5 जणांना लपवते.

म्हणून खरं मोजमाप: टोक (percentiles). सरासरीऐवजी अनुभवी teams **p95, p99** बघतात: “95% (किंवा 99%) गिन्हाईकांना या-पेक्षा-जलद उत्तर मिळतं; उरलेलं टोक किती वाईट?” आरतीचा p99 होता **8 सेकंद:** म्हणजे प्रत्येक शंभरातला एक जण भयंकर ताटकळत होता. एका मोठ्या app मध्ये “शंभरातला एक” म्हणजे रोज हजारो नाराज गिन्हाईक. आणि इथे एक कठोर सत्य: **धंदा सरासरीने मोजला जात नाही, टोकाने मोजला जातो.** सर्वात वाईट अनुभव घेणारा गिन्हाईकच सर्वात मोठ्याने बोलतो, सोडून जातो, वाईट review देतो. म्हणून मोठ्या systems मध्ये p99 (कधी p99.9) हाच खरा आरसा; सरासरी हा भूल देणारा आरसा.

Observability चं तत्त्व: न दिसणारं दुरुस्त करता येत नाही; आणि चुकीचं मोजणारं चुकीचं दुरुस्त करतं. आरतीने सरासरीवर विसंबून राहिली असती तर “सगळं ठीक आहे” म्हणत बसली असती, गिन्हाईकं गळत असताना. p99 बघितलं म्हणून खरी अडचण दिसली, payment-सेवा दुरुस्त झाली, टोक 8 सेकंदांवरून 0.5 वर आलं. दिसणं + योग्य मोजमाप = मोठ्या system चं आरोग्य.

महाग चूक

“सरासरी छान आहे, system निरोगी.” मोठ्या systems मधली सर्वात सामान्य आत्म-फसवणूक. सरासरी नेहमी गोड दिसते कारण ती बहुसंख्य- सुखी लोकांत अल्पसंख्य-दुःखी लपवते: पण तेच दुःखी लोक तुमचा धंदा ठरवतात (ते बोलतात, सोडतात, review देतात). सरासरीवर system “निरोगी” म्हणणं म्हणजे “गावात सरासरी उत्पन्न चांगलं आहे” म्हणत गरिबी नाकारणं. नेहमी टोक (p95/p99) बघा: तिथे खरं दुखणं राहतं.

नकाशावर

1.1-1.11 यंत्र...अपयश-बचाव

1.12 observability: logs + metrics + traces = दिसणं; सरासरी खोट बोलते, टोक (p99) खरं; न दिसणारं दुरुस्त होत नाही

सूत्रावर: दिसणं हे फुकट नाही (नोंदी-आकडे साठवायचा खर्च: Book 4, 5.4): पण न-दिसण्याची किंमत त्याहून प्रचंड (आंधळं दुरुस्तीकाम, गळणारे गिन्हाईक). दृश्यता विकत घेणं हा शहाणा सौदा.

स्वतः बघा (5 मिनिटं)

एक विचार-प्रयोग: तुमच्या गावातल्या 10 लोकांचं उत्पन्न: 9 जणांचं Rs 20,000, एकाचं Rs 10 लाख. सरासरी? ~1.2 लाख: “गाव श्रीमंत”! पण 9 जण गरीब. हीच सरासरीची फसवणूक. आता उलट: 9 जणांना app 0.1 सेकंदात उघडतं, एकाला 8 सेकंद: सरासरी “वेगवान” म्हणेल, पण तो एक जण संतापलेला. टोक बघायला शिका: आयुष्यातही, system मध्येही.

विचार करा

आरतीने सगळं शिकलं: पण एका सकाळी उठून बघते तर तिचं app नाही, **अर्ध internet** बंद: तिचे servers चालू, code चालू, पण ज्या मोठ्या cloud वर ती अवलंबून होती त्यातच बिघाड. तिच्यासारख्या हजारो companies एकदम ठप्प. आरती सुत्र: “एवढ्या मोठ्या, अब्जावधींच्या company सुद्धा कोसळतात? आणि एका छोट्या चुकीने अख्खं internet? हे कसं शक्य आहे?”

शक्य आहे, आणि घडलंय: वारंवार. सर्वात मोठ्या company नी सर्वात मोठे अपघात केलेत, आणि प्रत्येक अपघातात एक अनमोल धडा आहे. पुढचा chapter शव-विच्छेदन आहे: ज्या दिवशी जग बंद पडलं.

Chapter 1.13 [SPINE]: शव-विच्छेदन: ज्या दिवशी जग बंद पडलं

हा chapter उलटा: चार खऱ्या महाप्रलयांची शव-विच्छेदनं (तपासलेली). हेतू भीती नाही; उलट दिलासा आणि धडा: जगातल्या सर्वात हुशार teams नीही हे भोगलंय, म्हणजे अपयश लाज नाही: न शिकणं लाज आहे.

प्रलय 1: Amazon S3, Feb 2017: एका टंकलेखन-चुकीने. Amazon चा S3 (जगाचा अर्धा data जिथे राहतो ती साठवण-सेवा) चार तास बंद पडली. कारण? एक अधिकृत engineer, ठरलेल्या पद्धतीने billing-यंत्रणा दुरुस्त करताना, एका command मध्ये चुकीचा आकडा टाकला: ठरवलेल्यापेक्षा कितीतरी जास्त servers बंद झाले. आणि गंमत (कडू): Amazon चं “सगळं व्यवस्थित आहे का” दाखवणारं dashboard स्वतः S3 वरच चालत होतं: म्हणून S3 बंद पडल्यावर dashboard ही लाल दाखवू शकेना: Amazon ला Twitter वरून सांगावं लागलं! धडा: एका माणसाची एक टंकचूक जग बंद करू शकते: म्हणून धोक्याच्या commands ना रचनात्मक अडसर हवेत (Book 4, 4.5 चे brakes; एकाच command ने प्रचंड विध्वंस अशक्य करणं).

प्रलय 2: Knight Capital, 2012: 45 मिनिटांत \$460 million. ही सर्वात थरारक. एका trading-company ने नवें code आठ servers वर टाकायचं होतं; engineer एक server विसरला. त्या एका जुन्या server वर एक मृत, विसरलेलं जुनं code जागं झालं आणि वेड्यासारखे चुकीचे शेअर-व्यवहार करू लागलं. 45 मिनिटांत कंपनीने ~\$460 million गमावले: आणि कंपनीच बुडाली. सकाळी 8 वाजता 97 इशारा-emails आले होते (“जुनं code जागं झालंय”): कुणीही वाचले नाहीत. धडा दुहेरी: अर्धवट deployment (एक server विसरणं) = मृत्यू (म्हणून पुढे 3.7 त canary/सुरक्षित-deploy); आणि इशारे वाजले पण कुणी बघत नव्हतं = observability असून पाहणारा नसेल तर व्यर्थ (1.12 चा कडू पूरक).

प्रलय 3: Facebook, Oct 2021: 6 तास, आणि दारंही उघडेनात. एका नेहमीच्या देखभाल-command ने चुकून Facebook च्या सगळ्या data-centers चा जगाशी संपर्क तोडला (BGP/DNS: internet चा पत्ता-यंत्रणा). Facebook, WhatsApp, Instagram: 6 तास गायब. आणि सर्वात विचित्र: engineers ना दुरुस्त करायला data-center मध्ये शिरताच येईना, कारण दाराचे electronic बॅज-lock सुद्धा त्याच network वर होते जे बंद पडलं होतं! धडा: तुमची दुरुस्ती-यंत्रणा त्याच गोष्टीवर अवलंबून असेल जी बिघडली, तर तुम्ही अडकता (circular dependency): बचाव-यंत्रणा नेहमी वेगळी, स्वतंत्र हवी.

प्रलय 4: CrowdStrike, Jul 2024: 85 लाख यंत्रं, एका update ने. एका सुरक्षा-company ने एक सदोष update पाठवलं आणि जगभरची ~85 लाख Windows यंत्रं एकदम कोसळली: विमानतळ, रुग्णालयं, बँका ठप्प (इतिहासातला सर्वात मोठा IT-बिघाड, यंत्र-संख्येने). कारण: update **सगळ्यांना एकदम गेलं**, आधी थोड्यांवर तपासून नाही. धडा: **कुठलाही बदल सगळ्यांना एकदम देऊ नका; आधी थोड्यांवर** (हाच 3.7 चा canary; इथे त्याची गैरहजेरी अब्जांची पडली).

चार प्रलयांचं समान सूत्र: प्रत्येक ठिकाणी बिघाड **लहान** होता (एक टंकचूक, एक विसरलेला server, एक command, एक update); तो **पसरला** म्हणून महाप्रलय. म्हणजे 1.11 चा धडा जगाच्या पातळीवर: **अपयश टाळता येत नाही; पसरणं टाळता येतं.** आणि दुसरा, दिलासा देणारा धडा: या सगळ्या company नी नंतर **प्रामाणिक शव-विच्छेदनं जाहीर** केली (कुणाला दोष न देता, “रचना कशी बदलली”): तीच प्रौढ संस्कृती (3.6 त पूर्ण).

महाग चूक

”आमचं छोटं app; असे महाप्रलय आम्हाला नाहीत.” उलट: या company कडे जगातले सर्वोत्तम engineers, अब्जांची यंत्रणा, तरी कोसळल्या: कारण अपयश **कौशल्याचा** प्रश्न नाही, **गुंतागुंतीचा** अपरिहार्य साथीदार आहे. तुमचं छोटं app कोसळलं तर बातमी होणार नाही, पण गिन्हाईक जाईल तोच. धडा उलट घ्या: मोठेही कोसळतात, म्हणून **अपयश गृहीत धरून बांधणं** ही लाज नाही, शहाणपण आहे: आणि कोसळल्यावर **दोष न देता शिकणं** ही खरी प्रौढता.

नकाशावर

1.1-1.12 यंत्र...observability
1.13 चार प्रलय: S3 (टंकचूक) | Knight (विसरला server, \$460M)
| FB (दारंही बंद) | CrowdStrike (85 लाख, एकदम update);
बिघाड लहान, पसरला म्हणून प्रलय; canary + स्वतंत्र बचाव

सूत्रावर: गुंतागुंत विकत घेतली की अपयश-शक्यता फुकट मिळते: मोठेपणाची न-टाळता-येणारी किंमत. शहाणपण अपयश टाळण्यात नाही (अशक्य), **पसरू न देण्यात आणि शिकण्यात** आहे.

स्वतः बघा (5 मिनिटं)

कधी असा दिवस आठवा जेव्हा WhatsApp/Instagram/UPI काही तास बंद होतं (2021 पासून अनेकदा घडलंय). तेव्हा वाटलं होतं “एवढी मोठी company, कशी बंद पडली?": आता उत्तर माहीत आहे: गुंतागुंतीच्या जाळ्यात एक लहान बिघाड पसरला. मोठेपणा आणि नाजूकपणा एकत्रच येतात: हेच या

chapter चं सत्य.

विचार करा

आरतीचं app आता स्थिरावलंय: वेगवान, टिकाऊ, दृश्य, अपयश- सज्ज. पण महिना-अखेरीस cloud चं bill आलं आणि आरती हादरली: अपेक्षेच्या तिप्पट! सगळं “चालतंय” पण खर्च गिळंकृत करतोय. ती विचारते: “servers, database, CDN, हे सगळं तर लागतंच. मग bill एवढं का, आणि नक्की कशाचं?”

मोठ्या systems मध्ये bill हे code इतकंच रचनेचं फळ असतं: आणि त्यात एक लपलेला, कपटी खर्च असतो जो बहुतेकांना उशिरा कळतो. Cloud-bill चं शरीर: विभागाची शेवटची भिंत.

Chapter 1.14 [DEPTH]: Cloud-Bill चं शरीर आणि Egress-सापळा

आरतीच्या तिप्पट-bill च्या धक्क्यातून एक व्यावहारिक chapter. [DEPTH]: घाई असेल तर “egress-सापळा” आणि महाग-चूक वाचा; पण थांबलात तर तुमचं भविष्यातलं bill वाचेल.

Cloud चं भाडं कशाचं येतं: चार खण. Book 1 ने cloud दिलं (भाड्याने यंत्रं). त्या भाड्याचे चार मुख्य भाग: (1) **गणित** (compute: servers चालू ठेवणं: तास-गणिक); (2) **साठवण** (storage: database, फोटो, backups: GB-गणिक); (3) **वाहतूक** (data ची ने-आण); (4) **खास सेवा** (तयार database, queue, वगैरे: सोयीचे, पण महाग). बहुतेकांना पहिले दोन माहीत असतात; आणि नेमका तिसरा त्यांना बुडवतो.

Egress-सापळा (सर्वात कपटी खर्च). Cloud चा एक विचित्र नियम: **data आत आणणं (ingress) बहुधा फुकट; data बाहेर काढणं (egress) महाग.** माशाच्या गळाची उपमा: आत ओढणं सोपं, फुकट; बाहेर पडणं महाग. आरतीचं bill का फुगलं? तिचे फोटो एका cloud मध्ये, app दुसरीकडे: प्रत्येक गिन्हाईकाला फोटो दाखवायला data **एका cloud मधून बाहेर** पडत होता: लाखो वेळा, प्रत्येक वेळी egress-भाडं. आणि हा खर्च **वापर वाढला की वाढतो**, म्हणून सुरुवातीला दिसत नाही, यश आलं की अचानक चावतो. CDN (1.9) egress कमी करतं (जवळून देतं, मुख्य cloud मधून वारंवार बाहेर नको): म्हणजे CDN फक्त वेगासाठी नाही, **bill साठीही**.

आणि सर्वात मोठा दीर्घकालीन सापळा: lock-in. Egress महाग ठेवण्यामागचा cloud चा हेतू उघड आहे: **तुमचा data आत ओढून घ्यायचा, बाहेर काढणं महाग करून तुम्हाला अडकवायचं.** एकदा अब्जावधी फोटो एका cloud मध्ये साठले की दुसरीकडे हलवायला egress-भाडंच इतकं की तुम्ही अडकता. हा जाणीवपूर्वक सौदा आहे: cloud सोय देतं, बदल्यात तुम्हाला बांधून ठेवतं. जाणकार हे **आधी** ओळखतो आणि रचना अशी करतो की गरज पडली तर हलता येईल (Book 4, 2.8 चा खोगीर-नियम इथेही: एका जागी बांधून पडू नका).

सूत्र इथे रोखठोक: cloud = सोय + वेग विकत, पैसा + स्वातंत्र्य विकून. सुरुवातीला सोय प्रचंड (यंत्रं विकत घ्यायची नाहीत, मिनिटांत सुरू); पण वाढलं की egress, खास-सेवांचं भाडं, lock-in: किंमत चढते. म्हणून Book 4 चा तोच निर्णय (6.4): **पायाभूत सोयी भाड्याने, पण bill रोज मोजा आणि हलता येईल अशी रचना ठेवा.**

महाग चूक

”Cloud आहे, यंत्र आपोआप वाढतात (auto-scale), निश्चित.” Auto-scaling सोयीचं, पण तेच bill-राक्षसाचं दार: भार वाढला की यंत्र आपोआप वाढतात **आणि bill ही**: आणि कुणी लक्ष ठेवत नसेल तर एका चुकीच्या loop ने किंवा एका हल्ल्याने रातोरात लाखोंचं bill (खऱ्या घटना आहेत). नियम: **bill हे metric आहे, रोज बघा** (observability फक्त वेगासाठी नाही, खर्चासाठीही: 1.12 चा पूरक); आणि खर्चाला **मर्यादा** (alerts, budgets) लावा. न मोजलेलं cloud-bill हे न मोजलेल्या p99 इतकंच घातक.

नकाशावर

1.1-1.13 यंत्र...चार प्रलय

1.14 cloud-bill: गणित + साठवण + वाहतूक + खास-सेवा;
egress-सापळा (बाहेर काढणं महाग) + lock-in; bill हे
रोज बघायचं metric

सूत्रावर: cloud चा सौदा वेळेनुसार उलटतो: सुरुवातीला सोय स्वस्त, यशानंतर स्वातंत्र्य महाग. सौदा **कालानुरूप** बदलतो हे ओळखणं ही सर्वात सूक्ष्म नजर.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनवरच्या cloud-storage (Google Drive/iCloud) चा विचार करा: data **आत** टाकणं सोपं, फुकट-वाटतं; पण जागा भरली की पैसे, आणि सगळं दुसरीकडे हलवायचं म्हटलं की किती किचकट: हाच lock-in, ग्राहक-रूपात. मोठ्या companies तेच, egress-भाड्यासह अनुभवतात. सोयीमागचा सापळा एकदा दिसला की तुम्ही सावध खरेदीदार होता.

विचार करा

आरतीचा प्रवास संपत आलाय: एक laptop पासून जगभर पसरलेल्या, वेगवान, टिकाऊ, दृश्य, खर्च-जागरूक system पर्यंत. पण तिला एक गोष्ट खटकतेय: तिने हे सगळं **तुकड्या-तुकड्याने, भिंतीमागून भिंत** शिकलं. “आता हे सगळे तुकडे मिळून एक पूर्ण चित्र कसं दिसतं? आणि पुढच्या वेळी नवं app बांधताना हे सगळं **आधीच** कसं आखायचं, प्रत्येक भिंतीला आपटल्याशिवाय?”

हाच विभागाचा कळस: सगळे तुकडे एका चित्रात जोडणं, आणि “भिंत येण्याआधी दिसणं” ही नजर कमावणं. आरतीचं पूर्ण चित्र: विभागाचा शेवटचा chapter.

Chapter 1.15 [SPINE]: आरतीचं पूणे चित्र

विभागाचा कळस: सगळे तुकडे एका चित्रात. आरती मागे वळून बघते: एक laptop पासून इथवरचा प्रवास, आणि प्रत्येक भिंतीचा सौदा. हे चित्र पाठ करा; कुठलंही मोठं app याच आकाराचं असतं.

एका विनंतीचा आजचा प्रवास (सुरुवातीच्या सहा पायऱ्या, आता पूर्ण वाढलेल्या):

```

गिन्हाईकाचा फोन
|
v
CDN                (जवळचं static: फोटो, design: जवळून, जलद)
|
v
Load Balancer      (वाहतूक-पोलिस: मोकळ्या server कडे, मेलेले टाळून)
|
v
अनेक Servers      (विसराळू/stateless; horizontal scaling;
|                  प्रत्येक timeout+retry+circuit breaker ने सज्ज)
|--> Cache          (वारंवारचं उत्तर हातासरशी; "जुनं कधी फेकायचं")
|--> Queue          (जड/धीमी कामं रांगेत; idempotent)
|
v
Database
  Primary (लेखन) --replication--> Replicas (वाचन; थोडं जुनं)
  मोठं झालं तर --> Shards (तुकडे; भार वाटला)
  प्रत्येक शोध <-- Index (अनुक्रमणिका; जिथे शोध तिथेच)

आणि या सगळ्यावर:
  Observability (logs+metrics+traces; टोक/p99 बघा)
  अपयश-सज्जता (एक भाग पडला तरी पसरू नये)
  Bill-भान      (egress, auto-scale रोज मोजा)

```

नऊ सौद्यांचा सारांश (हीच खरी शिकवण):

एक-यंत्र vs अनेक	सोपेपणा <-> वाढ+टिकाव
ACID	वेग <-> खात्री
index	लेखन+जागा <-> वाचन-वेग
cache / CDN	ताजेपणा <-> वेग
replication	ताजेपणाची-खात्री <-> टिकाव+वाचन-वेग
sharding	सोपे-प्रश्न <-> लेखन-वाढ
queue	"लगेच झालं" <-> लवचिकता (+idempotency)
CAP (फूट-क्षणी)	C (अचूक-पण-थांबा) <-> A (उत्तर-पण-जुनं)
cloud	पैसा+स्वातंत्र्य <-> सोय+वेग

आणि “भिंत येण्याआधी दिसणं” ची नजर (आरतीचा खरा प्रश्न): आता आरती नवं app बांधताना प्रत्येक भिंतीला आपटायची वाट बघत नाही; ती **आधीच** विचारते: “ही माहिती वाढणार का (index)? वारंवार तेच विचारलं जाणार का (cache)? एक यंत्र पडलं तर (replication+load balancer)? कुठलं काम धीमं आहे (queue)? फूट पडली तर C की A?” **पण: एकही गोष्ट गरजेआधी बांधायची नाही.** नजर म्हणजे “भिंत ओळखणं”, घाई म्हणजे “भिंत नसताना भिंत बांधणं” (premature scaling: 1.1, 1.6 चा इशारा). प्रौढ engineer भिंत **येताना दिसते** पण **आल्यावरच** बांधतो: आधी नाही (उगाच गुंतागुंत), खूप उशिरा नाही (crash).

सर्वात मोठा धडा, एका ओळीत: आरतीने कुठलीही “जादूई भारी रचना” शिकली नाही; तिने **सौदे बघायला** शिकलं. “सगळ्यात चांगलं software” असं काही नसतं; फक्त “या गर्दीला, या माहितीला, या क्षणी योग्य” असतं. आणि उद्या गर्दी बदलली की योग्य रचनाही बदलेल. **रचना ही स्थिर गोष्ट नाही; ती सतत बदलणाऱ्या सौद्यांची जिवंत निवड आहे.** हीच Part 1 ची कमाई, आणि Part 2 चा पाया: कारण Part 2 त आपण हे सौदे **मोठ्या रचना-निर्णयांना** लावू: SQL की NoSQL, एक-मोठं-app की अनेक-लहान-सेवा.

महाग चूक

”आता मला सगळं येतं, पुढच्या app मध्ये हे सगळं आधीच टाकतो.” Part 1 ची शिकवण **उलटी** आहे: तुम्हाला सगळे **सौदे** कळले, म्हणजे आता तुम्ही प्रत्येक गोष्ट **गरजेपुरती** टाकाल. ज्ञानाचा उपयोग “सगळं आधीच बांधणं” नाही, तर **”कधी काय लागेल हे ओळखून, तेव्हाच बांधणं.”** नवशिक्याची चूक: काहीच न जाणणं. अर्धवट-शिकलेल्याची चूक: सगळं आधीच बांधणं. जाणकाराची नजर: भिंत ओळखणं, आणि तिची वाट बघणं.

नकाशावर

Part 1 पूर्ण: प्रवासाचा पूर्ण नकाशा

CDN -> LB -> stateless servers (+cache, +queue) -> DB
(primary/replicas/shards, +index); त्यावर observability +
अपयश-सज्जता + bill-भान. नऊ सौदे. नजर = भित आधी दिसणं,
पण गरजेआधी न बांधणं.

सूत्रावर: पूर्ण Part 1 एका वाक्यात: **मोठेपणा फुकट नसतो; प्रत्येक भिंतीवर काहीतरी विकून काहीतरी घ्यावं लागतं, आणि जाणकार तो सौदा जाणून करतो.** हेच अभियांत्रिकीचं सौंदर्य.

स्वतः बघा (5 मिनिटं)

तुमचं आवडतं मोठं app (Zomato, WhatsApp, कुठलंही) घ्या आणि वरचं “पूर्ण चित्र” त्याला लावून बघा: त्यात CDN असेल (images भरभर), load balancer (कधीच “एकच server पडला” दिसत नाही), cache (काही गोष्टी झटकन, refresh वर बदलतात), queue (“processing, थांबा”). तुम्ही आता app च्या मागचं जाळं “वाचू” शकता: हीच Part 1 ची कमाई.

विचार करा

Part 1 संपला. आरतीला आता एक मोठा निर्णय घ्यायचाय: तिचं अख्खं app एकाच मोठ्या code मध्ये आहे (सोपं होतं सुरुवातीला). आता 30 engineers काम करतायत आणि एकमेकांच्या पायात येतायत: एकाने बदल केला की दुसऱ्याचं मोडतं. “app चे तुकडे करावेत का? प्रत्येक टीमला स्वतःचा तुकडा? की एकत्रच बरं?” आणि दुसरा प्रश्न: “माझी काही माहिती ACID-कडक database मध्ये आहे, पण like-count सारखी सैल माहिती त्याच कडक जागी कशाला? वेगळी, सैल database वापरू का?”

हे दोन प्रश्न Part 2 चे दोन सर्वात मोठे वाद आहेत: monolith vs microservices, आणि SQL vs NoSQL. आणि दोन्हीचं उत्तर Part 1 च्या एकाच सूत्रात आहे: **सौदा बघा, fashion नको.** Part 2, पान पहिलं: रचनेचे मोठे निर्णय.

BOOK 5, PART 1 चा शेवट

जे तुमच्याकडे आता आहे

एक server	तीन कामं (एक-विचार-लक्षात ठेव); सोपं पण एकच अपयश-बिंदू; premature scaling टाळा
database/ACID	शिस्तीची वही: अखंड-सुसंगत-एकांत-टिकाऊ; खात्री विकत, वेग विकून; backup तपासलेला हवा
index	अनुक्रमणिका; वाचन जलद, लेखन+जागा विकून; जिथे शोध तिथेच (सगळीकडे नको)
cache	हातासरशी उत्तर; वेग विकत, ताजेपणा विकून; खरा प्रश्न "जुनं कधी फेकायचं"
replication	प्रती; टिकाव+वाचन-वेग विकत, ताजेपणा विकून; ताजं-हवं मुख्याकडे, जुनं-चालेल प्रतीकडे
sharding	तुकडे; लेखन-वाढ विकत, एकत्रित प्रश्न विकून; shard key = भार-भाकीत; शेवटचा उपाय
queue	रांग; धक्का शोषतो; "किमान एकदा" -> idempotency (दोनदा केलं तरी परिणाम एकदाच)
load balancer	वाहतूक-पोलिस; horizontal scaling (अनेक स्वस्त > एक महाग); अट: servers विसराळू
CDN	गिऱ्हाईकाजवळ static; वेग+bill दोन्ही; dynamic मुळाशी
CAP	फूट अटळ; मग C (अचूक-थांबा) की A (उत्तर-जुनं); पैसे->C, like->A
अपयश-सज्जता	timeout + retry(थांबत) + circuit breaker; एका भागाचं अपयश पसरू देऊ नका
observability	logs+metrics+traces; सरासरी खोटं, टोक(p99) खरं
चार प्रलय	S3/Knight/FB/CrowdStrike: बिघाड लहान, पसरला म्हणून प्रलय; canary + स्वतंत्र बचाव
cloud-bill	egress-सापळा + lock-in; bill रोज बघा

स्वतःची परीक्षा (4 प्रश्न, बोलून; उत्तरं खाली)

1. App हळू झालं. पहिलं निदान "मोठा server घेऊ" का नाही?

कारण 90% हळूपणा रचनेत असतो, ताकदीत नाही: गहाळ index, गहाळ cache, वाचनं मुख्यावर (replicas कडे वळवा). पैसे ओतण्याआधी नजर. (1.3, 1.4, 1.5)

2. Queue वापरता तर idempotency का हवीच?

queue ची हमी “किमान एकदा” = तेच काम दोनदा येऊ शकतं; idempotency शिवाय दुहेरी payment/ऑर्डर. (1.7)

3. पैशाच्या शिल्लकीला CAP मध्ये C की A, आणि का?

C (सुसंगती): फूट पडली तर “थांबा” चालेल, पण चुकीचा आकडा नाही. like-count ला उलट (A). (1.10)

4. System निरोगी आहे हे सरासरी-वेळेवरून का ठरवू नये?

सरासरी बहुसंख्य-सुखींत अल्पसंख्य-दुःखी लपवते; धंदा टोकाने (p99) मोजला जातो: तोच गिन्हाईक बोलतो-सोडतो. (1.12)

पुढच्या भागाची चाहूल

आरतीचे दोन मोठे निर्णय Part 2 उघडतात: **app चे तुकडे करावेत का** (monolith vs microservices) आणि **कडक database की सैल** (SQL vs NoSQL). Part 2 त हे मोठे रचना-वाद, data modeling, API-रचना, आणि search: सगळं Part 1 च्याच एका सूत्राने: **सौदा बघा, fashion नको**.

BOOK 5, PART 1 चा MINI-GLOSSARY

ACID	database ची शिस्त: अखंड-सुसंगत-एकांत-टिकाऊ (1.2)
cache	वारंवारचं उत्तर जलद जागी; ताजेपणा विकून (1.4)
CAP	फूट-क्षणी C की A; तीनातले दोनच (1.10)
CDN	गिन्हाईकाजवळ static ची प्रत (1.9)
circuit breaker	आजारी दुवा वेगळा काढणारा fuse (1.11)
egress	cloud मधून data बाहेर काढण्याचं (महाग) भाडं (1.14)
horizontal scale	अनेक स्वस्त यंत्र > एक महाग (1.8)
idempotency	दोनदा केलं तरी परिणाम एकदाच (1.7)
index	database ची अनुक्रमणिका; वाचन जलद (1.3)
load balancer	विनंत्या मोकळ्या server कडे वळवणारा (1.8)
p99	टोकाचं मोजमाप; सरासरीची फसवणूक टाळते (1.12)
partition	अनेक यंत्रांमधला संपर्क तुटणं (1.10)
queue	जड कामं रांगेत; धक्का शोषतो (1.7)
replication	database ची प्रत; lag = किती जुनी (1.5)
shard key	तुकडा कशाने पाडायचा; भार-भाकीत (1.6)
sharding	database चे यंत्रांवर तुकडे; लेखन वाटलं (1.6)
stateless	server काहीही स्वतःकडे साठवत नाही (1.8)
timeout	वाट किती बघायची, मर्यादा (1.11)
vertical scale	तेच यंत्र अधिक शक्तिशाली करणं (1.8)

THE BLUEPRINT

Book 5, Part 2: रचनेचे निर्णय

मागच्या भागाचा शेवट: आरतीचे दोन मोठे निर्णय टेबलावर: app चे तुकडे करावेत का, आणि कडक database की सैल. हा भाग अशा **मोठ्या रचना-निर्णयांचा** आहे: जिथे “भारी tech” चा मोह सर्वात जास्त, आणि सूत्र सर्वात जास्त उपयोगी.

सूत्र (पुन्हा)

मोठेपणा फुकट नसतो.
प्रत्येक रचना-निर्णय म्हणजे काहीतरी विकून काहीतरी विकत घेणं.
Trade-off न दिसणं हीच खरी चूक. (आणि: fashion नको, सौदा बघा.)

कपाटावर एक नजर

Book 1 The Machine	Book 2 The Thinking Machine
Book 3 The Land Game	Book 4 The Harness
Book 5 The Blueprint	<- हे, Part 2: रचना-निर्णय

तीन भागांचा नकाशा: तुम्ही इथे आहात

PART 1	प्रवास: एक user ते कोटी	(झाला)
PART 2	रचनेचे निर्णय	<- तुम्ही इथे
	data-रचना, SQL/NoSQL, DB आतून, API, monolith/microservices, search, real-time, migrations	
PART 3	Elite शिस्त + खऱ्या इमारती	(पुढे)

Part 1 ची उजळणी: 4 प्रश्न (बोलून; उत्तरं खाली)

1. App हळू: पहिलं निदान काय, “मोठा server” का नाही?

| रचना: गहाळ index/cache/replica. 90% हळूपणा रचनेत, ताकदीत नाही. (1.3-1.5)

2. Cache/CDN चा खरा अवघड प्रश्न?

| "जुनं कधी फेकायचं" (invalidation); वेग विकत, ताजेपणा विकून. (1.4, 1.9)

3. पैशाला CAP मध्ये C की A?

| C: फूट पडली तर थांबा, पण चुकीचा आकडा नाही. (1.10)

4. System-आरोग्य सरासरीने का नाही?

| टोक (p99) खरं; सरासरी अल्पसंख्य-दुःखी लपवते. (1.12)

या भागात काय

Data कशी रचायची, SQL-vs-NoSQL चा गाजलेला वाद, database आतून कशी शोधते-लिहिते, API म्हणजे करार, monolith-vs- microservices (आणि Amazon Prime Video ची उलटी गोष्ट), search कशी चालते, real-time (WhatsApp), आणि चालत्या system चं चाक बदलणं (migrations). प्रत्येक निर्णयाला तेच सूत्र: सौदा बघा, fashion नको.

विभाग 2: मोठे रचना-निर्णय

Part 1 चे सौदे छोट्या-छोट्या भिंतींचे होते. या भागातले निर्णय मोठे आहेत: एकदा घेतले की app चं भवितव्य वर्षानुवर्ष ठरवतात, आणि बदलायला महाग. **हे तुमच्या धंद्यात कुठे येईल:** इथेच “भारी नवं tech वापरू” चा मोह सर्वात जास्त, आणि तिथेच सर्वात जास्त पैसा-वेळ वाया जातो. हा भाग तुम्हाला fashion आणि सौदा वेगळे करायला शिकवेल: top engineer ची सर्वात मोठी ओळख.

Chapter 2.1 [SPINE]: Data कशी रचायची: Data Modeling

Part 2 चा पाया: माहिती **कशी मांडायची**. हा निर्णय सर्वात आधीचा आणि सर्वात टिकाऊ: कारण एकदा लाखो नोंदी एका रचनेत साठल्या की ती बदलणं म्हणजे चालत्या गाडीचं चाक बदलणं (2.11 चा विषय). म्हणून आधी नीट.

Data model म्हणजे काय. दुकानाच्या वहांची उपमा: कोणती माहिती कोणत्या वहीत, कोणते रकाने, आणि वहा एकमेकांशी कशा जोडलेल्या. आरतीच्या app ची माहिती: गिन्हाईक, दुकान, वस्तू, ऑर्डर. प्रश्न: या चार गोष्टी कशा मांडायच्या की (अ) माहिती **एकाच जागी** राहील (दुरुस्ती एकाच ठिकाणी) आणि (ब) हवं ते **जलद** मिळेल. या दोन इच्छांमध्ये सतत ताण आहे, आणि तोच data modeling चा गाभा.

पहिलं तत्त्व: एक सत्य, एक जागा (normalization). रमेशचा पत्ता प्रत्येक ऑर्डरमध्ये पुन्हा-पुन्हा लिहायचा का? नाही: रमेशची एक “गिन्हाईक-नोंद”, त्यात पत्ता एकदाच; ऑर्डर फक्त रमेशच्या **क्रमांकाकडे बोट** करते. फायदा: रमेशने पत्ता बदलला तर **एकाच** जागी बदल, सगळ्या ऑर्डरना लागू. जर पत्ता प्रत्येक ऑर्डरमध्ये कॉपी केला असता, तर 500 ऑर्डरमध्ये 500 जागी बदलावा लागला असता (आणि एखादी राहिली की चुकीचा पत्ता: data मधली विसंगती). हे “पुनरावृत्ती टाळणं, बोट-दाखवणं” म्हणजे normalization: **एक सत्य, एकाच जागी**.

पण इथेही सूत्र: normalization = शुद्धता विकत, वेग विकून. रमेशची ऑर्डर + त्याचा पत्ता + दुकानाचं नाव दाखवायला आता **तीन वहा जुळवाव्या** लागतात (ऑर्डर -> गिन्हाईक -> दुकान: याला “join” म्हणतात): शुद्ध, पण हळू. म्हणून कधीकधी जाणीवपूर्वक **उलट** केलं जातं: वारंवार-लागणारी माहिती मुद्दाम कॉपी करून एकत्र ठेवणं (denormalization): वेग मिळतो, पण आता “एक सत्य” सुटतं (बदल अनेक जागी). निर्णय कशावर? **वाचन जास्त की लेखन जास्त, आणि विसंगती किती परवडते:** पैसे-नोंदींना शुद्धता (normalize); वारंवार-दाखवायच्या याद्यांना वेग (थोडं denormalize). पुन्हा तोच index-चा (1.3) आत्मा: वाचन-वि.-लेखन चा सौदा, आता रचनेच्या पातळीवर.

दुसरं तत्त्व: नाती ओळखा. एका गिन्हाईकाच्या **अनेक** ऑर्डर (one-to-many); एका ऑर्डरमध्ये **अनेक** वस्तू आणि एक वस्तू **अनेक** ऑर्डरमध्ये (many-to-many). ही नाती नीट ओळखून मांडणं म्हणजे अर्थ data modeling. चुकीचं नातं मांडलं (उदा एका ऑर्डरला एकच वस्तू, नंतर गरज पडली अनेक) की पुढे पूर्ण रचना मोडून पुन्हा बांधावी लागते.

महाग चूक

”आधी code लिहू, data-रचना नंतर बघू.” उलटं शहाणपण: **data- रचना आधी, नीट.** कारण code बदलणं सोपं (आज लिहिलं, उद्या बदललं), पण **लाखो नोंदी साठल्यावर रचना बदलणं** म्हणजे महिन्यांचं, धोक्याचं काम (2.11). Twitter, Facebook: सगळ्यांनी सुरुवातीच्या घाईच्या data-रचनेची किंमत वर्षानुवर्ष भरली. Data- model वर एक दिवस जास्त घालवणं = पुढे महिने वाचवणं. “नाती” आधी कागदावर काढा, मग code.

नकाशावर

Part 2 चा नकाशा (वाढत जाईल):

2.1 data modeling: एक सत्य एका जागी (normalize) vs वेगासाठी कोपी (denormalize); नाती आधी ओळखा; रचना आधी, नीट

सूत्रावर: data-रचना हा सर्वात टिकाऊ सौदा: शुद्धता की वेग, आणि तो बदलायला सर्वात महाग. म्हणून इथे घाई सर्वात महाग, विचार सर्वात स्वस्त.

स्वतः बघा (5 मिनिटं)

कागदावर तुमच्या ओळखीच्या धंद्याची (दुकान, वर्ग, काहीही) माहिती चार पेद्यांत मांडा आणि बाणांनी नाती जोडा: कोण-कोणाशी one-to-many, कोण many-to-many. एखादी माहिती दोन जागी लिहावी लागतेय का बघा: तिथे normalization चा निर्णय आहे. हा साधा व्यायाम म्हणजे data modeling चा गाभा.

विचार करा

आरतीने विचारलं: “माझी ऑर्डर-नोंदी कडक database (PostgreSQL) मध्ये, ठीक. पण ‘रमेशने आज कोणकोणती दुकानं बघितली’, ‘कुठल्या वस्तूवर किती वेळा click’, अशी प्रचंड, सैल, दर सेकंदाला वाढणारी माहिती त्याच कडक, ACID database मध्ये कशाला कोंबू? तिथे शुद्धतेपेक्षा वेग-आणि-आकार हवा.”

अगदी नेमकं बोट. सगळी माहिती एकाच प्रकारच्या database मध्ये कोंबणं ही चूक; माहितीच्या स्वभावानुसार database ही वेगळी हवी. आणि इथूनच software मधला सर्वात गाजलेला वाद जन्मतो: SQL vs NoSQL. पुढचा chapter.

Chapter 2.2 [SPINE]: SQL vs NoSQL: गाजलेला वाद [दोन्ही बाजू]

Software मधला सर्वात गाजलेला, आणि सर्वात गैरसमजलेला वाद. आरतीच्या प्रश्नातून तो नीट उलगडू: आणि नेहमीप्रमाणे, उत्तर “कुठलं भारी” नसून “कधी कुठलं” असेल.

दोन घराणी. (1) **SQL / relational** (PostgreSQL, MySQL): माहिती **ठरलेल्या रकान्यांच्या टेबलांत**, नाती जोडलेली (2.1), आणि ACID ची कडक शिस्त (1.2). सरकारी फॉर्मची उपमा: प्रत्येक रकाना ठरलेला, शिस्त कडक, चूक होत नाही, पण साचा बदलायला अर्ज-परवानग्या. (2) **NoSQL** (MongoDB, Cassandra, Redis वगैरे: एक नाव नाही, अनेक प्रकार): माहिती **सैल, लवचिक** रूपात (प्रत्येक नोंद वेगळ्या आकाराची चालते), प्रचंड आकाराला- वेगाला सोयीची, पण कडक शिस्त सैल. कोन्या वहीची उपमा: काहीही, कसंही लिहा, वेगवान, पण शिस्त तुमच्यावर.

खरा फरक एका ओळीत: SQL शिस्त-आणि-नाती देतं (शुद्धता, गुंतागुंतीचे प्रश्न सोपे), वेग-लवचिकता विकून; NoSQL आकार- आणि-वेग देतं, शिस्त-नाती विकून. हा तोच जुना ACID-सौदा (1.2), आता database-निवडीत. आरतीचं उत्तर आता स्पष्ट: **ऑर्डर-पैसे-शिल्लक (नाती, शुद्धता हवी): SQL; click-log, बघितलेली-दुकानं (प्रचंड, सैल, वेग हवा): NoSQL.** एकाच app मध्ये दोन्ही: माहितीच्या स्वभावानुसार. याला “polyglot persistence” म्हणतात: अनेक database-भाषा एकत्र.

आणि आता 2026 चा कडू निकाल (हाच contested-गाभा):

NoSQL-उत्साहाची लाट (2010 च्या दशकात): “SQL जुनं, NoSQL सगळीकडे वापरा, तेच भविष्य!”
 प्रत्यक्ष अनुभव (आजवरचा): बहुतेकांना NoSQL ची गरज नव्हती; त्यांनी नाती (joins), शिस्त (ACID) गमावली आणि नंतर ती हाताने, code मध्ये, वाईट रीतीने पुन्हा बांधावी लागली.
 2026 ची स्थिती: PostgreSQL (SQL) हे developers मध्ये सलग सर्वात लोकप्रिय database (Stack Overflow पाहणी). आणि गंमत: आधुनिक SQL आता NoSQL ची लवचिकताही (JSON रकाने) देतं. “SQL मेलं” ही भविष्यवाणी उलटली.
 निकाल: NoSQL चुकीचं नाही; तो ****विशिष्ट**** गरजांसाठी (प्रचंड आकार, सैल-रचना, अति-वेग) उत्तम. पण ****डिफॉल्ट**** SQL असावं; NoSQL तेव्हा जेव्हा SQL खरेच कमी पडतं, fashion म्हणून नाही.

निवडीची कसोटी (छापून ठेवा): (1) माहितीला **नाती + शुद्धता** लागतात का (पैसे, ऑर्डर, नोंदी)? -> SQL. (2) माहिती **प्रचंड, सैल-आकाराची, अति-वेगवान** आहे आणि नाती कमी (logs, sensor- data, cache)? -> NoSQL. (3) **शंका असेल तर SQL:** कारण नाती नंतर सोडता येतात, पण नसताना पुन्हा

बांधणं महाग.

महाग चूक

”मोठं, आधुनिक app म्हणजे NoSQL; SQL जुन्या लोकांचं.” हा fashion-निर्णय हजारो startups ला महागात पडला: त्यांनी सुरुवातीलाच NoSQL निवडलं (कारण “cool”), मग कळलं की त्यांना ऑर्डर-गिन्हाईक-payment मधली नाती हवीच होती (जी SQL फुकट देतं), आणि ती त्यांनी code मध्ये हाताने, buggy रीतीने बांधली. **Database ची निवड “काय cool आहे” वर नाही, “माझ्या माहितीला काय लागतं” वर.** Fashion आणि सौदा वेगळे करणं: हीच या chapter ची, आणि पूर्ण Part 2 ची, खरी परीक्षा.

नकाशावर

- 2.1 data modeling: एक सत्य एका जागी; नाती आधी
- 2.2 SQL vs NoSQL: SQL=शिस्त+नाती, NoSQL=आकार+वेग;
डिफॉल्ट SQL, NoSQL गरजेला (fashion नको); दोन्ही एकत्र
(माहितीगणिक)

सूत्रावर: हा वाद म्हणजे सूत्राची शुद्ध परीक्षा: “भारी tech” चा मोह विरुद्ध “माझ्या माहितीचा सौदा.” जो सौदा बघतो तो बरोबर निवडतो; जो fashion बघतो तो दोनदा बांधतो.

स्वतः बघा (5 मिनिटं)

तुमच्या ओळखीच्या माहितीचे दोन प्रकार शोधा: एक जिथे **अचूकता आणि नाती** महत्त्वाची (बँक-खातं, विद्यार्थ्यांचे गुण), एक जिथे **प्रचंड आकार आणि वेग** पण अचूकता दुय्यम (कोणी काय बघितलं, किती वेळ). पहिल्याला SQL, दुसऱ्याला NoSQL: का, ते स्वतःला समजावा. ही जोडी ओळखता येणं = database-नजर.

विचार करा

आरतीला कुतूहल: “SQL ‘नाती जुळवून’ शोधतं, NoSQL ‘सैल-वेगवान’, हे कळलं. पण **आतून** काय वेगळं करतात? म्हणजे index (1.3) तर दोन्हीत असेल. एकच ‘शोध’ दोघांत इतका वेगळा का पडतो? database प्रत्यक्ष disk वर लिहिते-शोधते कशी?”

हाच पुढचा, थोडा खोलातला पण सुंदर chapter: database चं हृदय: B-tree आणि LSM-tree, दोन वेगळ्या लिहिण्या-शोधण्याच्या रीती, आणि त्यातून वाचन-वि.-लेखन चा तोच सौदा, आता सर्वात खोल पातळीवर.

Chapter 2.3 [DEPTH]: Database चं हृदय: B-tree आणि LSM-tree

[DEPTH]: हा chapter database च्या तळघरातला आहे. घाई असेल तर शेवटचा सौदा-ठोकळा वाचा. पण थांबलात तर “वाचन-वि.-लेखन” हा सौदा त्याच्या सर्वात खोल, सुंदर रूपात दिसेल: आणि मग SQL आणि NoSQL मधला आतला फरक कायमचा समजेल.

दोन प्रश्न प्रत्येक database ला: disk वर माहिती **कशी लिहायची** की (अ) शोधताना जलद सापडेल आणि (ब) लिहितानाही वेग राहील. या दोन इच्छांमध्ये सतत ताण, आणि जगात दोन मोठ्या युक्त्या आहेत: दोन वेगळे “सौदे.”

युक्ती 1: B-tree (SQL चं हृदय: वाचनासाठी बांधलेलं). सुव्यवस्थित ग्रंथालयाची उपमा: प्रत्येक पुस्तक त्याच्या ठरलेल्या, क्रमवार जागी. नवं पुस्तक आलं की **नेमक्या जागी** ठेवायचं (मध्ये सारून जागा करून). फायदा: शोधताना अचूक जागी, फार जलद (काही उड्यांत). किंमत: प्रत्येक नवं पुस्तक ठेवायला जागा करावी लागते: **लेखन थोडं हळू आणि विखुरलेलं** (disk वर इकडे- तिकडे जाऊन जागा करणं). B-tree = **वाचन जलद, लेखन थोडं महाग**. म्हणून जिथे वाचन जास्त, नाती-शोध जास्त: SQL/PostgreSQL इथे B-tree वापरतं.

युक्ती 2: LSM-tree (अनेक NoSQL चं हृदय: लेखनासाठी बांधलेलं). वेगळी युक्ती: नवं पुस्तक जागा शोधत बसू नको; **येईल तसं एका ढिगात, क्रमाने रचत जा** (फक्त शेवटी भर घाल: वेगवान). शोधायचं तेव्हा? अनेक ढिगांत बघावं लागेल (हळू). आणि मागाहून, सवडीने, ढिगांचं **एकत्रीकरण** (compaction) करून नीट लावायचं. फायदा: **लेखन प्रचंड जलद** (फक्त शेवटी भर). किंमत: वाचन थोडं हळू (अनेक ढिग), आणि compaction चा मागचा खर्च. LSM = **लेखन जलद, वाचन थोडं महाग**. म्हणून जिथे प्रचंड-वेगवान लेखन (sensor-data, logs, messages): Cassandra वगैरे NoSQL इथे LSM वापरतात.

आणि आता तो सुंदर, एकच सौदा दोन रूपांत:

B-tree (SQL): वाचन जलद <-> लेखन महाग. वाचन-जास्त कामाला.
LSM (NoSQL): लेखन जलद <-> वाचन महाग. लेखन-जास्त कामाला.

म्हणजे SQL-vs-NoSQL चा फरक फक्त “शिस्त-vs-सैल” नाही (2.2); तो **खोलात, disk वरच्या लेखन-वाचन सौद्याचा** आहे. आणि हा तोच index-चा सौदा (1.3: वाचन जलद, लेखन विकून), आता database च्या पायातच कोरलेला. software मध्ये **एकच सौदा पुन्हा-पुन्हा, वेगवेगळ्या पातळ्यांवर** दिसतो: हे ओळखता येणं म्हणजे खोल नजर.

हे तुम्हाला रोज बांधावं लागणार नाही (तयार databases हे करतात). मग का शिकायचं? कारण **database निवडताना** हा सौदा माहीत असला की तुम्ही बरोबर निवडता: “माझं app लेखन- जड की वाचन-जड?” या एका प्रश्नाने अर्थी निवड होते. आणि जेव्हा database “अचानक हळू” होते तेव्हा (उदा LSM ची compaction चालू असताना), कारण कळतं: अंधारात चाचपडत नाही.

महाग चूक

“Database आतून कशी चालते ते कशाला शिकायचं, तयार तर आहे.” तयार असलं तरी **निवड आणि निदान** तुमचं. LSM-database लेखन-जड कामाला निवडायची तिथे B-tree निवडली (किंवा उलट) की app झगडतं, आणि “database हळू आहे” म्हणत महागडं upgrade करत बसता: खरी अडचण चुकीच्या हृदयाची निवड असते. एका प्रश्नाचं (लेखन-जड की वाचन-जड) उत्तर हे 10x कामगिरीचा फरक ठरवू शकतं: तो प्रश्न विचारायला हे हृदय समजावं लागतं.

नकाशावर

- 2.1 data modeling 2.2 SQL vs NoSQL
- 2.3 DB चं हृदय: B-tree (वाचन जलद, SQL) vs LSM (लेखन जलद, NoSQL); तोच वाचन-वि.-लेखन सौदा, पायात कोरलेला

सूत्रावर: database च्या तळाशीही तोच एक सौदा: वाचन की लेखन. software मध्ये सौदे fractal आहेत: प्रत्येक पातळीवर तेच आकार, लहान-मोठे. एक सौदा नीट कळला की सगळे कळतात.

स्वतः बघा (5 मिनिटं)

दोन प्रकारे कपाट लावायचा विचार करा: (1) प्रत्येक वस्तू अकारविल्हे नेमक्या जागी (शोधायला सोपं, ठेवायला वेळ: B-tree); (2) येईल ती वस्तू पुढच्या रिकाम्या जागी टाका, महिन्याने एकदा सगळं आवरा (ठेवायला झटपट, शोधायला त्रास: LSM). तुम्ही घरात कोणती पद्धत वापरता, आणि का? तुमचं उत्तर तुमच्या “वाचन-जड की लेखन-जड” स्वभावावर अवलंबून: databases सुद्धा.

विचार करा

आरतीच्या app मध्ये एक भयंकर bug आला: दोन गिन्हाईकांनी **एकाच** क्षणी शेवटची एकच साखर-पोत ऑर्डर केली, आणि **दोघांनाही** “ऑर्डर घेतली” मिळालं: पण पोत एकच! Database कडक (SQL, ACID) असूनही असं कसं? आरती गोंधळली: “एकांत (Isolation, ACID मधलं I) म्हणजे तर दोन

व्यवहार मिसळत नाहीत ना? मग हे कसं घडलं?”

इथे ACID च्या सर्वात सूक्ष्म, सर्वात गैरसमजल्या भागाला हात लागला: race condition आणि transactions ची खरी खोली. “एकांत” म्हणजे नक्की किती एकांत, आणि तो कसा राखायचा: पुढचा chapter.

Chapter 2.4 [SPINE]: Transactions आणि Race Condition

आरतीचा “एकच पोतं, दोघांना ऑर्डर” हा bug software मधल्या सर्वात कपटी वर्गाचा आहे: **race condition** (शर्यत-दोष). आणि तो टाळणारं हत्यार म्हणजे transactions चं खरं आकलन.

Race condition म्हणजे काय. दोन माणसं एकाच क्षणी एकाच दाराने आत शिरायला बघतात: कोण आधी? “शर्यत” कोण जिंकतो यावर निकाल बदलतो: आणि तो अनिश्चित. आरतीच्या bug चं पडद्यामागचं दृश्य: दोन्ही ऑर्डर एकाच क्षणी database ला विचारतात “साखर आहे का?” -> दोघांनाही उत्तर “हो, 1 पोतं” (कारण अजून कुणीच घेतलं नाही) -> दोघेही “घेतो” म्हणतात -> दोघांनाही मिळालं. मधल्या **सूक्ष्म क्षणी** दोघांनी एकच “1 पोतं” बघितलं: हीच शर्यत. हे bugs भयंकर कारण ते **कधीतरीच** घडतात (नेमक्या एकाच क्षणी दोन विनंत्या आल्या तरच), म्हणून testing मध्ये दिसत नाहीत, आणि गर्दीच्या वेळीच फुटतात.

उपाय 1: Transaction (व्यवहार एकत्र बांधणं). ACID चं ‘A’ (Atomicity) आणि ‘I’ (Isolation) इथे कामाला. “साखर आहे का बघ + असेल तर घे + शिल्लक कमी कर”: या **तीन पायऱ्या एक अभंग गठ्ठा** म्हणून बांधायच्या (transaction). मग database हमी देते: हा गठ्ठा चालू असताना दुसरा कुणी मध्ये **घुसणार नाही** (Isolation), आणि गठ्ठा पूर्ण किंवा मुळीच नाही (Atomicity). बँकेची उपमा: “खात्यातून काढ + दुसऱ्यात टाक” हे दोन्ही किंवा काहीच नाही; मध्येच वीज गेली तरी अर्धवट नाही.

उपाय 2: कुलूप (locking). पण “मध्ये घुसणार नाही” कसं? Database एक **कुलूप** लावते: एक ऑर्डर साखरेच्या नोंदीवर काम करत असताना ती नोंद “कुलूपबंद”; दुसऱ्या ऑर्डरला **थांबावं** लागतं (कुलूप सुटपर्यंत). आता शर्यत संपली: एकच वेळी एकच. पहिल्याला पोतं मिळालं, दुसऱ्याला (थांबल्यावर) “संपलं” दिसलं: बरोबर.

पण इथे सूत्र, नेहमीप्रमाणे: कुलूप = शुद्धता विकत, वेग विकून. कुलूप लावलं की दुसऱ्याला **थांबावं** लागतं: म्हणजे हळू. आणि जास्त कुलूप = जास्त थांबे = कमी वेग. आणि एक भयंकर धोका: **deadlock** (परस्पर-कोंडी): ऑर्डर-अ ने साखरेचं कुलूप घेतलं आणि तेलाची वाट बघतोय; ऑर्डर-ब ने तेलाचं घेतलं आणि साखरेची वाट बघतोय: दोघेही एकमेकांची वाट बघत **कायमचे अडकले** (अरुंद पुलावर समोरासमोर दोन ट्रक: Book 6 त पुन्हा भेटेल). म्हणून database ना deadlock ओळखून एकाला “मागे फिर” सांगावं लागतं. निष्कर्ष: शुद्धतेसाठी कुलूप हवीत, पण जपून: जास्त कुलूप = हळू + deadlock चा धोका.

व्यावहारिक धडा: “एकाच वेळी अनेक लोक एकाच गोष्टीला हात लावतात” (शिल्लक, seat-booking, शेवटची वस्तू) तिथे race condition दबा धरून बसलेली असते. Top engineer तिथे **आधीच** transaction + योग्य कुलूप बांधतो; नवखा “असं दोघे एकाच क्षणी कसे येतील” म्हणत सोडतो, आणि

IPL-तिकीट/flash-sale च्या गर्दीत app कोसळतं. ”एकाच वेळी दोघे” हे नेहमी गृहीत धरा: Book 4 चं “अपयश गृहीत धरा” चं जुळं.

महाग चूक

”माझ्या testing मध्ये सगळं चाललं, bug नाही.” Race conditions हे testing ला सर्वात जास्त हुलकावणी देणारे bugs: कारण ते फक्त **नेमक्या एकाच क्षणी दोन विनंत्या** आल्या तरच दिसतात, आणि testing मध्ये तुम्ही एका वेळी एकच विनंती पाठवता. म्हणून “चाललं” म्हणजे “race नाही” नव्हे. जिथे सामायिक गोष्ट अनेकजण बदलतात तिथे transaction+lock **आधीच** बांधा, bug दिसायची वाट बघू नका: कारण तो गर्दीच्या, महत्वाच्या क्षणीच दिसेल: सर्वात महाग क्षणी.

नकाशावर

2.1-2.3 data modeling | SQL/NoSQL | B-tree/LSM
 2.4 transactions+race: “एकाच वेळी दोघे” गृहीत धरा; transaction (अभंग गट्टा) + lock (शुद्धता विकत, वेग+deadlock-धोका विकून)

सूत्रावर: शुद्धता (race टाळणं) फुकट नाही: कुलपं वेग खातात, deadlock चा धोका आणतात. “किती कडक एकांत हवा” हा सौदा: पैशाला कडक, like-count ला सैल.

स्वतः बघा (5 मिनिटं)

एका दाराची कल्पना करा जिथे एका वेळी एकच जण जाऊ शकतो. “कुलूप नसेल” तर दोघे धक्काबुक्की (race); “कुलूप असेल” तर एक थांबतो (सुरक्षित पण हळू). आता विचार करा: बँकेचं ATM एका वेळी एकच व्यवहार का करतं? कारण तुमच्या खात्यावर transaction+lock. तुम्ही रोज race-बचावाचा फायदा घेता, नकळत.

विचार करा

आरतीचं app आता अनेक सेवांचं बनलय: ऑर्डर-सेवा, payment- सेवा, delivery-सेवा. आणि प्रत्येक सेवेला दुसरीशी बोलावं लागतं. पण एका सेवेच्या टीमने आपल्या भागात बदल केला (एका माहितीचं नाव बदललं), आणि अचानक **दुसऱ्या** सेवेचं मोडलं: त्यांना कल्पनाही नव्हती. आरती वैतागली: “प्रत्येक सेवा दुसऱ्याशी बोलते, पण एकाने बदल केला की दुसरं फुटतं. या सेवांमध्ये काहीतरी **ठरलेला करार** नको का, जो एकतर्फी मोडता येणार नाही?”

हवाच: आणि त्या कराराचं नाव API. सेवांमधली सीमा, त्यांचा करार, आणि तो कसा टिकवायचा-
बदलायचा: पुढचा chapter, जो microservices च्या वादाचं दार उघडतो.

Chapter 2.5 [SPINE]: API: सेवांमधला करार

आरतीच्या “ठरलेला करार” या मागणीचं नाव **API** (Application Programming Interface). Book 2/4 मध्ये API भेटलं (यंत्राला हात); इथे ते सर्वात मूलभूत रूपात: **दोन software मधली सीमा आणि करार**.

API म्हणजे काय. hotel च्या menu-card ची उपमा: तुम्ही स्वयंपाकघरात घुसून “असं बनव” सांगत नाही; तुम्ही **menu** मधून मागता (ठरलेले पदार्थ, ठरलेली रीत). Menu हा तुमच्यातला आणि स्वयंपाकघरातला **करार**: तुम्हाला आत काय-कसं चालतं माहीत असायची गरज नाही (आचारी बदलला तरी menu तेच), आणि आचार्याला तुमची घाई सांभाळायची गरज नाही. API तेच: payment-सेवा म्हणते “मला असं विचारा (ऑर्डर-क्रमांक, रक्कम), मी असं उत्तर देईन (यशस्वी/अयशस्वी)”: हा करार. ऑर्डर-सेवेला payment च्या **आत** काय चालतं माहीत असायची गरज नाही.

API चा खरा फायदा: सीमा आणि स्वातंत्र्य. menu असेल तर आचारी आतली रेसिपी, भांडी, अगदी पूर्ण स्वयंपाकघर बदलू शकतो: **menu तेच राहिलं तर गिऱ्हाईकाला फरक पडत नाही**. API तेच देतं: करार (API) तोच ठेवला तर आतलं code, database, अगदी पूर्ण रचना बदलता येते, आणि वापरणाऱ्याला धक्का बसत नाही. आरतीच्या bug चं मूळ हेच होतं: एका टीमने **करारच** बदलला (menu मधला पदार्थ गायब केला), म्हणून दुसरं फुटलं. उपाय: **API हा पवित्र करार माना: तो एकतर्फी, अचानक बदलू नका**.

म्हणून API-रचनेचे नियम (करार लिहिण्याची कला): (1) **स्पष्ट आणि स्थिर**: काय मागायचं, काय मिळेल: नेमकं, बदलत नाही. (2) **जुनं तोडू नका (backward compatibility)**: नवं जोडायचं तर **जोडा**, जुनं काढू नका (जुने गिऱ्हाईक अजून जुनं वापरतात); menu त नवा पदार्थ घाला, जुना ठेवा. (3) **आवृत्त्या (versioning)**: मोठा बदल हवाच तर नवी आवृत्ती (v2), जुनी (v1) काही काळ चालू ठेवा, लोकांना हलायला वेळ द्या. (4) **चुका स्पष्ट सांगा**: “error” नाही, “ऑर्डर-क्रमांक चुकीचा” (Book 4, 4.2 चा शिकवणारा- error, इथे सेवांमध्ये). हे नियम पाळले की सेवा एकमेकांपासून **स्वतंत्र** होतात: एक बदलली तरी दुसरी शाबूत.

सूत्र इथे: API/करार = स्वातंत्र्य + स्पष्टता विकत, “पूर्ण लवचिकता” विकून. करार असेल तर आत काहीही बदला (स्वातंत्र्य), पण करार **पाळावाच** लागतो (बंधन). करार नसेल तर सगळं कुणालाही बदलता येतं (लवचिक), पण मग सगळे एकमेकांत गुंतून एकही सुरक्षित बदल अशक्य (आरतीचा bug). चांगली सीमा = चांगले शेजारी.

महाग चूक

”आतली गोष्ट आहे, दोन्ही सेवा आमच्याच, करार-बिरार कशाला: थेट एकमेकांची database वापरू.” हा सर्वात सामान्य, सर्वात घातक shortcut: एका सेवेने दुसरीच्या database ला **थेट** हात घातला की करार गेला, सीमा गेली: आता एकीचा कुठलाही आतला बदल दुसरीला अचानक फोडतो (आरतीचा bug कायमचा). नियम: **सेवांनी एकमेकांशी फक्त API ने बोलावं, थेट database ने कधीच नाही.** ही एक शिस्त म्हणजे microservices चालण्या-न-चालण्याची सीमारेषा (पुढचा chapter). Shortcut आज वेळ वाचवतो, उद्या महिने खातो.

नकाशावर

2.1-2.4 data modeling | SQL/NoSQL | B-tree/LSM | transactions
 2.5 API = सेवांमधला करार (menu); आत बदला, करार पाळा;
 जुनं तोडू नका, versioning; थेट database ला हात कधीच नाही

सूत्रावर: API हा “स्वातंत्र्य विरुद्ध बंधन” चा सौदा: करार पाळण्याचं बंधन घेतलं की आत बदलण्याचं स्वातंत्र्य मिळतं. चांगले करार चांगल्या रचनेचा पाया.

स्वतः बघा (5 मिनिटं)

कुठलंही app वापरताना “login with Google” बटण बघितलं असेल: ते app Google च्या **API** शी बोलतं (करार: “हा user खरा आहे का?”). App ला Google च्या आत काय चालतं माहीत नाही; Google ला app चं काही देणघेणं नाही: फक्त करार. एक API, दोन अनोळखी company, निर्दोष सहकार्य. करार-रचनेची ताकद तुम्ही रोज वापरता.

विचार करा

आता आरतीचा सर्वात मोठा प्रश्न: “करार (API) असतील तर मी app चे तुकडे करू शकते: ऑर्डर-सेवा, payment-सेवा, प्रत्येक वेगळी, वेगळी टीम. पण खरंच तुकडे करावेत का? एकाच मोठ्या code मध्ये सगळं (सोपं पण गर्दी) की अनेक वेगळ्या सेवा (स्वतंत्र पण गुंतागुंत)? सगळे म्हणतात microservices भारी: खरं का?”

हा Part 2 चा सर्वात मोठा, सर्वात गैरसमजलेला वाद: आणि इथे एका मोठ्या company ची **उलटी** गोष्ट आहे: Amazon ने microservices वरून परत monolith कडे जाऊन 90% खर्च वाचवला. Fashion विरुद्ध सौदा, सर्वात टोकाला. पुढचा chapter.

Chapter 2.6 [SPINE]: Monolith vs Microservices [दोन्ही बाजू + Amazon ची उलटी गोष्ट]

Part 2 चा सर्वात मोठा, सर्वात fashion-ग्रस्त वाद. आणि तो सोडवायला एक खरी, धक्कादायक गोष्ट: जिथे जगातल्या सर्वात मोठ्या company ने प्रवाहाच्या **उलट** जाऊन जिंकलं.

दोन रचना. (1) **Monolith (एकसंध):** पूर्ण app एकाच मोठ्या code मध्ये, एकत्र चालणारं. एका मोठ्या घराची उपमा: सगळ्या खोल्या एका छताखाली; आतल्या आत फिरणं सोपं-जलद, पण घर मोठं झालं की गर्दी. (2) **Microservices (अनेक सुटे):** app चे अनेक छोटे, स्वतंत्र सेवा-तुकडे, प्रत्येक वेगळा चालणारा, API ने बोलणारे (2.5). वेगवेगळ्या घरांची वसाहत: प्रत्येक स्वतंत्र, पण एका घरातून दुसऱ्यात जायला रस्ता-वाहन (network) लागतं.

Microservices काय देतं (आणि का सगळे भुलले): मोठ्या टीमला **स्वातंत्र्य**: ऑर्डर-टीम आपला तुकडा बदलते, payment-टीम आपला: एकमेकांच्या पायात न येता (2.4 चा आरतीचा वैताग सुटतो). प्रत्येक तुकडा वेगळा scale करता येतो (फक्त payment-सेवा गर्दीत वाढवा). एक तुकडा कोसळला तरी बाकी चालू (Book 4, 4.3 subagents चा दूरचा नातलग). 2015 च्या सुमारास हे इतकं गाजलं की “मोठं app = microservices” हे fashion झालं.

पण किंमत प्रचंड (जी fashion लपवते): एकाच घरात जे फुकट होतं (खोलीतून खोलीत चालणं) ते वसाहतीत **महाग** होतं: प्रत्येक सेवा-सेवा संवाद आता network वरून (हळू, latency: 1.9), फसू शकणारा (timeout/retry/circuit breaker: 1.11 आता **सगळीकडे** लागतं!). एका सोप्या कामाला दहा सेवांची साखळी: debugging भयंकर (कुठे बिघडलं?: observability अत्यावश्यक). आणि आरतीच्या race-सारखे प्रश्न आता सेवांमध्ये पसरतात (distributed transactions: भयंकर अवघड).

Microservices = टीम-स्वातंत्र्य विकत, प्रचंड गुंतागुंत विकून.

आणि आता ती उलटी गोष्ट (Amazon Prime Video, 2023): Amazon च्या Prime Video मधल्या एका टीमने त्यांची video- तपासणी सेवा microservices रचनेत बांधली होती (तुकडे, serverless). पण खर्च आणि गुंतागुंत वाढतच गेली. मग त्यांनी **उलट** केलं: तुकडे परत **एकाच process** मध्ये आणले (monolith कडे): आणि खर्च **90% घटला**, कामगिरी सुधारली. (नीट वाचा, कारण ही गोष्ट फार वेळा चुकीची सांगतात: **Amazon ने microservices सोडले नाहीत; एका टीमने एका सेवेला** तुकड्यांतून परत एकत्र आणलं, कारण त्या कामाला तुकड्यांची गरजच नव्हती, फक्त खर्च होता.) धडा सोन्याचा: **रचना fashion ने नाही, गरजेने.** त्या कामाला तुकड्यांचं स्वातंत्र्य नको होतं, म्हणून त्यांनी सौदा उलटवला: आणि प्रवाहाविरुद्ध जायची हिंमत हीच खरी अभियांत्रिकी.

निवडीची कसोटी (छापा):

Monolith निवडा:	छोटी-मध्यम टीम; सुरुवात; काम एकसंध; वेग आणि साधेपणा हवा. (बहुतेकांसाठी: हेच.)
Microservices निवडा:	मोठ्या अनेक टीमस एकमेकांच्या पायात येतात; भाग वेगवेगळे scale करावे लागतात; आणि गुंतागुंत पेलायची यंत्रणा (observability, अपयश-बचाव) आधीच आहे.
सोनेरी नियम:	"सुरुवात monolith ने; तुकडे तेव्हाच जेव्हा दुखणं (टीम-गर्दी) खरंच जाणवेल." उलट (आधीच microservices) = premature, महाग.

महाग चूक

"आम्ही आधुनिक आहोत, दिवस-1 पासून microservices." Startups ची सर्वात महाग fashion-चूक: 3 engineers, 100 users, आणि 15 microservices: प्रत्येक साध्या बदलाला दहा सेवांचा ताळमेळ, debugging नरक, आणि तुकड्यांचा फायदा (टीम-स्वातंत्र्य) शून्य (कारण टीमच लहान). Amazon-गोष्ट उलट शिकवते: गरज नसेल तर तुकडे **खर्च आहेत, फायदा नाही**. नियम: **monolith ने सुरू करा, दुखलं तर तोडा**. "तोडता येतं" (नंतर); "जोडणं" (परत एकत्र आणणं) महाग: म्हणून आधी एकत्र, नाईलाजाने वेगळं.

नकाशावर

2.1-2.5 data | SQL/NoSQL | B-tree/LSM | transactions | API
2.6 monolith vs microservices: micro = टीम-स्वातंत्र्य विकत,
गुंतागुंत विकून; Amazon Prime Video उलट गेलं (-90% खर्च);
सुरुवात monolith, तोडा गरजेला (fashion नको)

सूत्रावर: हा वाद म्हणजे सूत्राची सर्वात मोठी परीक्षा: सर्वात मोठा मोह (fashion), सर्वात मोठी लपलेली किंमत (गुंतागुंत). Amazon सारखं प्रवाहाविरुद्ध सौदा बघता येणं = सर्वोच्च नजर.

स्वतः बघा (5 मिनिटं)

तुमच्या घराचा विचार: सगळं एका घरात (monolith) सोयीचं की प्रत्येक कामाला वेगळी इमारत (microservices)? लहान कुटुंबाला एक घर पुरतं आणि सोपं; मोठ्या company ला वेगळे विभाग-इमारती लागतात, पण मग त्यांच्यात समन्वय-यंत्रणा (email, meetings = network+API). आकाराने

निर्णय बदलतो: तुमच्या app चा “आकार” ओळखा.

विचार करा

आरतीने monolith ठेवलं (शहाणपणाने). पण app मध्ये एक गरज वाढली: गिन्हाईकांना “साखर” search करता यायला हवं, आणि “साकर”, “sugar”, “साखरं” असं काहीही टाईप केलं तरी सापडायला हवं. तिचा नेहमीचा database (index सह) फक्त **नेमकं** जुळणारं शोधतो; “जवळपास” जुळणारं, स्पेलिंग-चुका सांभाळणारं शोधता येईना. “search तर index नेच होतं ना? मग हे वेगळं का?”

साधा शोध (index) आणि खरा “search” (जो अर्थ, स्पेलिंग, क्रम सांभाळतो) या वेगळ्या गोष्टी आहेत, आणि त्यामागे एक सुंदर रचना आहे: inverted index. पुढचा chapter: search आतून.

Chapter 2.7 [SPINE]: Search आतून: Inverted Index

आरतीचा प्रश्न: साधा शोध (index) आणि खरा search वेगळे कसे? उत्तर एका उलट्या युक्तीत आहे: आणि ती समजली की Google पासून app च्या search-बार पर्यंत सगळ्यामागचं तत्त्व कळतं.

साधा index वि. खरा search. database चा index (1.3): “रमेशचा फोन-नंबर -> ती नोंद.” हे **नेमकं जुळणं** शोधतं: अख्खा नंबर बरोबर हवा. पण search वेगळं मागतं: “साखर” टाईप केलं तर “साखरेचं पोतं”, “गूळ-साखर मिक्स”, “sugar free” : सगळं यायला हवं; स्पेलिंग-चूक (“साकर”) सांभाळायला हवी; आणि सर्वात जुळणारं **आधी** (क्रम). हे नेमक्या-जुळणीच्या index ला जमत नाही. यासाठी वेगळी रचना: **inverted index** (उलटा अनुक्रमणिका).

Inverted index म्हणजे काय (आणि “उलटा” का). सामान्य यादी: प्रत्येक वस्तू -> तिच्यातले शब्द (“पोतं-1: साखर, पांढरी, 1 किलो”). Search ला हे उलटं हवं: प्रत्येक **शब्द** -> तो कुठल्या वस्तूत आहे (“साखर -> पोतं-1, पोतं-5, ऑफर-3”). पुस्तकाच्या मागच्या index सारखं (1.3 ची उपमा, आता खरी): “साखर” शब्द शोधला की तो असलेल्या **सगळ्या** जागा एका झटक्यात. म्हणून “उलटा”: वस्तू-कडून-शब्द नाही, **शब्द-कडून-वस्तू**. आरती “साखर” टाईप करते -> inverted index मधून साखर असलेल्या सगळ्या वस्तू क्षणात.

खरा search जे जास्त करतो (तीन जादू): (1) **शब्द फोडणं आणि सुलभ करणं:** “साखरेचं”, “साखर”, “साखरं” : सगळं मुळाशी “साखर” वर आणणं (म्हणून वेगवेगळी रूपं जुळतात). (2) **स्पेलिंग-सहनशीलता:** “साकर” आणि “साखर” जवळचे, म्हणून जुळवणं (fuzzy). (3) **क्रम (ranking):** कुठलं आधी दाखवायचं: जास्त जुळणारं, जास्त लोकप्रिय, जवळचं दुकान: वर. Google चं संपूर्ण साम्राज्य या तिसऱ्या जादूवर (कुठलं पान आधी). यासाठी वेगळी search-यंत्रं असतात (Elasticsearch वगैरे): database नाही, search-साठीच बांधलेली.

सूत्र इथे: खरा search = लवचिक-हुशार शोध विकत, **जागा + गुंतागुंत + थोडा शिळेपणा** विकून. Inverted index वेगळी बांधावी- सांभाळावी लागते (जागा, यंत्रणा); आणि ती बहुधा मुख्य database पासून थोडी मागे असते (नवी वस्तू search मध्ये यायला क्षणभर: तोच replication-lag चा 1.5 चा आत्मा). म्हणून: **नेमकं जुळणं (फोन, क्रमांक) -> साधा index; लवचिक शोध (नाव, मजकूर) -> inverted index/search-यंत्र.** दोन वेगळी हत्यारं, दोन वेगळ्या गरजांना: पुन्हा तेच “माहितीच्या स्वभावाने निर्णय.”

महाग चूक

”Search हवा? मुख्य database वरच `LIKE %साखर%` ने शोधू” हा नवख्याचा सर्वात सामान्य फसवा shortcut: छोट्या data ला चालतो, पण तो **पूर्ण-चाळणी** (full scan: 1.3) असतो: लाखो नोंदीत प्रत्येक search म्हणजे अख्खी database चाळणं: app गुडघे टेकतं, आणि गर्दीत कोसळतं. खरा search ला inverted index/search-यंत्र लागतं: वेगळं, पण त्यासाठीच बांधलेलं. “database वर थेट LIKE search” हे flash-sale/गर्दीत मरणारं app चं सर्वात सामान्य मूळ आहे.

नकाशावर

2.1-2.6 data...monolith/microservices
 2.7 search: inverted index (शब्द -> वस्तू, उलटा); फोडणं + स्पेलिंग-सहन + क्रम(ranking); नेमकं->index, लवचिक->search-यंत्र

सूत्रावर: search ची हुशारी फुकट नाही: वेगळी यंत्रणा, जागा, थोडा शिळेपणा. “कुठला शोध हवा” (नेमका की लवचिक) ओळखून योग्य हत्यार निवडणं = नजर.

स्वतः बघा (5 मिनिटं)

Google वर मुद्दाम स्पेलिंग चुकवून शोधा (“जगातली सर्वात उंच इमारत”): Google “इमारत” समजून बरोबर देतं (fuzzy + ranking). आता फोन-contacts मध्ये अर्थ नाव टाईप करा: तेही जुळतं (inverted-सदृश). दोन्हीकडे “उलटा” शोध चालू आहे: शब्दाकडून-वस्तूकडे. एकदा हे दिसलं की search ची जादू यंत्र वाटते, चमत्कार नाही.

विचार करा

आरतीचं app आता आणखी एक गोष्ट मागतंय: delivery-boy कुठे आहे हे गिन्हाईकाला **live**, नकाशावर हलताना दिसायला हवं; आणि दुकान-गिन्हाईक यांच्यात **तात्काळ** chat. पण तिचं नेहमीचं request-response (गिन्हाईक विचारतो, server उत्तर देतो) यासाठी विचित्र वाटतं: गिन्हाईकाने दर सेकंदाला “boy कुठे?” विचारत राहायचं का? “सतत, आपोआप, दोन्ही बाजूंनी वाहणारी माहिती: हे request-response ने कसं जमणारं?”

जमणार नाही: request-response हे “विचारा-मग-उत्तर” चं तत्त्व; live/chat ला “सतत उघडा दुवा” लागतो. WhatsApp हे कोट्यवधी लोकांसाठी कसं करतं, तेच तत्त्व: पुढचा chapter: real-time.

Chapter 2.8 [SPINE]: Real-Time: WhatsApp चं तत्त्व

आरतीच्या live-tracking आणि chat च्या गरजेतून real-time communication ची संकल्पना. आणि इथे एक थक्क करणारी खरी गोष्ट आहे जी “मोठं म्हणजे प्रचंड टीम” हा गैरसमज मोडते.

Request-response ची मर्यादा. आतापर्यंतचं सगळं “गिऱ्हाईक विचारतो, server उत्तर देतो” (Book 1). पण chat/live-tracking वेगळं: server ला **स्वतःहून**, गिऱ्हाईकाने न विचारता, माहिती पाठवायची आहे (“boy आता इथे पोहोचला”, “नवा message आला”). दर सेकंदाला गिऱ्हाईकाने “काही नवीन?” विचारत राहणं (polling) म्हणजे दर मिनिटाला पोस्टमनला “पत्र आलं का?” विचारायला जाणं: वाया, महाग, आणि तरी उशिरा.

उपाय: सतत उघडा दुवा (persistent connection). पोस्टमनकडे जाण्याऐवजी घरात एक थेट **फोन-लाइन** ठेवायची: काही आलं की पोस्टमन **स्वतः** त्या लाइनवरून सांगतो. Software मध्ये याला WebSocket (आणि तत्सम) म्हणतात: गिऱ्हाईक आणि server मध्ये एक दुवा **उघडाच** राहतो; दोन्ही बाजू केव्हाही, एकमेकांना, ढकलू शकतात (push). नवा message आला की server लगेच ढकलतो; boy हलला की location लगेच पोहोचते: विचारावं लागत नाही. Chat, live-tracking, notifications: सगळं याच तत्त्वावर.

पण सूत्र, नेहमीप्रमाणे: उघडे दुवे = तात्काळपणा विकत, संसाधनं विकून. प्रत्येक उघडा दुवा server ची थोडी memory-लक्ष धरून ठेवतो. एक-दोन ठीक; पण **कोट्यवधी** लोक एकाच वेळी? कोट्यवधी उघडे दुवे सांभाळणं म्हणजे प्रचंड आव्हान: आणि इथे ती थक्क करणारी गोष्ट येते.

WhatsApp चं तत्त्व (खरी, तपासलेली गोष्ट): 2015 त WhatsApp **~90 कोटी** (900 million) लोकांना real-time chat देत होतं: कोट्यवधी उघडे दुवे, अब्जावधी messages रोज. आणि त्यांची engineering-टीम होती फक्त **~50 माणसं**. पन्नास. हे कसं? त्यांनी एक तत्त्व कट्टरपणे पाळलं: **सोपेपणा आणि योग्य हत्यार**. त्यांनी एका खास भाषेत (Erlang: लाखो सतत-दुवे सांभाळण्यासाठीच बनलेली) बांधलं, कमीत कमी features ठेवली, आणि प्रत्येक server कडून जास्तीत जास्त (एका server वर ~20 लाख दुवे) काढलं. धडा सोन्याचा (आणि Book 4, 9.3 शी थेट पूल): **मोठेपणा म्हणजे प्रचंड टीम/गुंतागुंत नव्हे; योग्य हत्यार + कट्टर सोपेपणा = 50 माणसं 90 कोटी लोकांना सेवा देतात.** आरतीसाठी दिलासा: तिलाही प्रचंड टीम लागणार नाही; योग्य निवडी लागतील.

महाग चूक

”Real-time हवं? सगळं WebSocket ने, सतत-उघडं करू!” उलट चूक: **सगळंच** real-time करणं म्हणजे विनाकारण कोट्यवधी उघडे दुवे, प्रचंड खर्च, गुंतागुंत. बहुतेक माहितीला real-time लागतच नाही: ऑर्डर-यादी, भाव: हे मागितल्यावर आलं तरी चालतं (request-response, स्वस्त). Real-time फक्त जिथे **तात्काळपणा खरंच महत्त्वाचा**: chat, live-location, alerts. नियम: **डिफॉल्ट request-response (स्वस्त, सोपं); real-time फक्त जिथे सेकंद मोजतात**. Fashion ने सगळं “live” करणारे app-मालक server- bill बघून रडतात.

नकाशावर

2.1-2.7 data...search

2.8 real-time: उघडा दुवा (WebSocket), server स्वतः ढकलतो;
तात्काळपणा विकत, संसाधनं विकून; WhatsApp 50 माणसं/90
कोटी = योग्य हत्यार + सोपेपणा; real-time फक्त जिथे सेकंद मोजतात

सूत्रावर: तात्काळपणा हा सर्वात मोहक अनुभव, पण संसाधन- महाग सौदा. आणि WhatsApp दाखवतं की सौदा हुशारीने केला तर मोठेपणा थोड्या माणसांतही पेलतो: सोपेपणा हाच सर्वात मोठा scaling-मंत्र.

स्वतः बघा (5 मिनिटं)

WhatsApp मध्ये कुणीतरी टाइप करतंय तेव्हा “typing...” दिसतं: तुम्ही काहीच विचारलं नाही, तरी माहिती **तुमच्याकडे आली** (server ने ढकलली: real-time). आता gmail उघडा आणि जुनी mail बघा: ती तुम्ही **मागितल्यावर** आली (request-response). दोन वेगळी तत्त्वं, दोन वेगळ्या गरजा. कुठे “ढकललं”, कुठे “मागितलं”: ओळखायला शिका.

विचार करा

आरतीचं app आता प्रचंड आणि गुंतागुंतीचं. आणि एक अपरिहार्य गरज आली: तिला एका जुन्या माहिती-रचनेत मोठा बदल करायचाय (2.1 चा निर्णय आता बदलायचाय), पण त्या रचनेत **लाखो** जिवंत नोंदी आहेत आणि app **चालूच** आहे: एक क्षणही बंद करता येत नाही. “चालत्या, भरलेल्या database ची रचना बदलायची, आणि app बंदही करायचं नाही: हे कसं शक्य आहे? चालत्या गाडीचं चाक कसं बदलणार?”

हाच Part 2 चा शेवटचा, आणि सर्वात नाजूक विषय: migrations. चालत्या system मध्ये बदल करण्याची कला, जिथे एक चूक लाखो नोंदी नष्ट करू शकते (GitLab ची रात्र आठवा). पुढचा chapter.

Chapter 2.9 [SPINE]: चालत्या गाडीचं चाक: Migrations

आरतीच्या “चालत्या database ची रचना बदलणं” या पेचावर हा chapter. आणि इथे software मधली सर्वात नाजूक कला आहे: कारण एक चूक = लाखो नोंदी नष्ट (GitLab ची रात्र, 1.2).

समस्येचं खरं रूप. Code बदलणं सोपं (जुनं फेकलं, नवं टाकलं). पण **लाखो जिवंत नोंदी** असलेल्या database ची रचना बदलणं (नवा रकाना, बदललेलं नातं, तुकडे करणं): हे धावत्या रेल्वेचे रूळ बदलण्यासारखं: गाडी चालूच आहे, प्रवासी आतच आहेत, आणि थांबवता येत नाही. याला **migration** म्हणतात, आणि त्याचं सोनेरी तत्त्व एकच: **कधीच एका मोठ्या उडीने नाही; नेहमी छोट्या, उलटवता-येणाऱ्या पावलांनी.**

सुरक्षित migration ची पद्धत (छापून ठेवा): विस्तार-मग-आकुंचन. समजा आरतीला एका रकान्याचं नाव/रचना बदलायचीय. **चुकीचा मार्ग:** एका रात्री सगळं एकदम बदलणं (जुनं काढून नवं): चुकलं तर सर्वनाश, आणि जुनं-code अजून जुनं-रकाना शोधत असेल तर लगेच सगळं फुटतं. **बरोबर मार्ग, चार पावलं:**

1. विस्तार: नवा रकाना जोडा (जुना ठेवूनच). आता दोन्ही आहेत. काहीच मोडत नाही (जुनं code जुनं वापरतं).
2. दुहेरी-लेखन: नवं code दोन्हीकडे लिहितं (जुना + नवा), जुनी माहिती हळूहळू नव्यात कॉपी करते. app चालूच.
3. वळण: सगळं तयार झाल्यावर code ला नवा रकाना वापरायला सांगा. तपासा. चुकलं तर लगेच जुन्याकडे परत (सुरक्षित).
4. आकुंचन: सगळं नव्यावर स्थिरावल्यावर, खात्री पटल्यावर, जुना रकाना काढा. आता स्वच्छ.

प्रत्येक पाऊल **छोटं आणि उलटवता-येणारं**: कुठल्याही क्षणी चुकलं तर मागे फिरता येतं, app बंद पडत नाही, नोंदी सुरक्षित. हे मंद वाटतं (चार पावलं, दिवस-आठवडे), पण तेच वाचवतं: घाईची एक मोठी उडी = GitLab ची रात्र.

आणि दोन लोखंडी नियम (Book 4, 4.5 चे भाऊ): (1) **आधी backup**, आणि तो तपासलेला (न तपासलेला backup = नसलेला backup: GitLab). (2) **आधी थोड्यांवर, मग सगळ्यांवर** (एका छोट्या भागावर migration चालवून बघा, मग पूर्ण: CrowdStrike चा 85-लाख-यंत्रांचा धडा, 1.13). Migration म्हणजे शस्त्रक्रिया: घाई नाही, तयारी सगळी, आणि परत-फिरण्याचा रस्ता नेहमी उघडा.

सूत्र इथे: सुरक्षित migration = जोखीम-घट विकत, वेग विकून. एका रात्रीची उडी जलद पण जीवघेणी; चार-पावलांचा रस्ता मंद पण सुरक्षित. आणि इथे सौदा नेहमी सुरक्षेच्या बाजूने: कारण वेगाची किंमत “थोडा उशीर”, जोखमीची किंमत “लाखो नोंदी + धंदा.” असला सौदा एकतर्फीच.

महाग चूक

“छोटा बदल आहे, रात्री एकदम करून टाकू, कोण बघतंय.” Migrations मधली सर्वात महाग, सर्वात सामान्य चूक: “छोटा बदल” लाखो नोंदींवर चालतो, आणि एक चुकीची ओळ सगळं पुसते (GitLab चा engineer “छोटं” काम करत होता). आणि रात्री-एकट्याने-घाईत म्हणजे backup नाही, तपासणी नाही, परत-फिरण्याचा रस्ता नाही. नियम: ****कुठलाही database-बदल = शस्त्रक्रिया = विस्तार-आकुंचन**

- backup + थोड्यांवर-आधी. **** “छोटा बदल” असं database-जगात काही नसतं; लाखो नोंदींवर सगळंच मोठं.**

नकाशावर

2.1-2.8 data...real-time

2.9 migrations: चालत्या गाडीचं चाक; विस्तार->दुहेरी-लेखन-> वळण->आकुंचन (छोटी, उलटवता-येणारी पावलं); backup तपासलेला + थोड्यांवर आधी; जोखीम-घट विकत, वेग विकून

सूत्रावर: इथे सूत्र सुरक्षेकडे झुकतं: काही सौदे एकतर्फी असतात. लाखो नोंदींशी खेळताना “वेग विरुद्ध सुरक्षा” मध्ये सुरक्षा नेहमी जिंकते: कारण चुकीची किंमत भरून येत नाही.

स्वतः बघा (5 मिनिटं)

घरातलं जुनं कपाट नव्या जागी हलवायचा विचार करा: सगळं एकदम उचलून टाकणं (काहीतरी फुटेल) vs एक-एक कप्पा हलवत, जुनं रिकामं होईपर्यंत नवं भरत जाणं (सुरक्षित, मंद). Migration ची विस्तार-आकुंचन पद्धत नेमकी दुसरी: जुनं-नवं काही काळ दोन्ही, मग हळूच जुनं काढणं. रोजच्या आयुष्यातही सुरक्षित बदलाचं तत्त्व तेच.

विचार करा

Part 2 संपत आलाय. आरतीने आता खूप शिकलंय: data-रचना, database-निवड, transactions, API, monolith/microservices, search, real-time, migrations. पण एक कौशल्य उरलंय जे या सगळ्याला जोडतं: “जेव्हा मला कुणाच्या तरी system चा नकाशा (architecture diagram) दाखवला जातो, तो **वाचता** यायला हवा: यात कुठे cache आहे, कुठे धोका आहे, कुठला सौदा केलाय: हे एका नजरेत ओळखता यायला हवं. ते कसं शिकायचं?”

हाच Part 2 चा कळस: सगळे तुकडे एकत्र बघून “नकाशा वाचणं.” कारण top engineer ची खरी ओळख नवं बांधण्यात इतकीच दुसऱ्याची रचना **वाचून सौदे-धोके ओळखण्यात** आहे. विभागाचा शेवटचा chapter.

Chapter 2.10 [SPINE]: Architecture Diagram वाचण्याची कला

Part 2 चा कळस: दुसऱ्याची रचना **वाचणं**. कारण top engineer ची अर्धी किंमत नवं बांधण्यात, अर्धी दुसऱ्याचा नकाशा बघून **सौदे आणि धोके एका नजरेत ओळखण्यात** आहे. हे कौशल्य मुलाखतीत, code-review त, आणि रोजच्या निर्णयांत सतत लागतं.

नकाशा म्हणजे काय. architecture diagram: पेट्या (सेवा, database, cache) आणि बाण (कोण कुणाशी बोलतं). Part 1 चं “आरतीचं पूर्ण चित्र” (1.15) हाच एक नकाशा होता. आता तो **वाचायचा**: म्हणजे पेट्या-बाण बघून मागचे सौदे-धोके ताडायचे.

वाचनाची सहा प्रश्नांची नजर (छापून ठेवा):

1. एका विनंतीचा रस्ता कुठला? बोट ठेवून पूर्ण प्रवास काढा: कुठून येते, कुठे-कुठे जाते, उत्तर कसं परत. (1.1, 1.15)
2. एकच अपयश-बिंदू कुठे? एक पेटी पडली तर सगळं थांबतं अशी जागा आहे का? (एकच database, एकच सेवा सगळे वापरतात: तिथे धोका.) (1.1, 1.8)
3. डेटा कुठे, कुठली database? पैसे SQL त आहेत ना? सैल-प्रचंड माहिती NoSQL त? की सगळं एकाच कॉबलंय? (2.2)
4. Cache कुठे, “जुनं कधी”? cache दिसतंय का? मग शिळेपणाचा प्रश्न विचारला गेलाय का? (1.4)
5. सेवा-सेवा कशा बोलतात? API ने (करार, सुरक्षित) की एकमेकांच्या database ला थेट हात (धोका!)? (2.5)
6. गर्दी/अपयश कुठे फुटेल? load balancer आहे? queue आहे (धक्का शोषायला)? timeout/retry? गर्दीत कुठली पेटी आधी पडेल? (1.7, 1.8, 1.11)

ही सहा-प्रश्नांची नजर कुठल्याही नकाशाला लावा: दोन मिनिटांत तुम्ही रचनेची ताकद आणि तडे दोन्ही सांगू शकाल. हेच “वाचता येणं.”

आणि सर्वात प्रौढ प्रश्न: सौदे कुठे केलेत, आणि ते बरोबर आहेत का? प्रत्येक नकाशा म्हणजे सौद्यांचा गट्टा (पूर्ण Part 1-2). नकाशा वाचताना विचारा: “इथे cache आहे म्हणजे यांनी वेग निवडला, ताजेपणा सोडला: या माहितीला ते चालेल का? इथे microservices आहेत म्हणजे टीम-स्वातंत्र्य निवडलं, गुंतागुंत घेतली: यांची टीम एवढी मोठी आहे का?” **सौदा दिसणं आणि तो या परिस्थितीला योग्य आहे का हे ताडता येणं** : हीच पूर्ण Book 5 ची कमाई, एका कौशल्यात.

हे कुठे वापराल: (1) **मुलाखत:** “design a system like X”: आता तुम्ही पेढ्या-बाण काढून सौदे बोलू शकता (हेच system- design मुलाखतीचं हृदय). (2) **नवं काम:** दुसऱ्याने बांधलेलं समजून घ्यायला. (3) **AI-युगात (Book 4 शी पूल):** agents code लिहितील, पण **रचना बरोबर आहे का, सौदे योग्य आहेत का** हे ठरवणारी नजर तुमची: आणि तीच leverage (Book 4, 9.2: निर्णय-नेमकेपणा). Code टाईप करणं स्वस्त झालं; **नकाशा वाचणं- आखणं** महाग राहिलं.

महाग चूक

”नकाशा भारी-गुंतागुंतीचा दिसतोय, म्हणजे यांनी चांगलं काम केलंय.” उलट बहुधा खरं: **गुंतागुंत हे कौशल्याचं लक्षण नाही, अनेकदा तो न-विचारलेल्या सौद्यांचा ढीग असतो.** खरा जाणकार सर्वात **सोपी** रचना शोधतो जी काम करते (WhatsApp चं 50- माणसं तत्त्व, 2.8; Amazon चं परत-monolith, 2.6). नकाशा वाचताना “किती भारी” ने भारावू नका; “कमीत कमी तुकड्यांत काम होतंय का, प्रत्येक तुकडा **कशासाठी** आहे” विचारा. जिथे एखादी पेटी “का आहे” चं उत्तर नाही, तिथे बहुधा fashion आहे, गरज नाही.

नकाशावर

Part 2 पूर्ण:

data modeling | SQL/NoSQL | B-tree/LSM | transactions |
API | monolith/microservices | search | real-time | migrations
2.10 नकाशा वाचणं: सहा प्रश्न (रस्ता, अपयश-बिंदू, database, cache, सेवा-संवाद, गर्दी); सौदा दिसणं + योग्य का ताडणं

सूत्रावर: पूर्ण Part 2 एका कौशल्यात उतरतं: नकाशा वाचणं म्हणजे **सौदे वाचणं**. आणि सौदे वाचता येणं म्हणजे रचना बांधता येणं: कारण दोन्ही एकच नजर.

स्वतः बघा (5 मिनिटं)

Book 5 च्या 1.15 मधलं “आरतीचं पूर्ण चित्र” परत उघडा आणि वरची सहा-प्रश्नांची नजर त्याला लावा: एका विनंतीचा रस्ता काढा, अपयश-बिंदू शोधा, cache कुठे बघा. दोन मिनिटांत तुम्ही तुमचाच नकाशा “वाचाल”: आणि हेच कौशल्य उद्या दुसऱ्याच्या (किंवा मुलाखतीतल्या) नकाशाला लागेल.

विचार करा

Part 2 संपला. आरतीकडे आता रचनेचे सगळे निर्णय आहेत, आणि ते वाचायची नजरही. पण एक गोष्ट उरलीये, आणि तीच कदाचित सर्वात महत्त्वाची: “हे सगळं मला **एकटीला** येतं. पण मोठं software 30-50 लोक **मिळून** बांधतात. ते एकमेकांच्या पायात न येता, चुका न करता, वेगात कसे बांधतात? WhatsApp चे 50 लोक, Zerodha चे 30: त्यांची **काम करण्याची पद्धत** काय आहे? रचना कळली; पण **elite टीम** कशी काम करते?”

हाच Part 3 चा विषय, आणि तो रचनेइतकाच महत्त्वाचा: git, testing, review, incident-संस्कृती: elite teams ची शिस्त, आणि मग WhatsApp/UPI/Zerodha/Netflix च्या खऱ्या इमारती उलगडून. Part 3: शिस्त आणि खऱ्या इमारती.

BOOK 5, PART 2 चा शेवट

जे तुमच्याकडे आता आहे

data modeling	एक सत्य एका जागी (normalize) vs वेगासाठी कॉपी (denormalize); नाती आधी; रचना आधी
SQL vs NoSQL	SQL=शिस्त+नाती, NoSQL=आकार+वेग; डिफॉल्ट SQL, NoSQL गरजेला (fashion नको)
B-tree vs LSM	वाचन-जलद (SQL) vs लेखन-जलद (NoSQL); तोच वाचन/लेखन सौदा, पायात
transactions/race	"एकाच वेळी दोघे" गृहीत धरा; transaction+lock (शुद्धता विकत, वेग+deadlock विकून)
API	सेवांमधला करार (menu); आत बदला, करार पाळा; थेट database ला हात कधीच नाही
monolith/micro	micro=टीम-स्वातंत्र्य विकत, गुंतागुंत विकून; Amazon उलट गेलं (-90%); सुरुवात monolith
search	inverted index (शब्द->वस्तू); नेमकं->index, लवचिक->search-यंत्र; database वर LIKE नको
real-time	उघडा दुवा; तात्काळपणा विकत, संसाधनं विकून; WhatsApp 50/90कोटी = योग्य हत्यार+सोपेपणा
migrations	विस्तार->दुहेरी-लेखन->वळण->आकुंचन; backup तपासलेला; थोड्यांवर आधी; जोखीम-घट विकत
नकाशा वाचणं	सहा प्रश्न; सौदा दिसणं + योग्य का ताडणं = Book 5 ची कमाई

स्वतःची परीक्षा (4 प्रश्न, बोलून; उत्तरं खाली)

1. नवं app: database डिफॉल्ट SQL की NoSQL, आणि का?

SQL: नाती+शुद्धता फुकट मिळतात, नंतर सोडता येतात; NoSQL ची खरी गरज (प्रचंड-सैल-वेगवान) असेल तरच तिकडे. Fashion नको. (2.2)

2. दोन सेवा आमच्याच आहेत; एकीने दुसरीच्या database ला थेट हात घातला तर?

करार (API) तुटला, सीमा गेली: आतला कुठलाही बदल दुसरीला फोडेल. सेवांनी फक्त API ने बोलावं. (2.5)

3. Amazon Prime Video ची गोष्ट काय शिकवते?

रचना fashion ने नाही, गरजेने: त्यांनी microservices वरून परत monolith कडे जाऊन 90% खर्च वाचवला. प्रवाहाविरुद्ध सौदा बघणं = नजर. (2.6)

4. चालत्या database ची रचना बदलायची: पद्धत?

विस्तार-आकुंचन: नवं जोडा (जुनं ठेवून) -> दुहेरी-लेखन -> वळण -> जुनं काढा; प्रत्येक पाऊल छोटं, उलटवता-येणारं; backup आधी. (2.9)

पुढच्या भागाची चाहूल

आरतीचा शेवटचा प्रश्न Part 3 उघडतो: **elite टीम कशी काम करते?** git (इतिहास + review), testing चे थर, CI/CD, debugging = निदान-कला, blameless incident-संस्कृती, आणि सुरक्षित deploy (canary). मग खऱ्या इमारती उलगडून: WhatsApp, UPI, Zerodha, Netflix: आणि AI-युगात हे सगळं का उरतं. Part 3: शिस्त आणि खऱ्या इमारती.

BOOK 5, PART 2 चा MINI-GLOSSARY

API	दोन software मधला करार (menu-तत्त्व) (2.5)
B-tree	वाचन-जलद database-रचना (SQL) (2.3)
deadlock	दोन व्यवहार एकमेकांची वाट बघत अडकणं (2.4)
denormalization	वेगासाठी माहिती मुद्दाम कॉपी करणं (2.1)
inverted index	शब्द -> वस्तू; खऱ्या search चा पाया (2.7)
LSM-tree	लेखन-जलद database-रचना (NoSQL) (2.3)
migration	चालत्या database ची रचना सुरक्षित बदलणं (2.9)
monolith	पूर्ण app एकाच code मध्ये (2.6)
microservices	app चे अनेक स्वतंत्र सेवा-तुकडे (2.6)
normalization	एक सत्य एका जागी; पुनरावृत्ती टाळणं (2.1)
NoSQL	सैल-लवचिक, आकार-वेगासाठी database (2.2)
race condition	एकाच वेळी दोघांनी हात घातल्याने अनिश्चित निकाल (2.4)
SQL/relational	शिस्त+नाती+ACID असलेली database (2.2)
transaction	अनेक पायऱ्या एक अभंग गट्टा (2.4)
versioning	API ची नवी आवृत्ती, जुनी चालू ठेवून (2.5)
WebSocket	सतत-उघडा दुवा; real-time साठी (2.8)

THE BLUEPRINT

Book 5, Part 3: Elite शिस्त आणि खऱ्या इमारती

मागच्या भागाचा शेवट: आरतीला रचना कळली, वाचता आली. पण शेवटचा प्रश्न उरला: **elite टीम कशी काम करते?** 30-50 लोक एकमेकांच्या पायात न येता, चुका न करता, वेगात कसे बांधतात? हा भाग त्याचा: git, testing, review, incident-संस्कृती: आणि मग जगातल्या खऱ्या इमारती उलगडून (WhatsApp, UPI, Zerodha, Netflix). शेवटी: AI-युगात हे सगळं का उरतं, अंतिम परीक्षा, आणि नजरेचे checklists.

सूत्र (शेवटच्या भागासाठी)

मोठेपणा फुकट नसतो.
प्रत्येक रचना-निर्णय म्हणजे काहीतरी विकून काहीतरी विकत घेणं.
Part 3 चा भर: रचना बांधणारी माणसं आणि त्यांची शिस्त.

कपाटावर एक नजर

Book 1 The Machine	Book 2 The Thinking Machine
Book 3 The Land Game	Book 4 The Harness
Book 5 The Blueprint	<- हे, Part 3: शिस्त + इमारती

Part 2 ची उजळणी: 4 प्रश्न (बोलून; उत्तरं खाली)

1. नवं app: database डिफॉल्ट काय?

SQL: नाती+शुद्धता फुकट, नंतर सोडता येतात. NoSQL खरी गरज असेल तरच. Fashion नको.
(2.2)

2. दोन सेवा कशा बोलाव्यात?

फक्त API ने (करार); एकमेकांच्या database ला थेट हात कधीच नाही. (2.5)

3. Monolith की microservices, सुरुवातीला?

Monolith; तुकडे तेव्हाच जेव्हा टीम-गर्दीचं दुखणं खरंच जाणवेल. Amazon उलट जाऊन 90% वाचवले. (2.6)

4. Migration ची सुरक्षित पद्धत?

विस्तार-आकुंचन: छोटी, उलटवता-येणारी पावलं; backup आधी. (2.9)

या भागात काय

Elite teams ची शिस्त (git, testing, CI/CD, debugging, blameless postmortem, canary), मग चार खऱ्या इमारती उलगडून, मग AI-युगाची रचना (Book 4 शी पूल), आणि अंतिम परीक्षा + design-नजरेचे checklists. रचना कळली; आता ती बांधणारी संस्कृती.

विभाग 3: Elite Teams ची शिस्त

रचना कितीही चांगली असो, ती बांधणारी माणसं शिस्तीने काम करत नसतील तर सगळं कोसळतं. हा विभाग त्या शिस्तीचा: git, testing, review, deploy, आणि सर्वात महत्वाचं: चूक झाल्यावर काय करायचं (incident-संस्कृती). **हे तुमच्या धंद्यात कुठे येईल:** तुम्ही एकटे बांधाल किंवा agents सोबत (Book 4), पण ही शिस्त तीच: कारण ती “अनेक हात, एक दिशा, कमी चुका” ची शिस्त आहे: आणि agent हाही एक हात आहे. Elite teams वेगळे नसतात कारण ते हुशार असतात; ते वेगळे असतात कारण त्यांची **पद्धत** चुका पचवते.

Chapter 3.1 [SPINE]: Git: इतिहास-यंत्र आणि Review-संस्कृती

Elite शिस्तीचा पाया एका साध्या पण क्रांतिकारी साधनात आहे: **git**. आरतीला 30 engineers सोबत काम करायचंय; त्या सगळ्यांचं code एकत्र, चुका न होता, कसं जमवायचं? git चं उत्तर दोन भागांत: इतिहास-यंत्र, आणि review-संस्कृती.

भाग 1: git = time-machine. Book 4 त हे थोडं भेटलं (4.5); आता पूर्ण. git प्रत्येक बदलाची नोंद ठेवतं: कोणी, कधी, काय, का बदललं: आणि कुठल्याही जुन्या क्षणी परत जाता येतं. लेखकाच्या draft-वहीची उपमा: प्रत्येक खाडाखोड जपलेली, म्हणून “तीन दिवसांपूर्वीचं वाक्य बरं होतं” वाटलं तर परत आणता येतं. Software मध्ये हे अमूल्य: एक बदल app फोडतो, git ने **लगेच** आधीच्या चालत्या क्षणी परत (Knight Capital ची \$460M-रात्र, 1.13: त्यांच्याकडे सुरक्षित परत-जाणं नव्हतं). “परत फिरता येणं” (Book 4, 4.5 चा brake) git देतं: म्हणून engineers **न घाबरता** प्रयोग करतात.

भाग 2: अनेक हात, एक code (branches + merge). 30 लोक एकाच code वर एकदम लिहिले तर गोंधळ (2.4 चा race, माणसांत!). git ची युक्ती: प्रत्येकाने आपली वेगळी **शाखा** (branch) घ्यायची: मुख्य code ला हात न लावता, स्वतःच्या प्रतीवर काम. काम झालं, तपासलं, की ती शाखा मुख्यात **जोडायची** (merge). प्रत्येक झाडाची वेगळी फांदी, नीट तपासून खोडाला जोडायची. आता 30 लोक समांतर काम करतात, एकमेकांच्या पायात न येता.

भाग 3 (सर्वात महत्वाचा): review-संस्कृती (code review). आणि इथे elite teams चं खरं रहस्य आहे. कुठलीही फांदी मुख्यात जोडण्याआधी, **दुसरा engineer ती वाचतो आणि मंजूर करतो** (pull request review). का? कारण: (1) दुसऱ्याची नजर तुमची चूक पकडते (तुम्हाला न दिसलेली); (2) ज्ञान पसरतं (दोघांना आता ते code कळतं, एकावर अवलंबित्व नाही); (3) दर्जा टिकतो (review पास होणारंच आत). Google, आणि जगातल्या प्रत्येक elite टीमचा लोखंडी नियम: **कुठलंही code एकट्याने मुख्यात जात नाही; दुसऱ्याची मोहोर लागतेच.** (Book 4, 5.3 चा “दुसऱ्याची झडती” इथे माणसांत; आणि AI-युगात reviewer कधी agent ही असू शकतो, पण मोहोरीचं तत्त्व तेच.)

सूत्र इथे: review = दर्जा + ज्ञान-प्रसार विकत, थोडा वेग विकून. एकट्याने थेट टाकणं जलद, पण चुका आणि एकाकी-ज्ञान. Review थोडं मंद, पण चुका आधीच पकडल्या, ज्ञान वाटलं. आणि हा सौदा elite teams नेहमी दर्जाच्या बाजूने करतात: कारण “थोडा उशीर” स्वस्त, “production मधली चूक” महाग.

महाग चूक

“घाई आहे, review वगळून थेट टाकतो, नंतर बघू.” Elite teams मध्ये review वगळणं ही जवळपास गुन्हा-मानली-जाणारी गोष्ट, आणि कारण साधं: **review न झालेलं code हे न-तपासलेलं गृहीत** आहे, आणि ते बहुधा गर्दीच्या-महत्त्वाच्या क्षणी फुटतं. “घाई” हे review वगळायचं कारण नाही; उलट घाईच्या वेळी review **जास्त** महत्त्वाचं (घाईत चुका जास्त). एकट्याने-थेट- टाकणं आज 10 मिनिटं वाचवतं, उद्या production-bug चे 10 तास खातं. नियम: **मोहोर नेहमी, अपवाद नाही.**

नकाशावर

Part 3 चा नकाशा (वाढत जाईल):

3.1 git: इतिहास-यंत्र (परत फिरता येणं) + branches (समांतर काम) + review (दुसऱ्याची मोहोर: दर्जा+ज्ञान-प्रसार)

सूत्रावर: elite शिस्तीची पहिली वीट: review चा सौदा (थोडा वेग देऊन दर्जा घेणं). माणसांची चूकही “अपयश गृहीत धरा” (Book 4) चा भाग: दुसऱ्याची नजर हा माणसातला brake.

स्वतः बघा (5 मिनिटं)

तुम्ही लिहिलेलं काहीही (message, अर्ज) पाठवण्याआधी दुसऱ्याला “वाचून बघ” म्हटल्यावर त्यांनी चूक/सुधारणा दाखवल्याचं आठवा: तुम्हाला न दिसलेली. तीच code review ची ताकद: **तुमचे डोळे तुमच्या चुकीला सरावलेले असतात; दुसऱ्याचे नाहीत.** म्हणून elite teams एकमेकांचं सगळं वाचतात, न चुकता.

विचार करा

आरतीच्या टीमने review सुरू केलं, चुका घटल्या. पण तरी काही चुका review मधून निसटतातच (माणूस थकतो, घाईत बघतो). आणि त्या production मध्ये पोहोचल्यावर गिऱ्हाईकाला दिसतात. आरती विचारते: “माणसाची नजर चुकतेच. पण एकच चूक पुन्हा- पुन्हा होऊ नये, आणि जुनं जमणारं नव्या बदलाने मोडू नये: याची **आपोआप** खात्री कशी करायची? दर वेळी 30 लोकांनी सगळं हाताने तपासणं अशक्य आहे.”

हवी ती आपोआप-तपासणी: आणि तिचं नाव testing. माणसाची नजर एकदाची; testing ची नजर दर वेळी, न थकता, न विसरता. Book 4 च्या evals चा (5.1) हा software-भाऊ. पुढचा chapter.

Chapter 3.2 [SPINE]: Testing चे थर

आरतीच्या “आपोआप, न थकता तपासणी” या मागणीचं नाव testing. Book 4 त हे agent साठी evals म्हणून भेटलं (5.1); इथे ते software साठी, आणि त्याला वेगळे थर आहेत: प्रत्येक थर वेगळी चूक पकडतो.

Testing म्हणजे काय. code ची **आपोआप परीक्षा**: “हे code या-इनपुटला हे-उत्तर देतं का?” एकदा लिहिली की **दर वेळी** चालते: माणूस थकतो, विसरतो; test न थकता, न विसरता, प्रत्येक बदलावर तपासते. गुणवत्ता-नियंत्रकाची उपमा: कारखान्यात प्रत्येक माल तपासणारा, न कंटाळता, तोच निकष लावणारा. Testing हा elite teams चा तो नियंत्रक.

तीन थर (प्रत्येक वेगळी चूक पकडतो):

1. Unit test (सुटा भाग): एक छोटा तुकडा एकटा तपासणं.
“भाव-मोजणीचं function 2 किलो x 40 = 80 देतं का?”
जलद, भरपूर, स्वस्त. पाया.
2. Integration test (जोड): दोन-तीन भाग मिळून तपासणं.
“ऑर्डर-सेवा + database + payment मिळून नीट चालतात का?”
कमी, थोडे मंद, पण खरे-जगाजवळ.
3. End-to-end test (पूर्ण): गिऱ्हाईकासारखं पूर्ण प्रवास.
“app उघडून, साखर शोधून, ऑर्डर करून, पैसे भरून: सगळं जमतं का?” सर्वात कमी, सर्वात मंद, सर्वात खरे.

थरांचं तत्त्व (testing pyramid): खाली भरपूर, वर थोडे. भरपूर unit tests (जलद-स्वस्त, रोज हजारो वेळा चालतात), मध्यम integration, थोडे end-to-end (मंद-महाग). का? कारण unit tests सेकंदात चालतात, end-to-end मिनिटं खातात: सगळंच end-to-end केलं तर testing इतकं मंद की कुणी चालवणारच नाही. पाया रुंद, कळस अरुंद: हीच शहाणी रचना.

सर्वात मोठा फायदा: घसरण-रोध (regression). आजची चूक दुरुस्त केली + तिची test लिहिली, तर उद्या कुणीतरी तेच पुन्हा मोडलं की **test लगेच ओरडते**: production आधी. Book 4, 5.5 चा “जुनं जमणारं नव्याने बिघडू नये” इथे शब्दशः. Elite teams चा नियम: **bug सापडला की आधी त्याची test लिहा (म्हणजे तो पुन्हा कधीच परत येणार नाही), मग दुरुस्त करा.** प्रत्येक चूक एकदाच; test कायमचा पहारा.

सूत्र इथे: testing = आत्मविश्वास + घसरण-रोध विकत, “आधीचा वेळ” विकून. Test लिहायला वेळ लागतो (आजचा खर्च); पण मग प्रत्येक बदल **न घाबरता** करता येतो (test राखतो: Book 4, 5.5 ची हिंमत). Test नसलेलं मोठं code म्हणजे प्रत्येक बदलाला भीती; test असलेलं म्हणजे धीट प्रयोग. वेग testing मुळे मरत नाही, वाढतो: फक्त सुरुवातीचा खर्च दिसतो, पुढची बचत नाही.

महाग चूक

”Testing म्हणजे वेळ वाया; code तर चालतंय.” सर्वात महाग अल्पदृष्टी. Test नसलेलं code सुरुवातीला जलद वाटतं, पण लवकरच प्रत्येक बदल भीतीदायक होतो (”हे बदललं तर काय मोडेल?”), आणि टीम बदल करणं टाळते (2.11 चा ”चालतंय ते मोडू नको” चा भाऊ): app गोठतं. उलट, test असलेलं code वर्षानुवर्ष धीटपणे बदलता येतं. Test हा ”वेळ वाया” नाही, तो **भविष्यातल्या वेगाचा विमा** आहे. जगातलं प्रत्येक टिकाऊ मोठं software भरपूर testing वर उभं आहे: अपवाद नाही.

नकाशावर

- 3.1 git: इतिहास + branches + review
- 3.2 testing: तीन थर (unit जलद-भरपूर / integration / e2e मंद-थोडे); घसरण-रोध; bug -> आधी test मग दुरुस्ती; वेगाचा विमा

सूत्रावर: testing चा सौदा वेळेत उलटतो: आजचा खर्च, उद्याची मुक्ती. Elite teams दीर्घ नजरेने हा सौदा करतात; नवखे आजच्या वेगाच्या मोहात तो टाळतात आणि उद्या गोठतात.

स्वतः बघा (5 मिनिटं)

तुम्ही एखादं महत्वाचं गणित/हिशेब करताना ”एकदा तपासून बघू” म्हणून उलटा हिशेब करता (80 / 40 = 2 किलो, बरोबर): ती एक मानसिक unit test आहे. आता कल्पना करा हे तपासणं **आपोआप, दर वेळी** झालं तर किती निश्चित. Software चं testing तेच, फक्त यंत्राने, हजारो वेळा, न थकता.

विचार करा

आरतीच्या टीमकडे आता git, review, testing आहे. पण तरी एक अडचण: एका engineer ने बदल केला, review झालं, tests पास झाले: पण ते **गिन्हाईकांपर्यंत पोहोचवायला** (deploy) दर वेळी कुणीतरी हाताने, रात्री जागून, अनेक पायऱ्या करायचं: आणि तिथेच चुका (एक server विसरणं: Knight Capital!). ”बदल तयार आहे, पण तो सुरक्षित, आपोआप, चुकल्याशिवाय गिन्हाईकांपर्यंत कसा न्यायचा?”

हवी ती आपोआप, सुरक्षित वाहतूक: build -> test -> deploy ची यंत्र-साखळी. तिचं नाव CI/CD, आणि ती elite teams च्या वेगाचं रहस्य आहे. पुढचा chapter.

Chapter 3.3 [SPINE]: CI/CD आणि सुरक्षित Deploy

आरतीच्या “आपोआप, सुरक्षित वाहतूक” या मागणीचं नाव CI/CD. आणि त्यासोबत deploy सुरक्षित करण्याच्या युक्त्या: ज्यांच्या गैरहजेरीने Knight Capital आणि CrowdStrike कोसळले (1.13).

CI/CD म्हणजे काय (साध्या भाषेत). दोन जोडलेल्या साखळ्या: **CI (Continuous Integration):** कुणीही code बदललं की **आपोआप:** build (जोडणी) + सगळे tests चालवा (3.2) + review-दार (3.1). पास तरच पुढे. **CD (Continuous Deployment/Delivery):** पास झालेलं code **आपोआप, सुरक्षितपणे** गिन्हाईकांपर्यंत. कारखान्याच्या assembly-line ची उपमा: कच्चा माल (code-बदल) एका टोकाला; प्रत्येक टप्प्यावर आपोआप तपासणी-दार; बाहेर पडतं ते तयार, तपासलेलं उत्पादन. हाताने-रात्री-जागून चं काम यंत्राने, दर वेळी सारखं, चुकल्याशिवाय.

पण deploy हा सर्वात धोक्याचा क्षण: तीन सुरक्षा-युक्त्या. नवं code गिन्हाईकांपर्यंत नेणं म्हणजे चालत्या विमानाचं engine बदलणं: इथेच सर्वात मोठे अपघात (Knight, CrowdStrike). म्हणून elite teams तीन युक्त्या वापरतात:

1. Canary (कोळशाच्या खाणीतला पक्षी): नवं code आधी फक्त 1-2% गिन्हाईकांना द्या. काही बिघडलं का बघा. ठीक असेल तरच हळूहळू सगळ्यांना. CrowdStrike ने हे केलं नाही: 85 लाख यंत्रांना एकदम सदोष update = इतिहासातला सर्वात मोठा IT-बिघाड. एक canary त्यांना वाचवता आला असता.
2. Blue-green (दोन रंग): जुनं (blue) चालू ठेवून नवं (green) बाजूला तयार करा; तयार झाल्यावर वाहतूक हळूच green कडे वळवा. बिघडलं तर लगेच blue कडे परत (एका सेकंदात).
3. Feature flag (स्विच): नवं feature code मध्ये आहे पण "बंद" स्विचवर; हवं तेव्हा थोड्यांसाठी "चालू", बिघडलं तर कोड न बदलता लगेच "बंद". प्रयोग आणि सुरक्षा एकत्र.

तिन्हींचं एक तत्त्व (Book 4, 4.5 + 7.4 चे भाऊ, आणि 1.13 च्या प्रलयांचा थेट धडा): **कुठलाही बदल सगळ्यांना एकदम देऊ नका; आधी थोड्यांवर, परत-फिरण्याचा रस्ता उघडा ठेवून.** हेच elite deploy चं हृदय.

आणि इथे एक थक्क करणारा विरोधाभास (elite teams चं खरं रहस्य): जे teams जास्त वेळा deploy करतात (दिवसातून अनेकदा!) ते **कमी** अपघात करतात. कसं? कारण छोटे-वारंवार बदल सुरक्षित असतात (एक छोटा बदल बिघडला तर शोधणं-दुरुस्त करणं सोपं); मोठे-कचित बदल

धोकादायक (शेकडो बदल एकदम, काहीतरी फुटलं तर कशाने ते शोधणं अशक्य). Knight Capital चा मोठा-कचित deploy विरुद्ध elite चा छोटा-वारंवार. धडा: **वेग आणि सुरक्षा विरोधी नाहीत; छोटे-वारंवार-आपोआप बदल दोन्ही देतात.**

महाग चूक

”Deploy म्हणजे मोठा प्रसंग: महिन्यातून एकदा, रात्री, सगळं एकदम, सगळे जागून.” हीच जुनी, धोकादायक पद्धत: मोठे-कचित- हाताने deploys = जास्त चुका, अवघड निदान (Knight ची रात्र). Elite उलट: **छोटे, वारंवार, आपोआप, canary सह.** “deploy भीतीदायक आहे” ही भावना म्हणजे तुमची deploy-पद्धत चुकीची आहे याचं लक्षण: सुरक्षित पद्धतीत deploy इतका सामान्य होतो की भीती उरत नाही. भीती हे सुधारणेचं आमंत्रण आहे, सहन करण्याची गोष्ट नाही.

नकाशावर

3.1-3.2 git | testing

3.3 CI/CD: आपोआप build+test+deploy साखळी; canary (थोड्यांना आधी) + blue-green + feature flag; छोटे-वारंवार = सुरक्षित; वेग आणि सुरक्षा एकत्र

सूत्रावर: इथे सूत्राला सुखद उलथापालथ: छोटे-वारंवार deploys मध्ये वेग **आणि** सुरक्षा दोन्ही मिळतात: कारण जोखीम छोटी राहते. काही ठिकाणी शहाणी रचना सौद्याचं गणितच बदलते.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनवरची apps आपोआप, वारंवार update होतात ना (तुम्हाला कळतही नाही)? ते CI/CD. आणि कधीकधी एखादं नवं feature “काही जणांना दिसतं, काहींना नाही” (feature flag/canary): company आधी थोड्यांवर तपासतेय. तुम्ही रोज elite deploy- शिस्तीच्या दुसऱ्या बाजूला उभे असता, नकळत.

विचार करा

आरतीकडे आता पूर्ण शिस्त आहे. पण एका रात्री, सगळं असूनही, app कोसळलंच: canary मधून निसटलेला एक bug. गिन्हाईकं संतापली. दुसऱ्या दिवशी टीम-मीटिंगमध्ये आरतीला मोह झाला: “कोणी केलं हे? कोणाची चूक?” पण तिला जाणवलं की दोषी शोधल्याने पुढच्या वेळी लोक चुका **लपवतील**, सांगणार नाहीत. “चूक झाली की काय करायचं: दोषी शोधायचा, की काहीतरी वेगळं?”

हाच elite teams आणि सामान्य teams मधला सर्वात मोठा फरक: चूक झाल्यावरची **संस्कृती**. दोषी शोधणं की रचना शिकणं. या एका निवडीवर टीम टिकते किंवा कुजते. पुढचा chapter: incident-संस्कृती.

Chapter 3.4 [SPINE]: Incident-संस्कृती: दोष नको, धडा हवा

आरतीच्या “दोषी शोधायचा की काहीतरी वेगळं” या प्रश्नावर हा chapter: आणि हा पूर्ण Book 5 मधला सर्वात महत्त्वाचा असू शकतो, कारण तो tech चा नाही, **माणसांचा** आहे: आणि माणसंच तर रचना बांधतात.

दोन संस्कृती, दोन भविष्यं. चूक (incident) झाल्यावर टीम दोन रस्त्यांपैकी एक घेते: (1) **दोष-संस्कृती:** “कोणी केलं? त्याला जबाबदार धरा.” परिणाम: लोक घाबरतात, चुका **लपवतात**, जोखीम टाळतात, प्रयोग थांबतो, आणि सर्वात वाईट: तीच चूक पुन्हा होते (कारण कुणी उघड बोललंच नाही, म्हणून रचना शिकली नाही). (2) **blameless संस्कृती (दोष-विरहित):** “व्यक्ती नाही, **रचना** का फसली? या चुकीला कुठली रचनात्मक फट कारणीभूत, आणि ती कशी बुजवायची?” परिणाम: लोक उघड बोलतात, प्रत्येक चूक रचना बळकट करते, आणि तीच चूक पुन्हा होत नाही.

Blameless postmortem (शव-विच्छेदन, दोषाशिवाय). Elite teams प्रत्येक मोठ्या incident नंतर एक लिखित शव-विच्छेदन करतात: **काय घडलं, का घडलं (रचनेच्या पातळीवर), आणि पुन्हा होऊ नये म्हणून रचनेत काय बदललं:** कुणाचं नाव-दोष नाही. आठवा: 1.13 मधल्या Amazon, Facebook, CrowdStrike या सगळ्यांनी **जाहीर**, प्रामाणिक postmortems लिहिली: “आमच्या रचनेत ही फट होती, आम्ही ती अशी बुजवली”: engineer ला बळीचा बकरा न करता. आणि Book 4 (1.7, 5.2) ची तीच शिकवण: **चूक ही context आहे, शिकण्याचं इंधन: लपवायची नाही, वाचायची.**

मूळ तत्त्व: चांगली माणसं वाईट रचनेत चुका करतात. Knight Capital चा engineer मूर्ख नव्हता; रचनेत “एक server विसरणं शक्य” ही फट होती (1.13). GitLab चा engineer निष्काळजी नव्हता; रचनेत “चुकीच्या server वर rm चालवणं सोपं + backup न-तपासलेले” या फटी होत्या (1.2). **व्यक्तीला दोष देणं म्हणजे रचनेतली खरी फट न बघणं:** आणि मग ती फट पुढच्या (चांगल्या) माणसाकडूनही तीच चूक करवते. म्हणून प्रश्न कधीच “कोणी?” नाही, नेहमी “रचनेत काय, आणि कसं बदलू?” (Book 4, 4.1 ची Hashimoto-सवय इथे टीम-पातळीवर: चूक -> रचनेत कायमची दुरुस्ती).

आरतीचा निर्णय: तिने दोषी शोधला नाही. तिने विचारलं: “हा bug canary मधून का निसटला? आपली canary-तपासणी कुठे कमी पडली?” उत्तर: canary फक्त 10 मिनिटं चालवत होते, हा bug 30 मिनिटांनी दिसे. दुरुस्ती: canary-वेळ वाढवला + या प्रकारची test जोडली. एक चूक, एक रचनात्मक सुधारणा, शून्य दोष: आणि टीमने शिकलं की **चूक सांगणं सुरक्षित आहे:** म्हणून पुढे ते जास्त उघड, जास्त धीट झाले.

महाग चूक

”जबाबदारी निश्चित केली पाहिजे; कोणीतरी उत्तरदायी हवं.” हे बरोबर **वाटतं** पण घातक: दोष-संस्कृती अल्पकाळात “शिस्त” वाटते, दीर्घकाळात **विष**: लोक चुका लपवतात (म्हणून त्या मोठ्या होऊन फुटतात), प्रयोग टाळतात (म्हणून नवं काही होत नाही), आणि उत्तम लोक सोडून जातात (भीतीच्या वातावरणातून). जबाबदारी **रचनेची** असावी, व्यक्तीची नाही. “उत्तरदायित्व” म्हणजे “रचना दुरुस्त करण्याची सामूहिक जबाबदारी”, “बळीचा बकरा शोधणं” नाही. ही एक सांस्कृतिक निवड टीम टिकवते किंवा कुजवते.

नकाशावर

3.1-3.3 git | testing | CI/CD

3.4 incident-संस्कृती: दोष नको (लपवणं, भीती), blameless postmortem हवं (रचना का फसली, कशी बदलू); चांगली माणसं वाईट रचनेत चुकतात; प्रश्न “कोणी?” नाही, “रचनेत काय?”

सूत्रावर: इथे सूत्र माणसांत उतरतं: दोष-मुक्त संस्कृती म्हणजे “अल्पकालीन-समाधान (बळी) विकून दीर्घकालीन-शिक्षण विकत घेणं.” Elite teams हा सौदा नेहमी शिक्षणाच्या बाजूने करतात.

स्वतः बघा (5 मिनिटं)

तुमच्या घरात/कामात कधी चूक झाल्यावर “कोणी केलं” पेक्षा “पुन्हा होऊ नये म्हणून काय बदलू” विचारल्याचं आठवा: फरक जाणवला का? दोष दिल्यावर माणूस बचावात्मक होतो, लपवतो; रचना विचारल्यावर उघड, सहकार्यशील. हेच तत्त्व 50-माणसांच्या टीमपासून घरापर्यंत सारखं. आजपासून एका चुकीला “कोणी” ऐवजी “काय बदलू” विचारून बघा.

विचार करा

आरतीची टीम आता खरोखर elite आहे: शिस्त, संस्कृती, सगळं. आणि तिला उत्सुकता वाटली: “आम्ही जे शिकलो: सोपेपणा, शिस्त, सौदे-नजर, दोष-मुक्त संस्कृती: हेच जगातल्या सर्वात मोठ्या, सर्वात यशस्वी companies वापरतात का? WhatsApp चे 50 लोक, UPI चे अब्जावधी व्यवहार, Zerodha ची 30-लोकांची टीम: यांच्या **आत** काय आहे? त्यांच्या रचना उलगडून बघता येतील का?”

हाच पुढचा विभाग, आणि Book 5 चा कळस: चार खऱ्या इमारती उलगडून. आणि तुम्हाला आश्चर्य वाटेल: त्या सगळ्यांचं रहस्य “भारी-गुंतागुंत” नसून **कट्टर सोपेपणा** आहे. विभाग 4: खऱ्या इमारती.

विभाग 4: खऱ्या इमारती

सिद्धांत पुरे; आता प्रत्यक्ष इमारती. चार खऱ्या, जगप्रसिद्ध systems उलगडून बघू: WhatsApp, UPI, Zerodha, Netflix. प्रत्येकातून Part 1-3 चे सौदे आणि शिस्त **प्रत्यक्ष** कसे वापरले ते दिसेल. **हे तुमच्या धंद्यात कुठे येईल:** हे नकाशा- वाचनाचे (2.10) खरे व्यायाम आहेत; आणि एक समान धागा दिसेल जो पूर्ण पुस्तकाचं सार आहे: **सर्वात यशस्वी systems सर्वात हुशार नसतात, सर्वात सोपी असतात.** आकडे तपासलेले.

Chapter 4.1 [SPINE]: इमारत: WhatsApp: 50 माणसं, 90 कोटी लोक

पहिली इमारत, आणि सर्वात थक्क करणारी. 2015 त WhatsApp ~90 कोटी (900 million) लोकांना real-time chat देत होतं, अब्जावधी messages रोज: आणि engineering-टीम ~50 माणसं. (2014 त Facebook ने ~\$19 अब्जांना विकत घेतलं तेव्हा एकूण ~55 कर्मचारी, ~45 कोटी users: आकडे तपासलेले.) हे कसं? उत्तर पूर्ण पुस्तकाचं सार आहे.

रहस्य 1: कट्टर सोपेपणा (feature नकार). WhatsApp ने जे केलं नाही ते जास्त महत्वाचं: जाहिराती नाहीत, games नाहीत, फालतू features नाहीत, रंगीबेरंगी गुंतागुंत नाही. फक्त: संदेश पोहोचवा, विश्वासाने, जलद. एक काम, कट्टरपणे. कमी features = कमी code = कमी bugs = कमी माणसं. Book 4, 9.3 चा “एकट्याचा कारखाना” इथे 50-लोकांचा: सोपेपणा हाच सर्वात मोठा scaling- मंत्र.

रहस्य 2: योग्य हत्यार (Erlang). त्यांनी एक खास भाषा वापरली (Erlang): जी लाखो सतत-दुवे (2.8 चे WebSocket-सारखे) सांभाळण्यासाठीच दशकांपूर्वी (टेलिफोन-यंत्रणांसाठी) बनली होती. एका server वर ~20 लाख जोडण्या. योग्य कामाला योग्य हत्यार: “cool” भाषा नाही, या कामाला बनलेली भाषा (2.2 चा “fashion नको, सौदा बघा” इथे भाषेच्या निवडीत).

रहस्य 3: नकाशा उलगडून (Part 1-2 चे सौदे). WhatsApp मध्ये तुम्हाला दिसेल: (1) messages कायमचे साठवत नाहीत (पोहोचले की server वरून पुसतात): storage स्वस्त, गुंतागुंत कमी: एक धाडसी सौदा (2.1 चा data-model, टोकाला नेलेला). (2) real-time साठी सतत-दुवे (2.8). (3) प्रचंड-वेगवान, सैल लेखनासाठी योग्य database (LSM-कुळ, 2.3). प्रत्येक निर्णय “या कामाला काय लागतं” वर, “काय भारी दिसतं” वर नाही.

समान धागा (पूर्ण पुस्तकाचा): WhatsApp ने कुठलीही “जादूई भारी रचना” वापरली नाही. त्यांनी एक काम निवडलं, ते कट्टर सोपेपणाने केलं, योग्य हत्यार वापरलं, आणि प्रत्येक सौदा जाणून केला. म्हणून 50 माणसं 90 कोटींना पुरली. हा तुमच्यासाठी (आणि आरतीसाठी) सर्वात मोठा दिलासा: मोठेपणाला प्रचंड टीम/ पैसा/गुंतागुंत लागत नाही; सोपेपणा आणि योग्य निवडी लागतात. Book 4 चं leverage-सूत्र (9.3) इथे 2015 तच खरं ठरलं होतं.

महाग चूक

“मोठं app म्हणजे भरपूर features, भरपूर माणसं, भरपूर गुंतागुंत.” WhatsApp हे उलट सिद्ध करतं: **features वाढवणं सोपं, नाकारणं अवघड:** आणि नाकारण्यातच खरी शिस्त. प्रत्येक जोडलेलं feature = जास्त code + bugs + माणसं + गुंतागुंत (Book 4, 4.2/4.4 चा “किती tools” तोच धडा). यशस्वी

होण्यासाठी जास्त करावं लागतं हा भ्रम; बहुधा **कमी, पण उत्तम** करावं लागतं. “काय करू” पेक्षा “काय करू नको” हा कठीण आणि किमती प्रश्न.

नकाशावर

विभाग 3 (शिस्त): git | testing | CI/CD | incident-संस्कृती

4.1 WhatsApp: 50/90कोटी; कट्टर सोपेपणा (feature नकार) + योग्य हत्यार (Erlang) + जाणीवपूर्वक सौदे; मोठेपणा = सोपेपणा, प्रचंड-टीम नाही

सूत्रावर: WhatsApp पूर्ण पुस्तकाचं सूत्र जगून दाखवतं: प्रत्येक सौदा जाणून, सर्वात सोपी रचना निवडून, त्यांनी मोठेपणा थोड्या माणसांत पेलला. सोपेपणा हा सर्वात कमी लेखलेला superpower.

स्वतः बघा (5 मिनिटं)

WhatsApp उघडा आणि मोजा: त्यात किती “features” आहेत? मुख्य: संदेश, call, status. संपलं जवळपास. आता एखादं गचडलेलं app आठवा (डझनभर features, गोंधळ). कुठलं तुम्ही रोज वापरता, कुठलं टिकलं? सोपेपणाची ताकद तुमच्या स्वतःच्या फोनवर दिसते.

विचार करा

WhatsApp एका company चं. पण आरतीला उत्सुकता: “एका company ला 50 माणसं पुरली. पण **UPI** तर पूर्ण देशाची यंत्रणा: अनेक बँका, अब्जावधी व्यवहार, सगळे एकमेकांशी जोडलेले. एका company चं system आणि अख्ख्या देशाचं system: यात रचनेचा फरक काय? आणि इतक्या वेगवेगळ्या बँका एका जाळ्यात कशा बांधल्या?”

उत्तम प्रश्न: कारण UPI ही एका company ची इमारत नाही, तर **अनेकांना जोडणारा करार** (2.5 चा API, राष्ट्रीय पातळीवर!). UPI ची इमारत, आणि तिने जगाला काय शिकवलं: पुढचा chapter.

Chapter 4.2 [SPINE]: इमारत: UPI: करार जो देश जोडतो

दुसरी इमारत, आणि सर्वात वेगळी: कारण UPI ही एका company ची नाही, तर **अख्ख्या देशाची** यंत्रणा. आणि तिचं रहस्य पूर्ण Part 2 च्या एका chapter मध्ये आहे: API म्हणजे करार (2.5).

आकडे आधी (तपासलेले): UPI जुलै 2026 त एका महिन्यात **~23-24 अब्ज (billion)** व्यवहार हाताळत होतं: दिवसाला ~76 कोटी. जगातली सर्वात मोठी real-time payment यंत्रणा, आणि ती **कुठल्याही एका company ची नाही.**

रहस्य: UPI हे app नाही, करार आहे. इथे 2.5 चा API-धडा राष्ट्रीय पातळीवर बघा. UPI च्या आधी: प्रत्येक बँकेचं वेगळं app, वेगळी यंत्रणा, एका बँकेतून दुसऱ्यात पैसे पाठवणं म्हणजे दिव्य. UPI ने काय केलं? एक **समान करार** (common API) बनवला: “पैसे पाठवायचे तर असं विचारा, असं उत्तर मिळेल”: आणि सगळ्या बँकांना त्या एका कराराशी जोडलं. आता Google Pay, PhonePe, Paytm, कुठलंही app, कुठलीही बँक: सगळे **एकाच कराराची भाषा** बोलतात. Book 4 चा MCP (4.4) आणि Book 5 चा API (2.5): दोन्हीचं हे राष्ट्रीय, भौतिक रूप. **पायाभूत करार सरकारने/सामायिक संस्थेने बनवला आणि खुला केला, म्हणून वरचा प्रचंड बाजार बहरला.**

नकाशा उलगडून (Part 1 चे सौदे, देश-पातळीवर): UPI मध्ये तुम्हाला दिसेल: (1) **प्रचंड वाचन-लेखन:** म्हणून sharding (1.6: व्यवहार बँक/प्रदेशानुसार वाटलेले), replication (1.5), load balancing (1.8): सगळं देश-आकारात. (2) **पैसा = कडक CAP-C** (1.10): फूट पडली तर “थांबा, पुन्हा प्रयत्न करा” (तुम्ही अनुभवलयं!): चुकीचा व्यवहार कधीच नाही: बँकिंग नेहमी अचूकतेच्या बाजूने. (3) **idempotency** (1.7): तोच व्यवहार दोनदा गेला तरी पैसे एकदाच कापले (म्हणून तुम्ही निर्धास्त refresh करता). प्रत्येक सौदा या पुस्तकातलाच, पण अब्जावधींच्या प्रमाणात.

UPI ची outage-कथा (प्रामाणिक, तपासलेली): एप्रिल 2025 त UPI ला मोठा बिघाड झाला (काही तास). कारण? बँकांच्या apps कडून “व्यवहार झाला का?” या तपासणी-विनंत्यांचा **पूर** (काही जुने व्यवहार पुन्हा-पुन्हा तपासत होते): यंत्रणा गुदमरली. उपाय (1.11 चा धडा प्रत्यक्ष): NPCI ने या तपासणी-विनंत्यांवर **मर्यादा** घातली (rate limiting). धडा: अब्जावधींची यंत्रणाही retry-पुराने (1.11: बेभान retry = आगीत तेल) कोसळू शकते; आणि उपाय तोच जो आपण शिकलो: मर्यादा, throttle.

महाग चूक

”मोठं यश म्हणजे एका company ने सगळं स्वतः बांधणं, मालकी ठेवणं.” UPI उलट शिकवतं: **सर्वात मोठा प्रभाव खुल्या कराराने येतो, बंद मालकीने नाही.** जर प्रत्येक बँकेने स्वतःचं बंद app ठेवलं असतं, तर आजचं UPI नसतं. करार खुला केला, मालकी सोडली, म्हणून सगळे जोडले गेले आणि प्रचंड मूल्य निर्माण झालं (Book 4, 2.6/4.4 चा “पायाभूत गोष्ट फुकट, वरचा बाजार जिंका” तोच धडा). कधीकधी सर्वात मोठं जिंकणं म्हणजे **नियंत्रण सोडणं.**

नकाशावर

विभाग 3 (शिस्त): `git...incident`

4.1 WhatsApp: सोपेपणा + योग्य हत्यार

4.2 UPI: करार जो देश जोडतो (API राष्ट्रीय); खुला करार >
बंद मालकी; पैसा=CAP-C, idempotency, sharding देश-आकारात;
retry-पुराने अब्जावधींची यंत्रणाही कोसळते (rate limit)

सूत्रावर: UPI दाखवतं की सर्वात मोठा सौदा कधी “मालकी विकून पोहोच विकत घेणं” असतो: खुला करार म्हणजे नियंत्रण सोडून प्रचंड-मूल्य घेणं. सौद्याचं सर्वात उदार रूप.

स्वतः बघा (5 मिनिटं)

पुढच्या वेळी UPI ने पैसे पाठवताना विचार करा: तुमचं app (PhonePe/GPay), समोरच्याचं वेगळं app, दोन वेगळ्या बँका: आणि तरी सेकंदात पैसे पोहोचतात. हे शक्य आहे कारण सगळे **एकाच कराराची भाषा** बोलतात. तुम्ही रोज 2.5 चा API-धडा अब्जावधींच्या प्रमाणात वापरता, नकळत.

विचार करा

WhatsApp खासगी company, UPI राष्ट्रीय यंत्रणा: दोन टोकं. आरतीला आता एक वेगळाच प्रश्न: “या दोन्हींनी वेगवेगळ्या रचना वापरल्या. पण एक भारतीय company जी शेअर-बाजाराचं काम करते: Zerodha: ती फक्त **30 लोकांच्या** tech-टीमने भारतातली सर्वात मोठी broker झाली, म्हणे. आणि त्यांचं तत्त्व ‘कंटाळवाणं tech वापरा’ असं आहे: नवीन-भारी काहीच नाही. हे कसं जमतं? सगळे तर नवीन tech च्या मागे धावतात.”

Zerodha ची गोष्ट पूर्ण पुस्तकाच्या सूत्राचा कळस आहे: “कंटाळवाणं tech जिंकतं” हे विधान तुम्हाला आता खोलवर पटेल. पुढचा chapter.

Chapter 4.3 [SPINE]: इमारत: Zerodha: कंटाळवाणं Tech जिंकतं

तिसरी इमारत, आणि भारतीय अभिमानाची: Zerodha. भारतातली सर्वात मोठी stock-broker, कोट्यवधी गिन्हाईक, अब्जावधींचे रोजचे व्यवहार: आणि tech-टीम फक्त ~30 माणसं (CTO कैलाश नाथ). त्यांचं तत्त्व पूर्ण पुस्तकाच्या सूत्राचा कळस आहे, आणि ते धक्कादायक वाटतं: **”कंटाळवाणं tech वापरा.”**

”Boring technology” म्हणजे काय. सगळे नवीन, चकचकीत, “cool” tech च्या मागे धावतात (नवी database, नवी भाषा, नवी fashion). Zerodha उलट: **जुनं, सिद्ध, कंटाळवाणं, पण भरवशाचं** tech वापरा. त्यांची मुख्य database? PostgreSQL (जुनी, सिद्ध: 2.2). नवीन-चमकदार NoSQL नाही, microservices चा डोंगर नाही: बहुतांश एकसंध, साधी रचना (2.6 चा monolith, अभिमानाने). का? कारण **कंटाळवाणं tech = माहीत असलेले सौदे, कमी आश्चर्य, कमी रात्री-जागरणं.**

यामागचं खोल शहाणपण (सूत्राशी थेट): नवीन tech म्हणजे **न-माहीत सौदे:** ते कुठे फुटेल, कसं वागेल, कुणालाच पूर्ण माहीत नाही (कारण नवीन). जुनं, कंटाळवाणं tech म्हणजे **हजारो लोकांनी वर्षानुवर्ष वापरून सगळे सौदे-धोके उघड केलेले:** तुम्हाला आश्चर्य वाटणार नाही. Zerodha पैशाचं काम करते (2.2: पैसा = कडक शुद्धता): तिथे “आश्चर्य” म्हणजे गिन्हाईकाचे पैसे गेले. म्हणून ते जाणीवपूर्वक **कंटाळवाणं-पण-भरवशाचं** निवडतात. Book 2 च्या money-map ची भाषा: **दुर्मिळता आणि भरवसा यांना भाव आहे; नवेपणाला नाही.**

अजून एक Zerodha-वैशिष्ट्य: कमी माणसं, कमी घाई. ~30 लोकांची टीम, “deadline नसलेली” संस्कृती (त्यांच्या स्वतःच्या लेखनातून), आणि tech-debt (जुनं-गचडलेलं code) आक्रमकपणे साफ करणं. आणि ते बरंचसं **FOSS** (मुक्त-स्रोत) जगाला परत देतात. WhatsApp चा सोपेपणा-मंत्र (4.1) इथे भारतीय broker- रूपात: **कमी माणसं, सोपं-सिद्ध tech, घाई नाही, दर्जा आधी.**

हे तुमच्यासाठी सर्वात महत्त्वाचं का: कारण हे तुमच्या मार्गाशी थेट जुळतं. तुम्हाला Google/Netflix ची अब्ज-डॉलर रचना बांधायची नाही; तुम्हाला **भरवशाचं, टिकाऊ, थोड्या माणसांत चालणारं** काहीतरी बांधायचंय. आणि त्यासाठी Zerodha हा आदर्श: नवीन- भारी tech चा मोह टाळा, सिद्ध-कंटाळवाणं निवडा, सोपं ठेवा, दर्जावर भर. **”कंटाळवाणं” हा अपमान नाही; तो प्रौढतेचा दागिना आहे.**

महाग चूक

”आम्ही आधुनिक आहोत, नवीनतम tech वापरतो.” नवीन tech वापरणं हे कौशल्याचं लक्षण **वाटतं**, पण अनेकदा ते अनुभवाचं लक्षण असतं: अनुभवी engineer ला माहीत असतं की नवीन tech = न-माहीत सौदे = रात्री-जागरणं. “आम्ही latest वापरतो” चा अभिमान अनेक startups ना महागात पडला: fashion-ग्रस्त निवडींनी अस्थिर, अवघड-देखभाल systems, गिन्हाईकांचे गेलेले पैसे. Zerodha उलट: कंटाळवाणं, स्थिर, म्हणून सर्वात मोठं. **नवेपणाच्या मोहाला “हे या समस्येला खरंच लागतं का, की फक्त cool आहे” ने तपासा:** पूर्ण पुस्तकाचा “fashion नको, सौदा बघा” इथे शिखरावर.

नकाशावर

विभाग 3 (शिस्त): git...incident

4.1 WhatsApp: सोपेपणा 4.2 UPI: खुला करार

4.3 Zerodha: कंटाळवाणं tech जिकतं (~30 लोक); जुनं-सिद्ध = माहीत सौदे, कमी आश्चर्य; पैशाला भरवसा > नवेपणा; कमी माणसं, घाई नाही, दर्जा आधी

सूत्रावर: Zerodha सूत्राचा सर्वात प्रौढ अर्थ दाखवतं: सर्वात शहाणा सौदा कधी “नवेपणाचा मोह विकून भरवसा विकत घेणं” असतो. Boring म्हणजे known trade-offs; known म्हणजे शांत झोप.

स्वतः बघा (5 मिनिटं)

तुमच्या आयुष्यातलं एक “कंटाळवाणं पण भरवशाचं” साधन आठवा (जुनी सायकल, जुना फोन जो कधीच बंद पडत नाही). आणि एक “चकचकीत पण त्रासदायक” (नवं gadget जे सतत बिघडतं). महत्वाच्या, रोजच्या कामाला तुम्ही कुठलं निवडता? Zerodha पैशाच्या कामाला तेच निवडतं: कंटाळवाणं-भरवशाचं. प्रौढ निवड नेहमी भरवशाच्या बाजूने.

विचार करा

WhatsApp सोपं, UPI खुलं, Zerodha कंटाळवाणं: तिन्हीनी सोपेपणा-भरवसा निवडला. पण आरतीला शेवटचा प्रश्न: “हे सगळे ‘अपयश टाळायचा’ प्रयत्न करतात. पण Netflix बदल ऐकलंय की ते **मुद्दाम स्वतःचे servers पाडतात**: ‘Chaos Monkey’ नावाचं काहीतरी जे यादृच्छिकपणे यंत्रं बंद करतं. अपयश टाळायचं सोडून मुद्दाम अपयश घडवायचं? हे वेडेपणा नाही का?”

वेडेपणा नाही, तर सर्वात प्रगल्भ शहाणपण: आणि तो Book 4 च्या red-teaming (7.4) चा infrastructure-भाऊ आहे. मुद्दाम- अपयशातून टिकाव कसा येतो: शेवटची इमारत, Netflix.

Chapter 4.4 [SPINE]: इमारत: Netflix: मुद्दाम अपयश घडवणारे

शेवटची इमारत, आणि सर्वात विरोधाभासी: Netflix मुद्दाम स्वतःचे servers पाडतं. आरतीला हे वेडेपणा वाटलं; प्रत्यक्षात हे Book 5 च्या सर्वात प्रगल्भ धड्याचं मूर्त रूप आहे: **अपयश गृहीत धरा (1.11), आणि ते खरं होण्याआधी स्वतः घडवून बघा.**

Chaos Monkey ची गोष्ट (तपासलेली). ~2010-11 च्या सुमारास Netflix cloud कडे स्थलांतरित होत होतं (एका जुन्या database- बिघाडाने त्यांना धडा शिकवला होता). त्यांनी एक विचित्र यंत्र बनवलं: **Chaos Monkey**: जे कामाच्या वेळेत, यादृच्छिकपणे, त्यांचेच चालू servers **बंद पाडतं**. मुद्दाम. रोज. का हा वेडेपणा?

तर्क सुंदर आहे: “अपयश होणारच (1.11: मोठं system कधीच पूर्ण चालू नसतं). मग ते **आम्ही निवडलेल्या वेळी, आमच्या देखरेखीखाली** घडावं: की रात्री 3 वाजता, गिन्हाईकांच्या गर्दीत, आम्ही झोपेत असताना घडावं?” Chaos Monkey रोज servers पाडतं, म्हणून Netflix ला **भाग पडतं** प्रत्येक भाग “एक server पडला तरी चालला पाहिजे” असा बांधायला (1.8 चं stateless + load balancer, कट्टरपणे). जो भाग एका server-मृत्यूने कोसळतो तो Chaos Monkey **लगेच** उघड करतो: गिन्हाईकाच्या आधी. अग्निशमन-दलाची कवायत रोज करण्यासारखं: खरी आग लागल्यावर गोंधळ नको.

हे Book 4 शी थेट जुळतं (red-teaming, 7.4): Book 4 त आपण agent वर स्वतः हल्ला करायला शिकलो (शत्रूच्या आधी फट सापडावी). Chaos Monkey तेच infrastructure साठी: **स्वतःच्या system वर स्वतः “हल्ला” (अपयश) करून, दुर्बल जागा शत्रू (खरा बिघाड) येण्याआधी शोधणं**. दोन्हीचं एक तत्त्व: टिकाव हा गृहीत धरून येत नाही; तो सतत तपासून, मुद्दाम ताण देऊन कमवावा लागतो. याला “chaos engineering” म्हणतात, आणि ती आता elite teams ची प्रौढ शिस्त आहे.

Netflix चे इतर धडे (नकाशा उलगडून): (1) प्रचंड microservices (2.6): पण त्यांच्याकडे ते **खरंच** लागतात (प्रचंड टीम, जगभर scale): fashion नाही, गरज. (2) CDN चा प्रचंड वापर (1.9: video जगभर, जवळून: Netflix च्या रहदारीचा मोठा भाग). (3) “एक भाग पडला तरी अनुभव चालू राहील” ची कट्टर रचना (graceful degradation): शिफारसी नाही आल्या तरी video चालतो. प्रत्येक निर्णय “अपयश होणारच” गृहीत धरून.

पूर्ण विभागाचा समान धागा (चार इमारती एकत्र): WhatsApp (सोपेपणा), UPI (खुला करार), Zerodha (कंटाळवाणं-भरवशाचं), Netflix (मुद्दाम-अपयश). वरवर वेगळे, पण एक सूर: **कुणीही “जादूई हुशारी” वर अवलंबून नाही; सगळे रचनेच्या शिस्तीवर, जाणीवपूर्वक सौद्यांवर, आणि अपयश-गृहीत-धरण्यावर उभे**. हेच पूर्ण Book 5 चं सार: **मोठं software हुशारीने नाही, शिस्तीने आणि सौद्यांच्या नजरेने बांधलं जातं.**

महाग चूक

”आमचं system चालतंय, त्याला हात लावायचा कशाला (मुद्दाम पाडणं तर दूरच).” “चालतंय” ही सर्वात धोकादायक शांतता: कारण तुम्हाला माहीत नाही ते **का** चालतंय, आणि कधी थांबेल. Netflix चा धडा: **जे तुम्ही तपासलं नाही ते तुम्हाला माहीत नाही**. अपयश-सज्जता “कधीतरी बघू” वर सोडली की ती नेमकी सर्वात वाईट क्षणी (खऱ्या बिघाडात) तपासली जाते: सर्वात महाग परीक्षा. मुद्दाम, नियंत्रित अपयश आजच घडवणं म्हणजे उद्याचा अनियंत्रित प्रलय टाळणं. शांततेवर विसंबू नका; ताण देऊन तपासा.

नकाशावर

विभाग 3 (शिस्त): git...incident
 4.1-4.3 WhatsApp (सोपेपणा) | UPI (खुला करार) | Zerodha (कंटाळवाणं)
 4.4 Netflix: Chaos Monkey (मुद्दाम अपयश) = red-teaming चा infra-भाऊ; टिकाव तपासून कमवायचा; graceful degradation; चार इमारतींचा सूर: शिस्त+सौदे > हुशारी

सूत्रावर: Netflix सूत्राला त्याच्या धाडसी टोकाला नेतं: टिकाव विकत घ्यायचा तर “आजची शांतता” विकानी लागते (मुद्दाम ताण देऊन). सर्वात प्रगल्भ सौदा: नियंत्रित वेदना विकत, अनियंत्रित प्रलय टाळून.

स्वतः बघा (5 मिनिटं)

अग्निशमन-कवायतीचा (fire drill) विचार करा: शाळा/office मध्ये मुद्दाम “आग लागली” म्हणून सराव. वेडेपणा वाटतो? नाही: खरी आग लागल्यावर गोंधळ नको म्हणून. Chaos Monkey तेच, servers साठी. आणि तुमच्या स्वतःच्या तयारीतही: महत्त्वाच्या गोष्टीचा backup **तपासून** बघितलात का, की “असेल” म्हणून सोडलं? (GitLab ची रात्र, 1.2.) मुद्दाम तपासणं ही प्रगल्भता.

विचार करा

चार इमारती बघून झाल्या, सगळी शिस्त कळली. आरतीला आता एक अस्वस्थ करणारा प्रश्न: “हे सगळं मी शिकले: रचना, सौदे, शिस्त, इमारती. पण Book 4 म्हणालं की आता **agents code लिहितात**. मग हे सगळं शिकण्याचा उपयोग काय? जर यंत्रच बांधणार, तर माझ्या या ज्ञानाला किंमत उरते का? की हे सगळं आता जुनं झालं?”

हा सर्वात महत्त्वाचा प्रश्न, आणि त्याचं उत्तर तुमचं भविष्य ठरवतं. उत्तर छोटं: agents code लिहितील, पण **रचना बरोबर आहे का, सौदे योग्य आहेत का** हे ठरवणारी नजर तुमचीच: आणि तीच आता सर्वात मौल्यवान. AI-युगाची रचना: पुढचा chapter.

Chapter 4.5 [SPINE]: AI-युगाची रचना: नजर हीच leverage

आरतीचा अस्वस्थ प्रश्न: agents code लिहितात, मग हे शिकण्याचा उपयोग काय? हा chapter त्याचं उत्तर आहे, आणि तो Book 4 शी थेट पूल बांधतो. उत्तर एका ओळीत: **code लिहिणं स्वस्त झालं; रचना बरोबर आहे का ठरवणारी नजर महाग राहिली: आणि तीच आता तुमची सर्वात मोठी संपत्ती.**

काय बदललं, काय नाही. Book 4 (9.2-9.3) चा leverage- सरकण्याचा धडा इथे software-रचनेत उतरतो. बदललं: **code टाईप करणं** (agents करतात, जलद-स्वस्त). **नाही** बदललं: “कुठली रचना, कुठला सौदा, कुठली database, हे बरोबर आहे का” हा निर्णय. आठवा (2.10): top engineer ची अर्धी किंमत नवं बांधण्यात, अर्धी **रचना वाचून सौदे-धोके ओळखण्यात**. आता बांधणं agent करतो; म्हणून **वाचणारी-ठरवणारी नजर** ही उरलेली, आणि म्हणून सर्वात मौल्यवान. Code = कच्चा माल (स्वस्त); नजर = मूल्य (Book 4, 9.1 चा थर 5-6).

agent-सोबत बांधण्याची नवी शिस्त (Book 4 + Book 5 एकत्र):

1. स्पष्ट करार आधी (Book 4, 4.7 spec-driven): agent ला “काय बांधायचं + कुठले सौदे” स्पष्ट सांगा. “किराणा-app बांध” नाही; “SQL वापर (पैसे आहेत), monolith ठेव (टीम छोटी), हे API-करार” : तुमची Book 5 ची नजर agent ला दिशा.
2. रचना तुम्ही ठरवा, code agent लिहो: database-निवड (2.2), monolith/micro (2.6), CAP (1.10): हे निर्णय तुमचे; अंमलबजावणी agent ची.
3. नकाशा वाचा (2.10): agent ने बांधलेलं वाचून सौदे-धोके तपासा: “इथे cache आहे, ‘जुनं कधी’ विचारलं का? इथे थेट database-हात आहे का (2.5 चा भंग)?”
4. Book 4 ची सगळी शिस्त लागू: evals (test), brakes, review (agent चं code ही reviewer ला/तुम्हाला दाखवा), observability.

म्हणजे Book 5 शिकल्याने काय मिळालं: agent ला दिशा देण्याची आणि त्याचं काम तपासण्याची नजर. Agent ला system-design कळत नाही असं नाही; पण **तुमच्या** परिस्थितीला कुठला सौदा योग्य (छोटी टीम? पैसे? किती गर्दी?) हे **तुम्ही** ठरवता: agent अंमलात आणतो. रचना न कळणारा माणूस agent ला “किराणा-app बांध” म्हणतो आणि agent जे देईल ते घेतो (बहुधा over-engineered, चुकीचे सौदे); रचना कळणारा माणूस agent ला नेमकी दिशा देतो आणि त्याचं काम तपासतो. **तोच फरक थर-4 (wrapper) आणि थर-5-6 (खंदक) मधला** (Book 4, 9.1).

पूर्ण दोन पुस्तकांचं लग्न: Book 4 ने “AI कसं वापरून बांधायचं” दिलं (harness); Book 5 ने “काय बांधायचं, कुठली रचना” दिलं (blueprint). एकत्र: **तुम्ही agent ला योग्य गोष्ट, योग्य रचनेत, सुरक्षितपणे बांधायला लावू शकता: आणि त्याचं काम तपासू शकता.** हीच 2026 ची सर्वात मौल्यवान जोडी: आणि आता ती तुमच्याकडे आहे.

महाग चूक

”AI सगळं बांधतं, आता system-design शिकायची गरज नाही.” सर्वात महाग गैरसमज. उलट: **जितकं बांधणं स्वस्त, तितकी ‘काय-आणि-कशी बांधायची’ ची नजर मौल्यवान.** Agent ला रचना कळते, पण तुमच्या संदर्भातले सौदे (टीम-आकार, पैसा, गर्दी, data-स्वभाव) तुम्हीच द्यायचे; आणि त्याचं काम तपासायची नजरही तुमचीच. रचना न कळणारा agent-वापरकर्ता म्हणजे आंधळा माणूस driverless गाडीत: गाडी चालवेल, पण **कुठे जायचं** माहीत नसेल तर चुकीच्या जागी वेगात पोहोचेल. नजर हीच steering.

नकाशावर

विभाग 3-4: शिस्त + चार इमारती

4.5 AI-युगाची रचना: code स्वस्त, नजर महाग; agent ला दिशा (तुमचे सौदे) + तपासणी (नकाशा वाचा) तुमची; Book 4 (कसं) + Book 5 (काय) = मौल्यवान जोडी

सूत्रावर: शेवटचा, सर्वात मोठा सौदा-उलगाडा: जेव्हा बांधणं फुकट होतं, तेव्हा **काय बांधायचं आणि कुठला सौदा** हा निर्णय सर्व मूल्याचा होतो. तुमची रचना-नजर हीच तुमची न-नकलता-येणारी संपत्ती.

स्वतः बघा (5 मिनिटं)

कुठल्याही AI ला “एक किराणा-app ची रचना सुचव” विचारा. उत्तर वाचा आणि 2.10 ची सहा-प्रश्नांची नजर लावा: त्याने कुठली database सुचवली, का? Monolith की micro? Cache कुठे? आणि विचारा: “माझी टीम 2 माणसांची आहे, हे बदलतं का?” बघा उत्तर कसं बदलतं. तुम्ही agent ला **दिशा देताय आणि तपासताय:** हीच AI-युगातली तुमची भूमिका, प्रत्यक्ष.

विचार करा

Book 5 संपत आलय. आरतीचा प्रवास पूर्ण: एक laptop पासून जगभर पसरलेल्या system पर्यंत, आणि आता agents ला दिशा देणाऱ्या नजरेपर्यंत. शेवटचा प्रश्न, जो पुस्तकाचं सगळं एका परीक्षेत बांधेल: “मी हे सगळं शिकले: नऊ सौदे, मोठे निर्णय, elite शिस्त, खऱ्या इमारती, AI-युगाची नजर. पण हे खरंच **मुरलं** का? की फक्त वाचून गेलं? हे मला कसं कळणार?”

तेच अंतिम परीक्षा ठरवेल: दोन पुस्तकांच्या (5, आणि Book 4) रचना-नजरेची. आणि मग पाच checklists, जे तुम्ही आयुष्यभर टेबलावर ठेवाल. Book 5 चा शेवट, पुढे.

Chapter 4.6 [SPINE]: अंतिम परीक्षा

नियम आधीसारखेच: पुस्तक न उघडता, **बोलून** उत्तर द्या; गाभा खाली, आधी स्वतःचं बांधा मग ताडा. हेतू गुण नाही: **रचना-नजर मुरली का** हे बघणं.

भाग अ: सौदे मुरले का (5 प्रश्न)

1. App हळू झालं. तुमचं पहिलं पाऊल, आणि “मोठा server घेऊ” का नाही?

रचना तपासा: गहाळ index (1.3)? गहाळ cache (1.4)? वाचनं replicas कडे वळवली का (1.5)? 90% हळूपणा रचनेत, ताकदीत नाही. पैसे ओतणं शेवटचा उपाय.

2. “आमचं नवं app सगळं NoSQL + microservices वर!” : तुमचे प्रश्न?

डेटाला नाती-शुद्धता लागते का (मग SQL: 2.2)? टीम एवढी मोठी आहे का की microservices चं टीम-स्वातंत्र्य खरंच लागतं (2.6)? की हे fashion आहे? Amazon उलट गेलं (-90%).

3. पैसे-व्यवहाराला CAP मध्ये C की A; आणि queue वापरली तर आणखी काय हवं?

C (फूट पडली तर थांबा, पण अचूक: 1.10). Queue सोबत idempotency हवीच (दोनदा-कापले-जाणं टाळायला: 1.7).

4. चालत्या, लाखो-नोंदींच्या database ची रचना बदलायची: पद्धत?

विस्तार-आकुंचन: नवं जोडा (जुनं ठेवून) -> दुहेरी-लेखन -> वळण -> जुनं काढा; छोटी उलटवता-येणारी पावलं; backup तपासलेला आधी (2.9, GitLab).

5. System निरोगी आहे हे कशाने ठरवाल, सरासरीने का नाही?

टोक (p99): सरासरी अल्पसंख्य-दुःखी लपवते, आणि तोच गिन्हाईक धंदा ठरवतो (1.12). आणि दिसण्यासाठी logs+traces (5.4-Book4/1.12).

भाग ब: शिस्त, इमारती, AI-युग (5 प्रश्न)

6. चूक झाली. “कोणी केलं” का विचारू नये?

दोष-संस्कृती लोकांना चुका लपवायला लावते; blameless postmortem विचारतो “रचनेत काय फट, कशी बुजवू” (3.4). चांगली माणसं वाईट रचनेत चुकतात.

7. WhatsApp/Zerodha चा समान धडा एका ओळीत?

मोठेपणा हुशारीने नाही, **सोपेपणा + योग्य निवडी + जाणीवपूर्वक सौदे** यांनी: 50 माणसं/90कोटी, ~30 लोक/सर्वात मोठा broker (4.1, 4.3). कंटाळवाणं-सोपं जिकतं.

8. Netflix मुद्दाम servers का पाडतं, आणि हे Book 4 शी कसं जुळतं?

अपयश होणारच; ते आपल्या-वेळी-देखरेखीखाली घडवून दुर्बल जागा आधी शोधणं (chaos engineering: 4.4). हे red-teaming (Book 4, 7.4) चा infra-भाऊ: टिकाव तपासून कमवायचा.

9. Deploy सुरक्षित करायच्या तीन युक्ती, आणि CrowdStrike काय शिकवतं?

Canary (थोड्यांना आधी) + blue-green + feature flag (3.3). CrowdStrike ने 85 लाख यंत्रांना एकदम सदोष update दिलं: canary त्यांना वाचवता आला असता (1.13).

10. Agents code लिहितात, मग system-design शिकण्याचा उपयोग काय?

Code स्वस्त झालं; **रचना बरोबर आहे का, कुठला सौदा योग्य** ही नजर महाग राहिली (4.5). तुम्ही agent ला दिशा (तुमचे सौदे) देता आणि त्याचं काम तपासता (नकाशा वाचा). नजर = leverage (Book 4, 9.2).

आणि आता **खरी** अंतिम परीक्षा: वरचे दहा “जमले का” नाही: **खालचा एक**, ज्याचं उत्तर पुस्तकात नाही.

पूर्ण Book 5 एका वाक्यात बांधता येतं. आधी स्वतः लिहा, मग खाली बघा:

”सर्वात चांगलं software असं काही नसतं; फक्त **या गर्दीला, या माहितीला, या टीमला, या क्षणी योग्य** असतं: आणि प्रत्येक रचना-निर्णय म्हणजे काहीतरी विकून काहीतरी घेणं. जो हा सौदा **जाणून** करतो तो engineer; जो fashion बघतो तो दोनदा बांधतो. आणि agents च्या युगात, हा सौदा जाणण्याची नजर हीच माणसाची उरलेली, आणि सर्वात मौल्यवान, संपत्ती आहे.”

चार इमारती हेच जगल्या: WhatsApp चा सोपेपणा, UPI चा खुला करार, Zerodha चं कंटाळवाणं भरवसा, Netflix चं मुद्दाम-अपयश: कुणीही “जादूई हुशारी” वर नाही, सगळे **सौद्यांच्या नजरेवर** उभे.

शेवटचं “स्वतः बघा” (आयुष्यभराचं)

आजपासून, कुठलंही software, कुठलीही रचना, कुठलाही “हे tech वापरू” चा प्रस्ताव: तीन प्रश्न विचारायची सवय लावा: “**इथे कुठला सौदा केलाय: काय विकून काय घेतलं?**” (Book 5), “**हा सौदा या परिस्थितीला योग्य आहे का, की fashion आहे?**” (Book 5), आणि “**हे मी agent ला बांधायला लावलं, तर दिशा आणि तपासणी माझी असेल का?**” (Book 4 + 5). हे तीन प्रश्न तुम्हाला रचना वापरणाऱ्यांकडून रचना ठरवणाऱ्यांकडे नेतील: आणि तिथेच पैसा, आणि स्वातंत्र्य, आहे.

Chapter 4.7 [SPINE]: पाच हात-नकाशे (टेबलावर चिकटवा)

वाचायचं नाही, **वापरायचं**. पाच एक-पानी याद्या: रचना ठरवताना, database निवडताना, deploy करताना, नकाशा वाचताना, आणि agent-सोबत बांधताना समोर ठेवायच्या.

नकाशा 1: नवं system: रचना-निर्णय (सुरुवातीला)

- [] सुरुवात सोपी: एक monolith, एक database; गरजेआधी तुकडे/scale नको (premature टाळा) (1.1, 2.6)
- [] डेटा: नाती+शुद्धता हवी? -> SQL (डिफॉल्ट). प्रचंड-सैल-वेगवान? -> NoSQL. शंका -> SQL (2.2)
- [] नाती आधी कागदावर (one-to-many, many-to-many) (2.1)
- [] कुठली माहिती किती जुनी चालेल? (cache/CAP निर्णय) (1.4, 1.10)
- [] सेवा वाढवल्या तर: फक्त API ने बोलतील, थेट DB-हात नाही (2.5)

नकाशा 2: Scale ची भिंत आली (क्रम पाळा)

- [] आधी: index (जिथे शोध) + cache (वारंवारचं) (1.3, 1.4)
- [] मग: replicas कडे वाचनं वळवा (ताजं-हवं मुख्याकडे) (1.5)
- [] मग: load balancer + अनेक stateless servers (1.8)
- [] जड कामं queue त (idempotent!) (1.7)
- [] शेवटी, नाईलाजाने: sharding (लेखनच आवाक्याबाहेर तर) (1.6)
- [] "मोठा server" (vertical) हा शेवटचा/तात्पुरता उपाय

नकाशा 3: अपयश-सज्जता + deploy (production आधी)

- [] प्रत्येक बाहेरची विनंती: timeout (वाट अनंत नाही) (1.11)
- [] retry: थांबत (backoff) + मोजून + idempotent (1.11)
- [] circuit breaker: आजारी दुवा वेगळा (1.11)
- [] deploy: canary (थोड्यांना आधी) + परत-फिरण्याचा रस्ता (3.3)
- [] छोटे-वारंवार deploys > मोठे-कचित (सुरक्षित) (3.3)
- [] backup आहे + तो तपासलेला आहे (GitLab!) (1.2, 2.9)

नकाशा 4: नकाशा वाचणं (कुठलीही रचना, 2 मिनिटांत)

- [] एका विनंतीचा पूर्ण रस्ता काढा (कुठून कुठे) (1.15)
- [] एकच अपयश-बिंदू कुठे? (एक पडलं तर सगळं थांबतं) (1.8)
- [] database: पैसे SQL त? सैल NoSQL त? की सगळं कोंबलं? (2.2)
- [] cache कुठे? "जुनं कधी फेकायचं" विचारलंय का? (1.4)
- [] सेवा-संवाद: API ने की थेट-DB-हात (धोका)? (2.5)
- [] गर्दी/अपयश कुठे फुटेल? (queue? LB? timeout?) (1.11)
- [] प्रत्येक पेटी "का आहे"? उत्तर नसेल तर fashion (2.10)

नकाशा 5: Agent-सोबत बांधणं (AI-युग)

- [] रचना-निर्णय मी घेतो (SQL/NoSQL, mono/micro, CAP):
agent अंमलात आणतो (4.5)
- [] स्पष्ट करार आधी: काय + कुठले सौदे (Book 4, spec) (4.5)
- [] agent चं code: नकाशा-वाचन (नकाशा 4) लावून तपासा
- [] Book 4 ची शिस्त: test/evals + brakes + review + observ.
- [] over-engineering तपासा: agent ला "सोपं ठेव" सांगा
(WhatsApp/Zerodha धडा) (4.1, 4.3)

पाचही नकाशांमागचं एक वाक्य: प्रत्येक निर्णयाला एकच प्रश्न: "इथे कुठला सौदा, आणि तो या परिस्थितीला योग्य आहे का?" Fashion ला नाही, सौद्याला हो. हीच पूर्ण Book 5 ची, आणि top engineer ची, एकमेव नजर.

MASTER GLOSSARY: Book 5 चे सगळे रचना-शब्द

(शब्द -> एक ओळ -> chapter.)

ACID	database ची शिस्त: अखंड-सुसंगत-एकांत-टिकाऊ (1.2)
API	दोन software मधला करार (menu-तत्त्व) (2.5)
B-tree	वाचन-जलद database-रचना (SQL) (2.3)
blameless	चूक -> दोष नको, रचना का फसली (3.4)
blue-green	जुनं चालू ठेवून नव्याकडे सुरक्षित वळण (3.3)
cache	वारंवारचं उत्तर जलद जागी; ताजेपणा विकून (1.4)
canary	नवं code आधी थोड्यांना (CrowdStrike-धडा) (3.3)
CAP	फूट-क्षणी C की A; तीनातले दोनच (1.10)
CDN	गि-हार्डकाजवळ static ची प्रत; वेग+bill (1.9)
chaos engineering	मुद्दाम अपयश घडवून टिकाव तपासणं (4.4)
CI/CD	आपोआप build+test+deploy साखळी (3.3)
circuit breaker	आजारी दुवा वेगळा काढणारा fuse (1.11)
code review	दुसऱ्याची मोहोर; दर्जा+ज्ञान-प्रसार (3.1)
deadlock	दोन व्यवहार एकमेकांची वाट बघत अडकणं (2.4)
egress	cloud मधून data बाहेर काढण्याचं (महाग) भाडं (1.14)
feature flag	नवं feature स्विचवर; बंद/चालू करता येतं (3.3)
git	इतिहास-यंत्र + branches + review (3.1)
horizontal scale	अनेक स्वस्त यंत्रं > एक महाग (1.8)
idempotency	दोनदा केलं तरी परिणाम एकदाच (1.7)
index	database ची अनुक्रमणिका; वाचन जलद (1.3)
inverted index	शब्द -> वस्तू; खऱ्या search चा पाया (2.7)
load balancer	विनंत्या मोकळ्या server कडे वळवणारा (1.8)
LSM-tree	लेखन-जलद database-रचना (NoSQL) (2.3)
migration	चालत्या database ची रचना सुरक्षित बदलणं (2.9)
monolith	पूर्ण app एकाच code मध्ये (2.6)
microservices	app चे अनेक स्वतंत्र सेवा-तुकडे (2.6)
normalization	एक सत्य एका जागी; पुनरावृत्ती टाळणं (2.1)
p99	टोकाचं मोजमाप; सरासरीची फसवणूक टाळते (1.12)
partition	अनेक यंत्रांमधला संपर्क तुटणं (1.10)
queue	जड कामं रांगेत; धक्का शोषतो (1.7)
race condition	एकाच वेळी दोघांच्या हातामुळे अनिश्चित निकाल (2.4)
replication	database ची प्रत; lag = किती जुनी (1.5)
sharding	database चे यंत्रांवर तुकडे; लेखन वाटलं (1.6)
SQL/relational	शिस्त+नाती+ACID database (2.2)
stateless	server काहीही स्वतःकडे साठवत नाही (1.8)
testing	unit/integration/e2e थर; घसरण-रोध (3.2)
timeout	वाट किती बघायची, मर्यादा (1.11)
transaction	अनेक पायऱ्या एक अभंग गड्डा (2.4)
vertical scale	तेच यंत्र अधिक शक्तिशाली करणं (1.8)
WebSocket	सतत-उघडा दुवा; real-time साठी (2.8)

पाचवं पुस्तक संपलं. Machine जादू नाही (1); AI जादू नाही (2); जमीन जादू नाही (3); harness जादू नाही (4); आणि मोठं software जादू नाही (5): **सगळं सौदे आणि शिस्त आहे, आणि दोन्ही शिकता येतात.**

पाच पुस्तकांचा एकच सूर: **जादू शोधू नका; सौदे बघा.** गरज (1), यंत्र (1-2), विश्वास (4), रचना (5): प्रत्येक ठिकाणी तेच: काहीतरी विकून काहीतरी घ्यायचं, आणि जो तो सौदा **जाणून** करतो तोच जिंकतो.

आणि AI-युगात, जेव्हा बांधणं फुकट होतं, तेव्हा उरतं फक्त एक: **कुठला सौदा योग्य हे ठरवणारी नजर.** ती नजर आता तुमच्याकडे आहे. सुरू करा.